



JOI 2025/2026 ファイナルステージ Day1 伝説の団子食通 (Legendary Dango Eater) 解説

解説: 蜂矢 倫久 (Mitsubachi)

Step 0

問題概要

0

問題概要

- 味が甘いか辛いかのどちらかである団子の列があります
- ランレングス圧縮した形で与えられます
- 団子の区間を食べることを考えます
- いくつかの連続部分列に区切ります
- 連続部分列で、(甘い味の数) - (辛い味の数) $\geq K$ を満たす個数を最大化したいです
- Q 通りの区間について解いてください

0

制約

- $N \leq 500\,000$ (ランレングス圧縮後の区間数)
- $Q \leq 500\,000$
- $K \leq 10^9$
- $A_i \leq 10^9$ (各区間の個数)

小課題番号	N	Q	K	A	配点
1		10			6 点
2			2		5 点
3			10		18 点
4				総和 500 000	27 点
5	200 000	200 000			17 点
6					27 点

Subtask 1

$$Q \leq 10$$

1 条件の言い換え

- 甘い味の団子を $+1$, 辛い味の団子を -1 とみなす
- (甘い味の数) - (辛い味の数) は単に総和となる
- 以降この設定で考える
- 実装上は $A_2, A_4, \dots$ を -1 倍するなどに対応できる

1

最適な操作

- まず最適な区切り方について考える
- 総和が K になったら切っていい
- 0 になっても切っていい
- これでシュミレーションができる形に
- クエリあたり $O(NA)$ にまずなる $\rightarrow$ 時間制限に間に合わない

1

最適な操作

- まず最適な区切り方について考える
- 総和が K になったら切っていい
- 0 になっても切っていい
- これでシュミレーションができる形に
- クエリあたり $O(NA)$ にまずなる $\rightarrow$ 時間制限に間に合わない
- よく考えるとランレングス圧縮した後の区間 1 つ分は同時に処理できる

1

シュミレーション

- 総和が K になったら切っていい
- 0 になっても切っていい
- よく考えるとランレングス圧縮した後の区間 1 つ分は同時に処理できる
- $ans = 0, rem = 0$
- for a in A :
 - $rem \leftarrow \max(0, rem + a)$
 - $ans \leftarrow ans + rem / K$
 - $rem \leftarrow rem \% K$

1

シュミレーション

- 総和が K になったら切っていい
- 0 になっても切っていい
- よく考えるとランレングス圧縮した後の区間 1 つ分は同時に処理できる
- これでクエリあたり $O(N)$ になった
- $O(NQ)$ で解けました

1

今後について

- このシミュレーションを考えると、 $A_1, A_3, \dots$ については K で割ったあまりに変えていいことが分かる
- 商の分は後で累積和として計算すれば良い
- $A_1, A_3, \dots$ は K 未満と帰着することができる
- 同様に考えると $A_2, A_4, \dots$ も K 以下とできる
- 以降この形で考えます

Subtask 2

$$K \leq 2$$

2

考察

- $K = 1$ と $K = 2$ が解ければ良い
- $K = 1$: 甘い味の数を数えれば良い
- 累積和を使えば $O(N + Q)$
- $K = 2$ を解く 先ほどのシュミレーションを思い返す
- $\text{ans} = 0, \text{rem} = 0$
- for a in A :
 - $\text{rem} \leftarrow \max(0, \text{rem} + a)$
 - $\text{ans} \leftarrow \text{ans} + \text{rem} / K$
 - $\text{rem} \leftarrow \text{rem} \% K$

2 $K = 2$

- $K = 2$ を解く 先ほどのシュミレーションを思い返す
- $\text{ans} = 0, \text{rem} = 0$
- for a in A :
 - $\text{rem} \leftarrow \max(0, \text{rem} + a)$
 - $\text{ans} \leftarrow \text{ans} + \text{rem} / K$
 - $\text{rem} \leftarrow \text{rem} \% K$
- 負の a を処理する前の rem は 0 か 1
- つまり負の a を処理した後の rem は必ず 0

2

解法

- $K = 2$ を解く 先ほどのシュミレーションを思い返す
- よって辛い味のものを跨ぐ必要はない
- したがって、甘い味の a について、 $a / 2$ の総和を取れば OK
- これも累積和で $O(N + Q)$
- 以上より、どちらの K でも $O(N + Q)$ で解けました

Subtask 3

$$K \leq 10$$

3 シュミレーションの復習

- シュミレーションを振り返る
- $ans = 0, rem = 0$
- for a in A :
 - $rem \leftarrow \max(0, rem + a)$
 - $ans \leftarrow ans + rem / K$
 - $rem \leftarrow rem \% K$
- この初期値の ans が 0 ではなく x であると最終結果の ans はどう変わる？
- 単に x 足すだけ

3 シュミレーションの復習

- シュミレーションを振り返る
- つまり、大事なものは rem の値
- ある区間について $i = 0, 1, \dots, K - 1$ について, (ans, rem) の初期値が $(0, i)$ としてシュミレーションを走らせると (ans, rem) は最終的にどうなるかを求めることを考える
- これは Segment Tree に乗る形になる
- ノードのマージも $O(K)$ でできる

3

セグメント木

- したがって、 $O((N + Q \log N) K)$ などで解くことができる
- 関数合成を Segment Tree に乗せるという考え方が近いと思う
- ういろう(Uiro) の小課題 5 などが同じ考え方

3 休憩用スライド

- 話疲れてそうなのでちょっと休憩
- Dango in USA の写真を貼る



Subtask 4

A の総和 $\leq 500\ 000$

4

前準備

- ランレングス圧縮してあるのをバラす
- Sample 1 なら 2, 1, 2, 4, 3 なので
- +1, +1, -1, +1, +1, -1, -1, -1, -1, +1, +1, +1 という形
- これにおける区間クエリが解ければ良い
- A の総和を S とする

4

シュミレーションの復習

- シュミレーションを振り返る
- $ans = 0, rem = 0$
- for a in A:
 - $rem \leftarrow \max(0, rem + a)$
 - $ans \leftarrow ans + rem / K$
 - $rem \leftarrow rem \% K$
- この初期値の ans が 0 ではなく x となると最終結果の ans はどう変わる？
- 単に x 足すだけ

4

シュミレーションの復習

- シュミレーションを振り返る
- 最初に rem が 0 になるところが分かったとする
- つまり、そこで区切りますよという場所
- すると、 l からスタートして r に行くまで何回区切りますかと見ることができる

4

シュミレーションの復習

- シュミレーションを振り返る
- 最初に rem が 0 になるところが分かったとする
- つまり、そこで区切りますよという場所
- すると、1 からスタートして r に行くまで何回区切りますかと見ることができる
- これは**ダブリング**！

4

ダブリング

- シミュレーションを振り返る
- 最初に rem が 0 になるところを求めよう
- シミュレーションの動きを考えると最初に 0 以下 or K 以上に到達する場所であるとみなせる
- これは prefix の $[\text{min}, \text{max}]$ が $[1, K - 1]$ をはみ出すのはいつかということになる
- Segment Tree で二分探索すれば全体で $O(S \log S)$

4

具体例

- Sample 2 の $K = 3$, $A = (2, 1, 2, 4, 3)$ で考える
- $+1, +1, -1, +1, +1, -1, -1, -1, -1, +1, +1, +1$ にバラす
- 左から始めると
- $+1, +1, -1, +1, +1 / -1 / -1 / -1 / -1 / +1, +1, +1$
- という感じに
- よって答えは 2 ですねということになる
- 実装上は -1 からスタートする区間は飛ばすと楽だと思う

4

ダブリング

- シミュレーションを振り返る
- 最初に rem が 0 になるところが $O(S \log S)$ で求まった
- あとはこれでダブリングをすれば良い 前準備は $O(S \log S)$
- クエリあたりは $O(\log S)$ になる
- 全体で $O((S + Q) \log S)$ で解けました

4

おまけ

- 小課題 1 で説明した通り $A_i \leq K$ とできるので, これをすることで $O((NK + Q) \log (NK))$ など解くことができる
- 小課題 2 はいけそう
- 小課題 3 はギリギリな気がする メモリ制限の話もありそう

Subtask 6

追加制約なし

6

小課題 5 と 6 の話

- 小課題 5 と 6 は 2.5 倍ぐらいの差なので、定数倍が悪かったりどこかで余計な $\log$ や $\sqrt{\quad}$ がついていると小課題 5 までしか通らないと思います
- ここでは $A_2, A_4, \dots$ を -1 倍するのは一旦やめたとします

6

ダブリングの思考

- 先ほどのダブリングを A_i が大きくても使えるようにしたい
- A_i を足して $\text{mod } K$ をして, $A_{\{i+1\}}$ を引いて, ... を繰り返して最初に 0 以下になるところを求めたとする (A_{pos} とする)
- その間でいくつ取ることができるかを考える
- $[A_i, A_{\text{pos}})$ の区間で何個作れますかという話になる
- これは A_i から $A_{\{\text{pos} - 1\}}$ までの総和を K で割った商になる
- 貪欲をすれば良い

6

具体例

- Sample 2 の $K = 3$, $A = (2, 1, 2, 4, 3)$ で考える
- A_1 からスタートする
- $+2 \rightarrow +1 \rightarrow +3 \pmod{3}$ をして 0 に) $\rightarrow -4$ となる
- よって A_1 からスタートしたら $[A_1, A_3]$ の間はこれの総和を K で割った 1 個取れると言える, 加えて A_4 以降(実際には A_4 は負の部分なので A_5 以降)と独立して考えられる

6

条件整理

- 結局のところ A_i が大きくなっても最初に 0 以下になる場所を探したいということになる
- これができればあとはダブリング
- ただ難しいのが, $\text{mod } K$ を取ったりするところ
- 条件の形としては, 各 i について, $(A_i - A_{i+1} + A_{i+2} - \dots + A_{j-1}) \text{ mod } K \leq A_j$ なる最小の j を知りたいことに

- 結局のところ A_i が大きくなっても最初に 0 以下になる場所を探したいということになる
- これができればあとはダブリング
- 愚直にやると $O(N^2)$ になってしまう
- 実は $O(NK)$ での評価は可能なので小課題 2, 3 はこれでも通る
- 高速に求める方法を紹介

6

式変形

- 各 i について, $(A_i - A_{i+1} + A_{i+2} - \dots + A_{j-1}) \bmod K \leq A_j$ なる最小の j を知りたい
- A_i の交互和の累積和 $s_1, s_2, \dots$, を考えると上の式は以下に
- $(s_{j-1} - s_{i-1}) \bmod K \leq A_j$
- $j \geq x$ かどうかというのは $(s_{x-1} - s_{i-1}) \bmod K \leq A_x$ が $x = i, i+1, \dots, j$ で成り立つかということになる

6

式変形

- 各 i について, $(A_i - A_{i+1} + A_{i+2} - \dots + A_{j-1}) \bmod K \leq A_j$ なる最小の j を知りたい
- $j \geq x$ かというのは $(s_{x-1} - s_{i-1}) \bmod K \leq A_x$ が $x = i, i+1, \dots, j$ で成り立つかということになる
- i が大きい順に j を求めることにする
- すると, s_{i-1} が $\bmod K$ でこの区間にあるような i については j はこの値以下みたいな制約が増えていく形になる
- 座圧をすれば 区間更新 / 一点取得になる

6

2つの方針

- i が大きい順に j を求めることにする
- すると, $s_{\{i-1\}}$ が $\text{mod } K$ でこの区間にあるような i については j はこの値以下みたいな制約が増えていく形になる
- $(s_{\{x-1\}} - s_{\{i-1\}}) \text{ mod } K \leq A_x$ 由来の制約を制約 x とする
- 座圧をすれば 区間更新 / 一点取得になる
- 2つの方針がある
 - ① set で頑張る方法
 - ② データ構造を書く方法

6

① set で頑張る

- まだ j が未定の (i, s_i) の集合を管理しておく
- $(s_{\{x-1\}} - s_{\{i-1\}}) \bmod K \leq A_x$ 由来の制約を制約 x とした
- これを $x = 1, 2, \dots$ の順に追加していく
- (i, s_i) もこの順に追加していく
- 制約 x を追加したとき, それで確定する i (つまり, $j = x$ で初めて制約 x に違反して $j = i$ となる i) は set の `lower_bound` などで高速に取得できる
- $O(N \log N)$ などできる

6

② データ構造を書く

- 区間更新 / 一点取得の形になった
- これは x が大きい方から制約 x を足していく
- まず Lazy Segment Tree で解ける
- $O(N \log N)$
- でももっと楽に書ける
- オーダーは同じだけど定数倍がよくなる

6

② データ構造を書く

- 区間更新 / 一点取得の形になった
- これは x が大きい方から制約 x を足していく
- まず Lazy Segment Tree で解ける
- $O(N \log N)$
- でももっと楽に書ける
- オーダーは同じだけど定数倍がよくなる
- その名は **Dual Segment Tree** (双対セグメント木) !

6

② データ構造を書く

- **Dual Segment Tree** とは？
- チューターの tatyam さんの記事を見るととても参考に
- <https://hackmd.io/@tatyam-prime/DualSegmentTree>
- ザックリいうと，Segment Tree の一点更新と区間の prod 取得を逆にしたよということ

② データ構造を書く

- $N = K = 8$ で具体例を説明
- 下は Segment Tree でよくある図 (0-indexed)
- まだ何の制約もないという状態

[illegible]

6

② データ構造を書く

- mod K で $[2, 5)$ なら $7 (= N - 1)$ で引っ掛かりますという制約を追加
- 普通の Segment Tree で $[2, 5)$ の区間クエリを求める感じ

N							
N				N			
N		7		N		N	
N	N	N	N	7	N	N	N

6

② データ構造を書く

- mod K で $[3, 7)$ なら $6 (= N - 2)$ で引っ掛かりますという制約を追加
- 普通の Segment Tree で $[3, 7)$ の区間クエリを求める感じ

N							
N				N			
N		7		6		N	
N	N	N	6	7	N	6	N

6

② データ構造を書く

- mod K で 4 なら今いくつかを知りたい
- 普通の Segment Tree で 4 の一点更新をする感じでできる！

N							
N				N			
N		7		6		N	
N	N	N	6	7	N	6	N

6

② データ構造を書く

- これが操作あたり $O(\log K)$ なのは Segment Tree の計算量を知っていれば理解できると思う
- 今回は K が大きいので座圧が必要
- 座圧したらサイズは N になる
- よって操作あたり $O(\log N)$ になる
- 全体で $O(N \log N)$

6

解法

- とりあえず $O(N \log N)$ で一手先が分かった
- 別の方法として平方分割がありうるが、小課題 5 までかも
- あとはダブリングすれば OK
- 全体で $O((N + Q) \log N)$ となる

Step 7

得点分布

7

得点分布

- 理想:

点数	1	2	3	4	5	6	人数	累計
100	0	0	0	0	0	0	30	30

- 選抜という意味では差がつかないな

7

得点分布

- 現実:

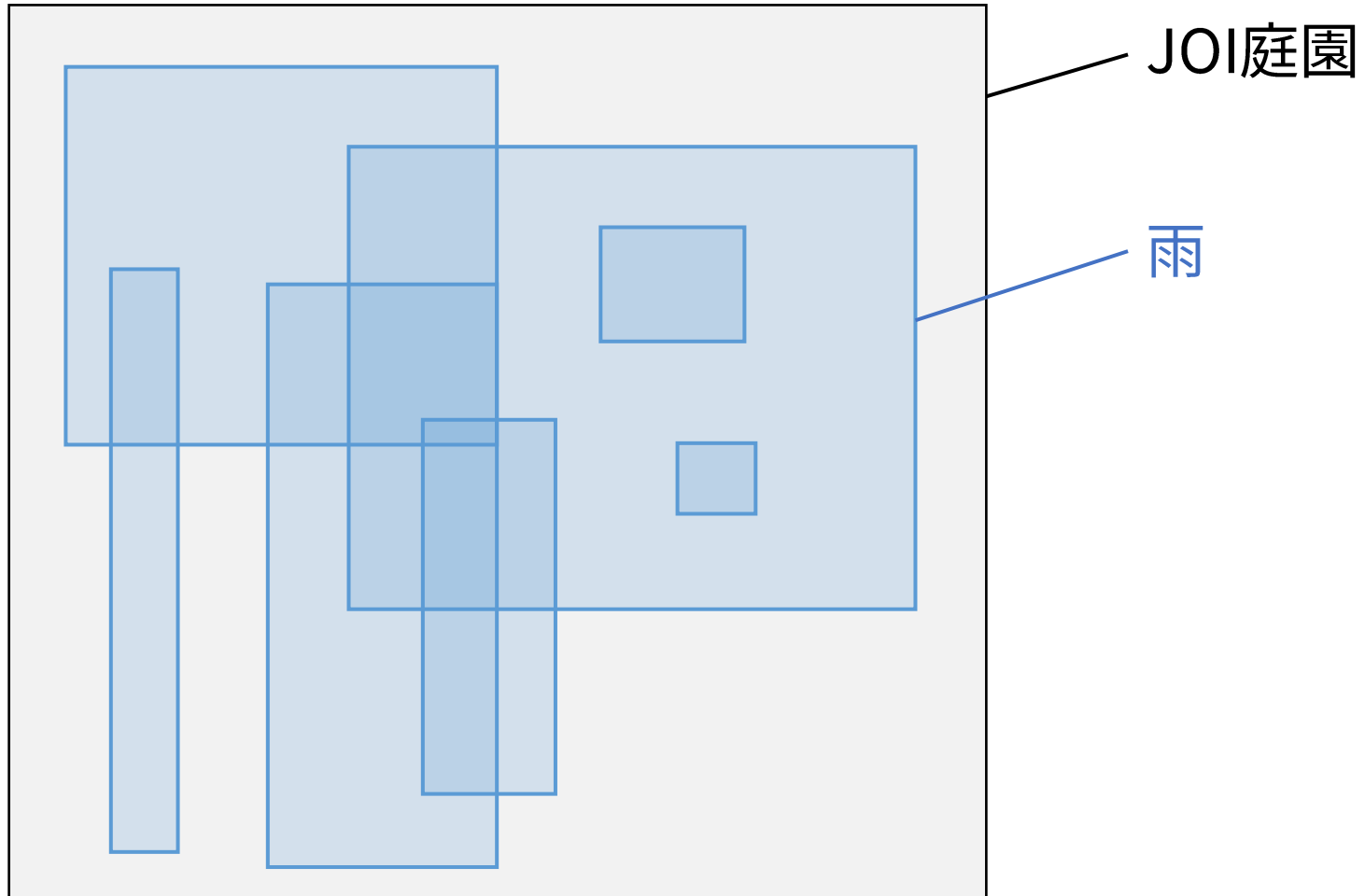
点数	1	2	3	4	5	6	人数	累計
100	0	0	0	0	0	0	4	4
56	0	0	0	0			2	6
38	0	0		0			2	8
29	0	0	0				7	15
27				0			1	16
～ 11	こ	の	辺				14	30

庭園 3 (garden) 解説

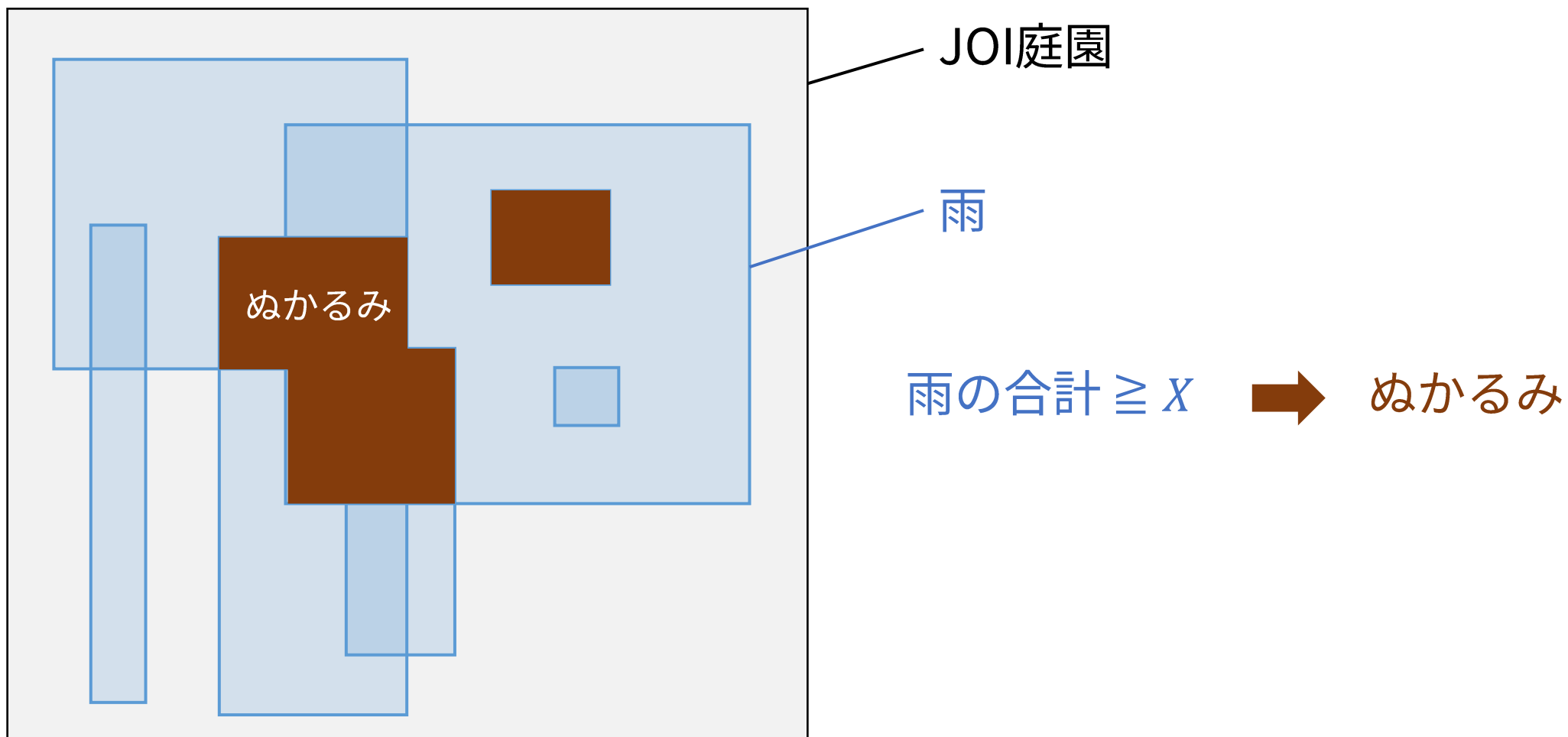
JOI 2025/26 ファイナルステージ 1 日目

解説担当 : 田村唯(Nachia)

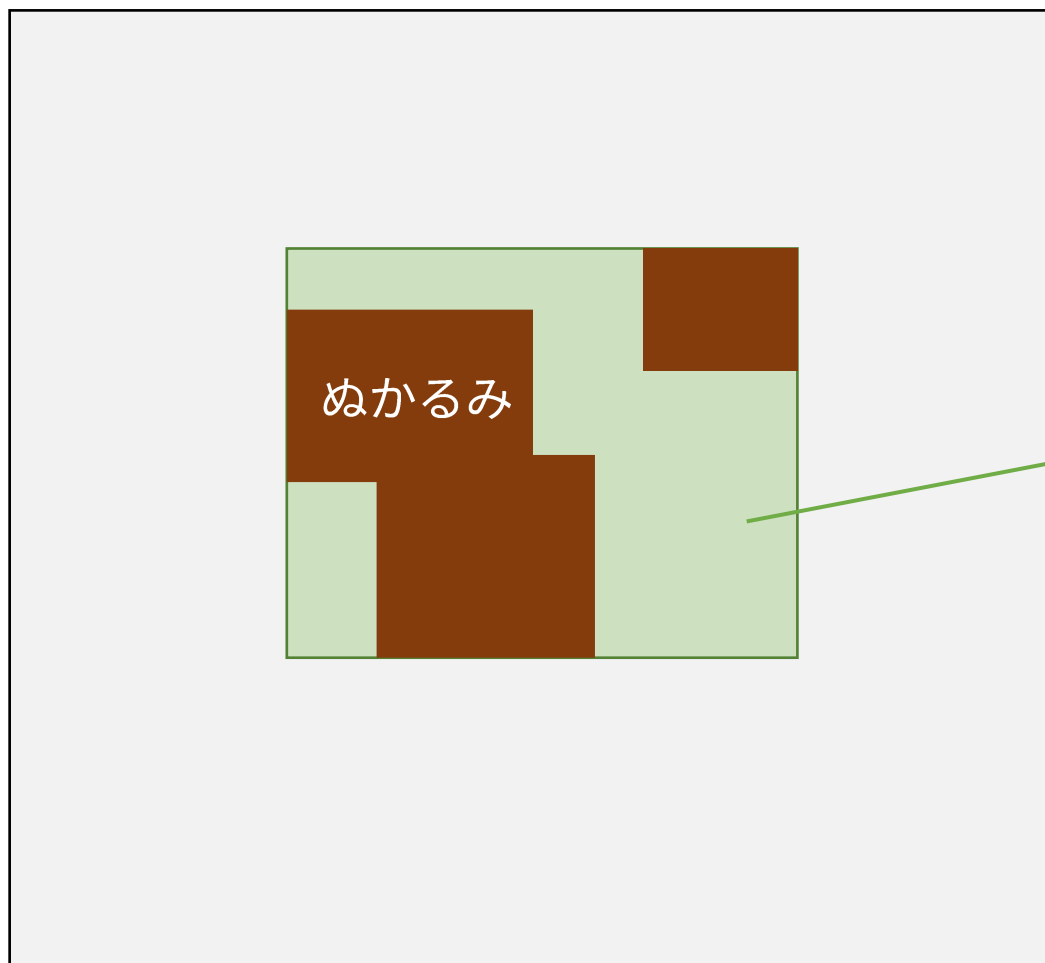
問題概要



問題概要



問題概要



JOI庭園

立ち入り禁止

(雨が降るごとに
面積を求める)

制約・配点

$$N \leq 200\,000$$

$$H \leq 10^9$$

$$W \leq 10^9$$

1

$$X = 1$$

2

$$W = 1$$

3

$$N \leq 300$$

4

$$N \leq 5\,000$$

5

-

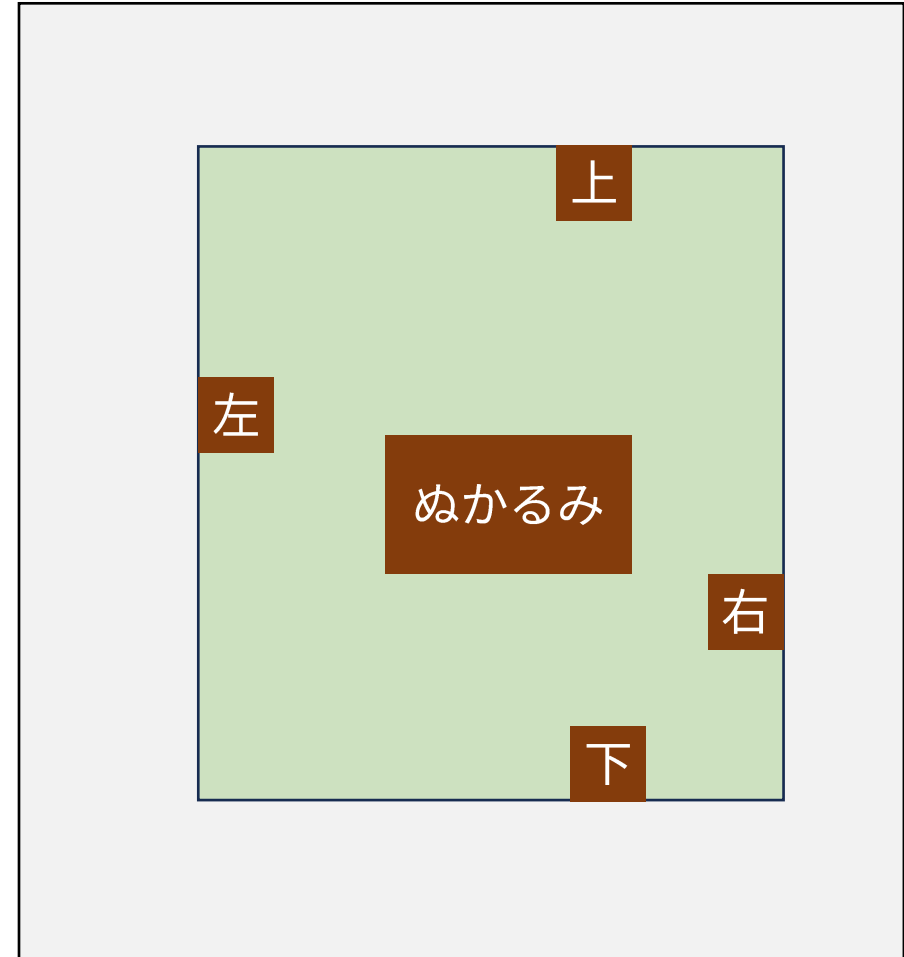


1

小課題 1 $X = 1$

立ち入り禁止領域の選び方

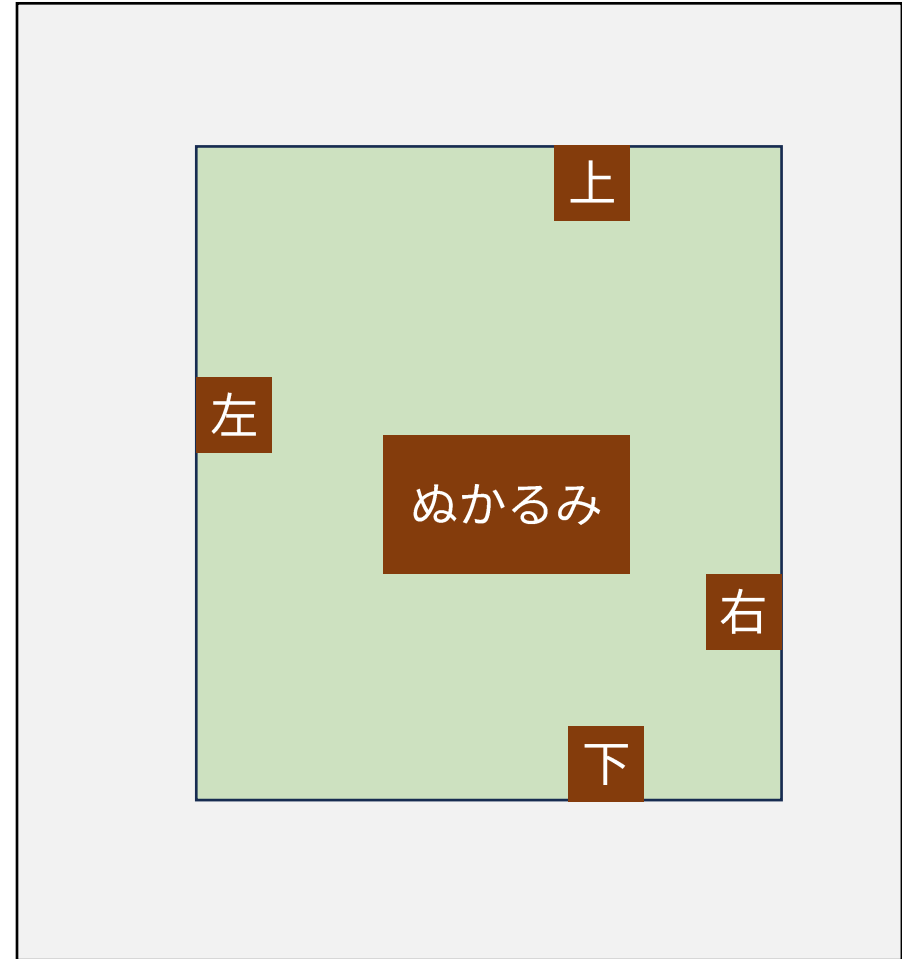
禁止領域の左(/右/上/下)の境界は、
最も左(/右/上/下)のぬかるみの場所。



小課題 1 $X = 1$

小課題 1 では、
雨が降った場所すべてがぬかるみなので、
雨全体の左(/右/上/下)端を求める。

左端の min , 右端の max などを管理。
計算量 $O(N)$.



小課題 3 $N \leq 300$

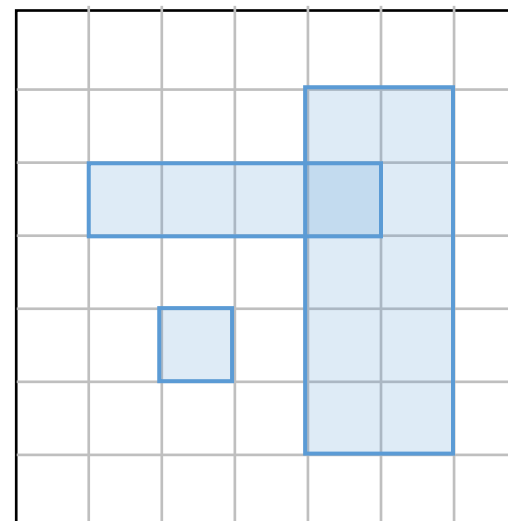
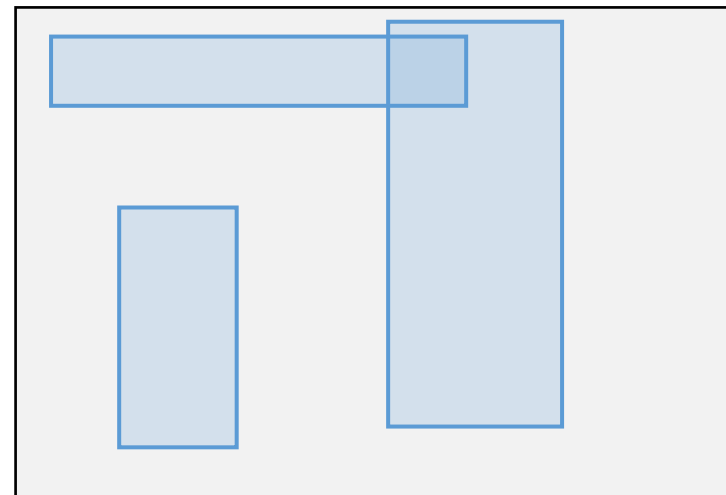
マス目の圧縮

各長方形の境界以外は無視できる．

縦横それぞれ，座標を変換（圧縮）する．

★ 長方形の境界とならない座標を削除する．

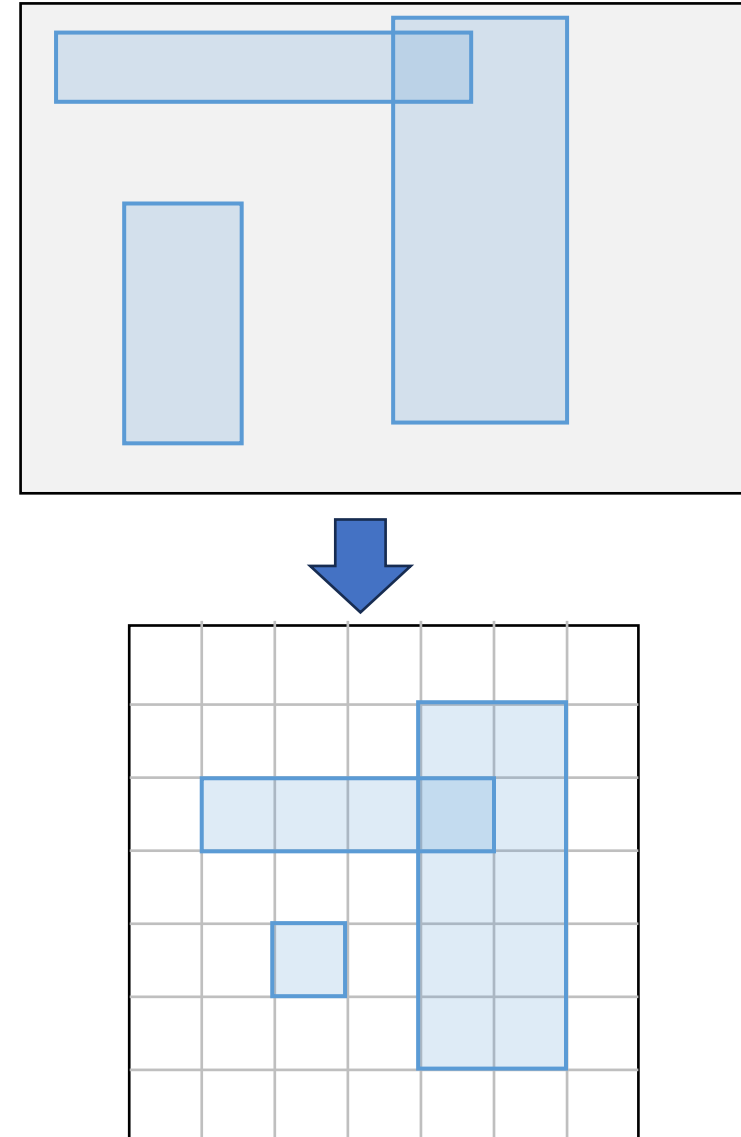
H, W はおよそ $2N$ 以下になる．



小課題 3 $N \leq 300$

変換後は，各処理を for ループで行っても
計算量は $O(N^3)$ である．

最後に，変換後で求めた上下左右端を，
もとの座標に戻す． ☐



小課題 2 $W = 1$

縦方向は 1 マスしかないという制約.

横方向を圧縮すると, およそ 400 000 マスになる.

小課題 3 の方法で計算量は $O(N^2)$ になるが, 高速化不足.

★ セグメント木で高速化できる.

小課題 2 $W = 1$

- 値が x 以上である最も左のマスを見つけない
 - ★ 最大値を管理するセグメント木を用いた二分探索
- 連続する部分に一様に加算したい
 - ★ 遅延評価

計算量は $O(N \log N)$ になる. $\square$

小課題 2 $W = 1$

セグメント木は↓のような形なので、

「左半分にぬかるみがあるか？」等がすぐに分かる → 二分探索が高速



(マス目)



小課題 4 $N \leq 5\,000$

- 小課題 2 の方法を繰り返す ➡ 計算量 $O(N^2 \log N)$ だが、高速化不足.

(かなり大雑把だが)

遅延評価セグメント木の操作が N^2 ($\doteq 2.5 \times 10^7$) 回ほど

- セグメント木の工夫で高速化は難しい.

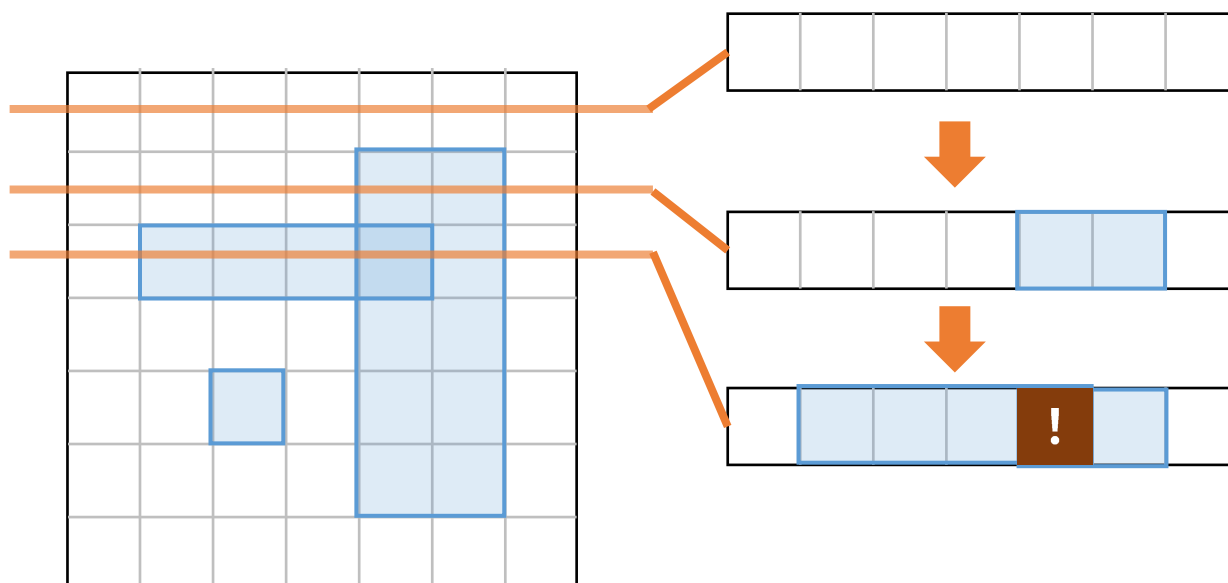
★ 「部分長方形に一様加算・部分長方形内の最小値」は APSP-hard に関連

→ 計算量 $N(\log N)^{O(1)}$ が絶望的

小課題 4 $N \leq 5\,000$

★ 1つの時点だけ答えを求めるのは高速化できる．

1行分をセグメント木で表して、上から下へ走査 → 上端がわかる
これを4方向でやる．



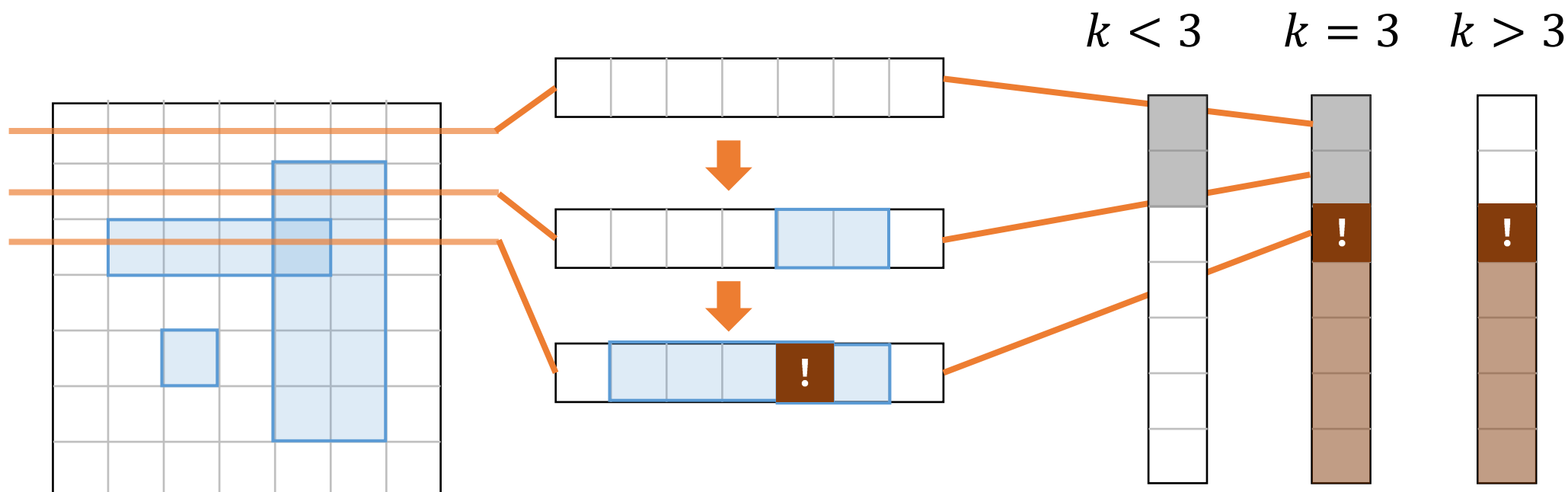
計算量は $O(N \log N)$ ．

小課題 4 $N \leq 5\,000$

★ 1つの時点の結果から、他の時点の結果の一部も分かる。

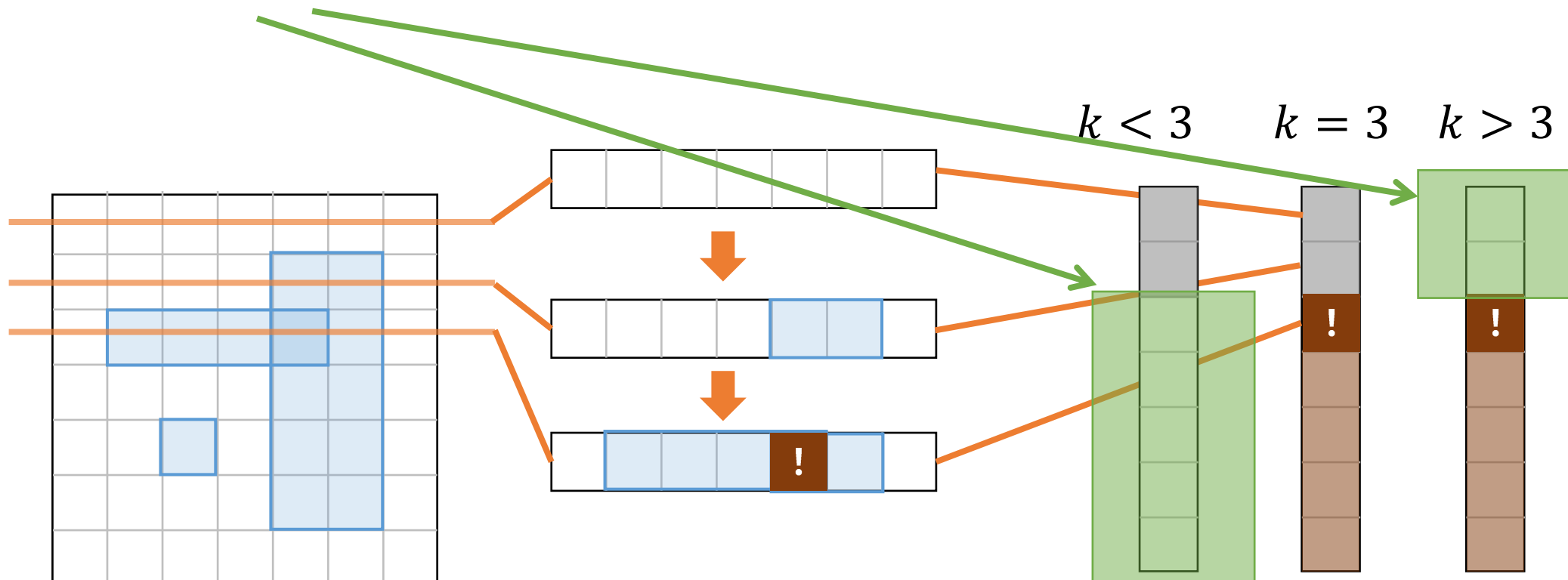
加算を減らすとき、ぬかるみは増えない。

加算を増やすとき、ぬかるみは減らない。



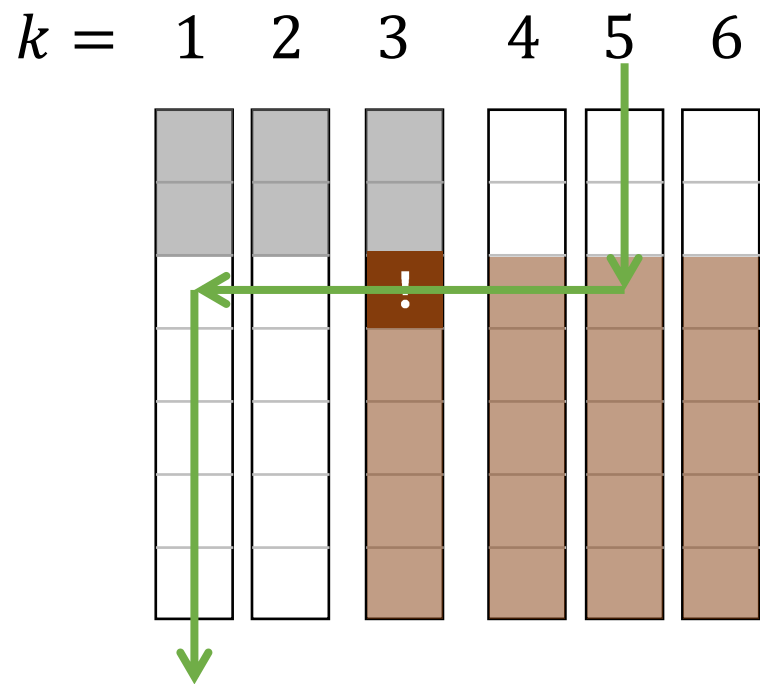
小課題 4 $N \leq 5\,000$

- ★ 1つの時点の結果から，他の時点の結果が一部分かる．
残った2つの領域で同じことをすればよい．



小課題 4 $N \leq 5\,000$

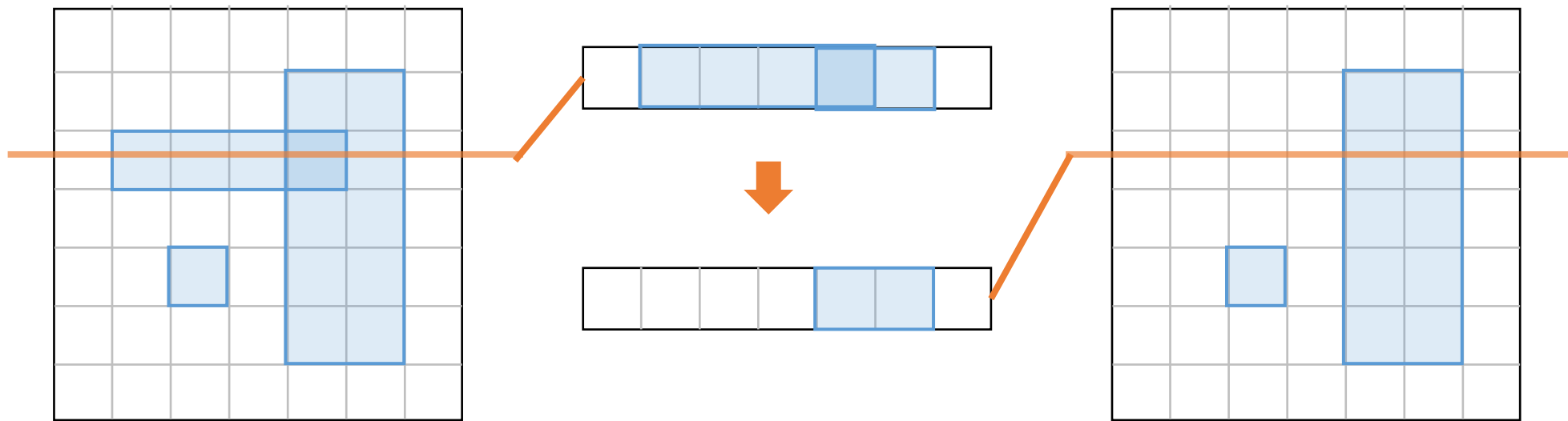
次はこんな順番で走査できると，一度にできてうれしい．



- 左方向の移動（加算を1回キャンセルする）操作が必要．

小課題 4 $N \leq 5\,000$

「加算を 1 回キャンセルする」操作

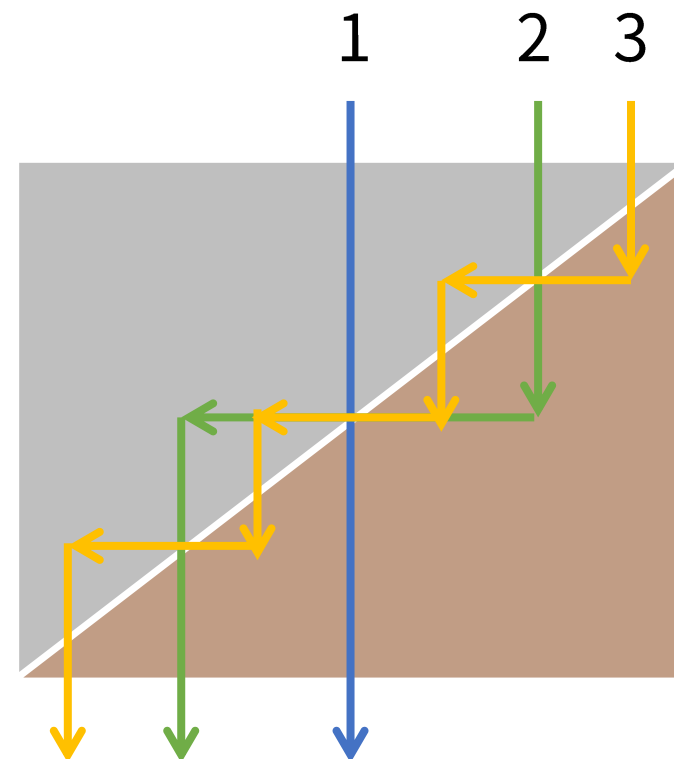


小課題 4 $N \leq 5\,000$

これを $O(\log N)$ 回繰り返すと，すべて求まる．
計算量は $O(N(\log N)^2)$ ． $\square$

(かなり大雑把だが)

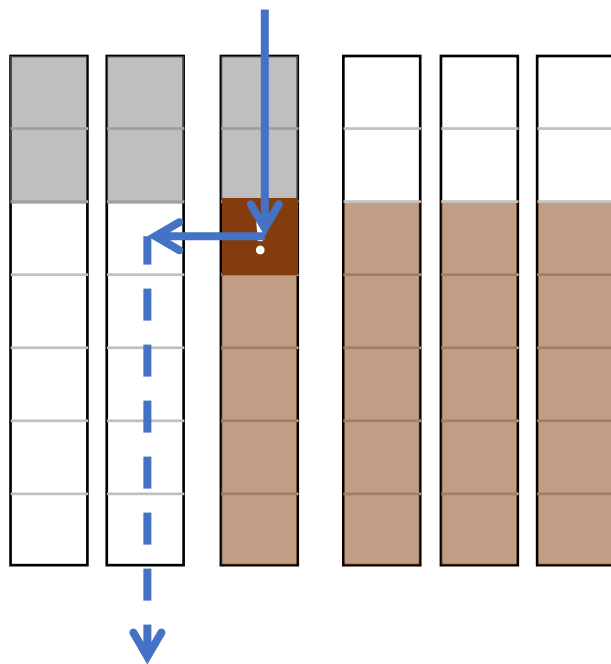
遅延評価セグメント木の操作を
 $8N \log_2 N$ ($\doteq 2.7 \times 10^7$) 回ほど
行うことになり，満点は難しい．



小課題 5 ($N \leq 200\,000$)

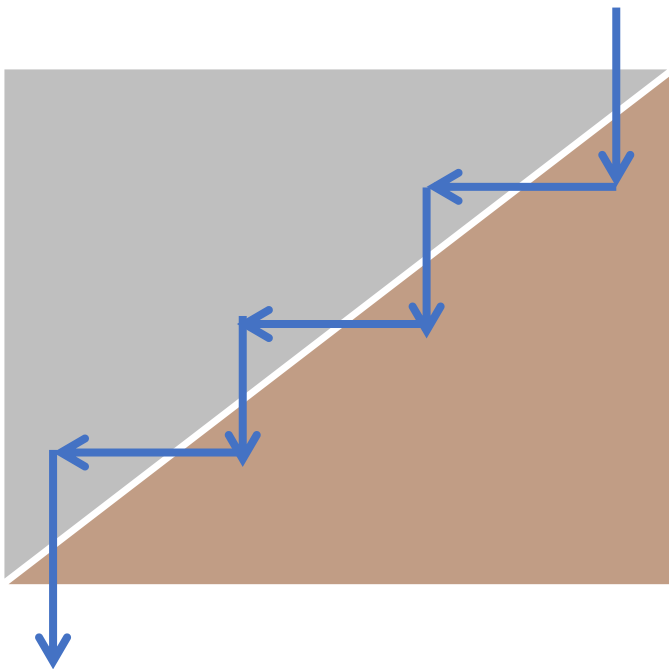
ぬかるみを発見したときに直前の加算をキャンセルすると、
すぐに直前の時点の処理に移れる。

$k =$ 1 2 3 4 5 6

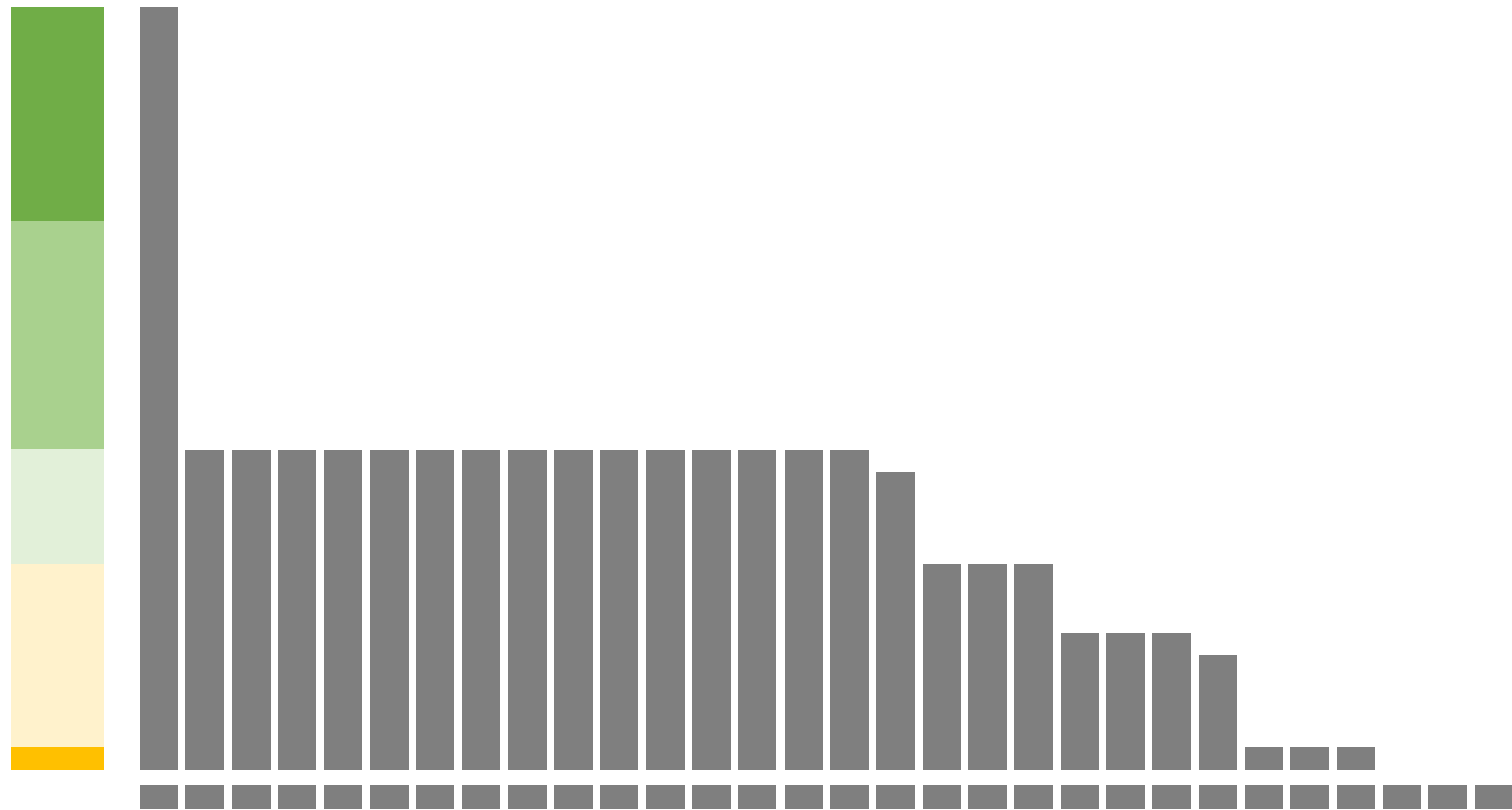


小課題 5 ($N \leq 200\,000$)

$k = N$ から始めれば，1 回の走査ですべて求まる．
計算量は $O(N \log N)$ ． $\square$



得点分布

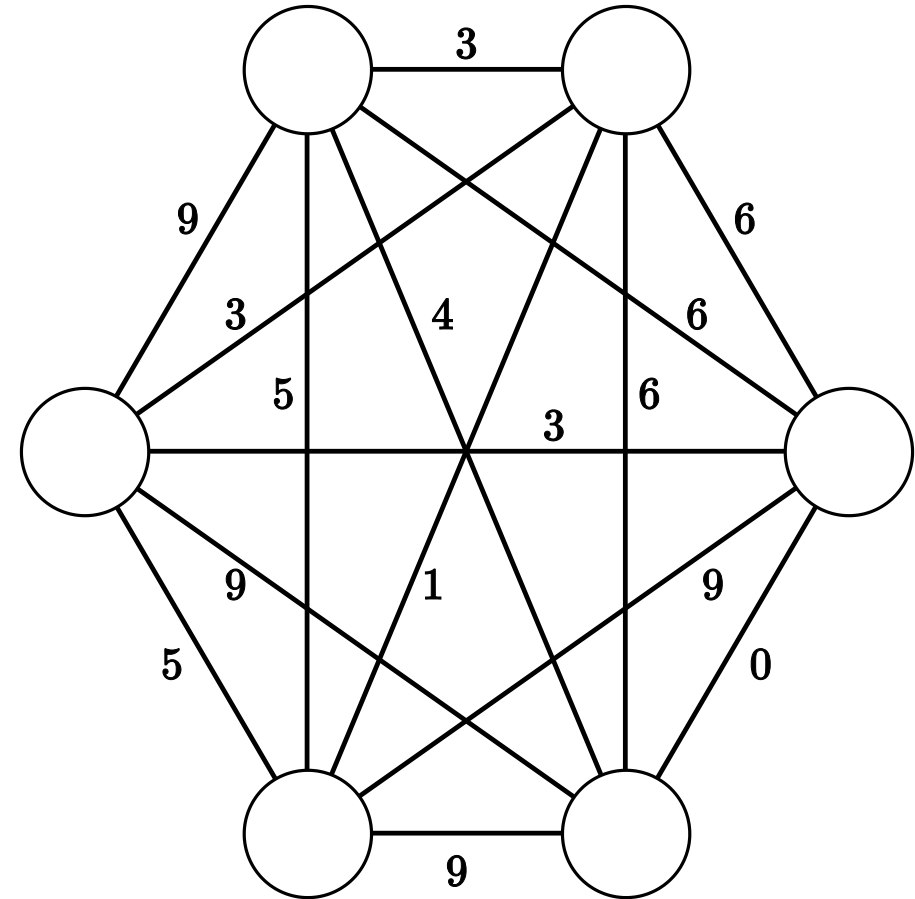


JOI 2025/2026 ファイナルステージ Day 1 – マルチコミュニケーション 2 (Multi Communication 2) 解説

解説担当: 山縣 龍人 (tatyam)

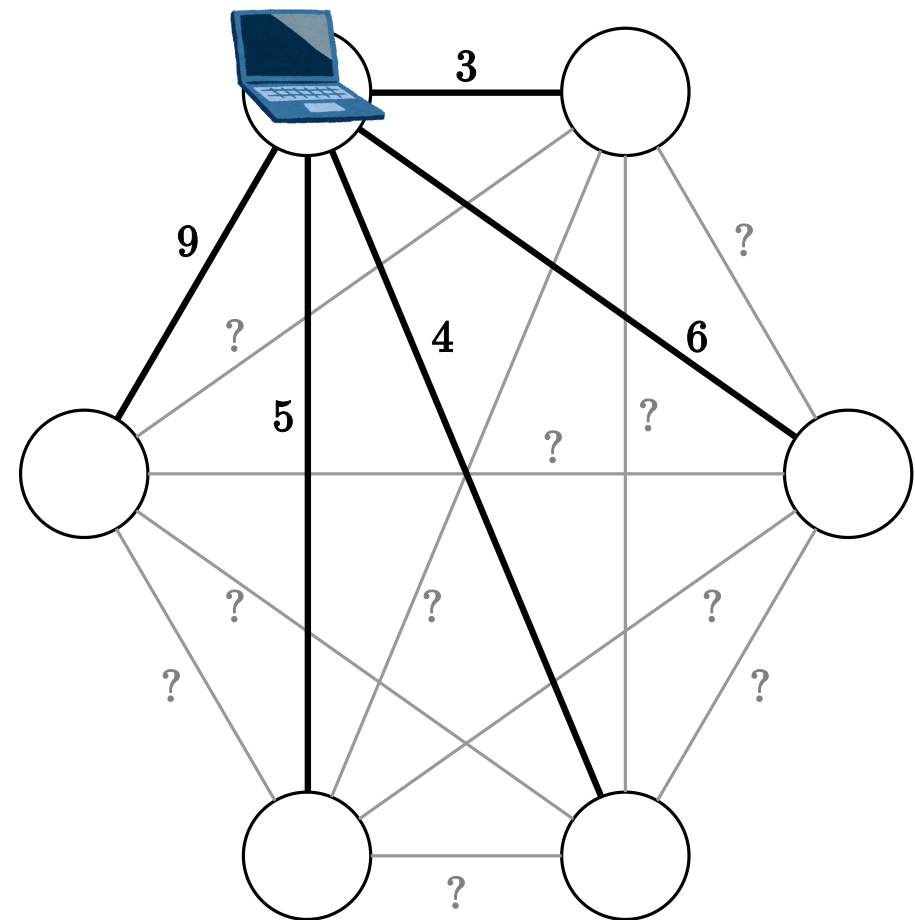
問題概要

- N 頂点の重み付き完全無向グラフがある.



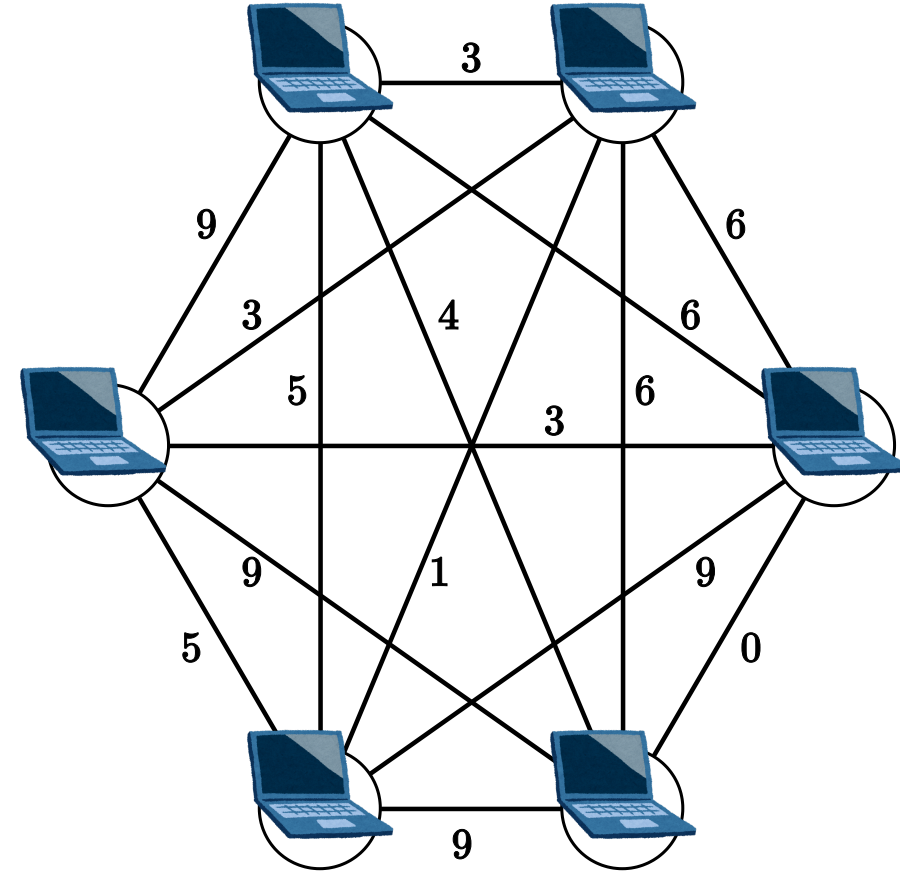
問題概要

- N 頂点の重み付き完全無向グラフがあるが、重みは隠されている.
- 各頂点は、その頂点に隣接する辺の重みが分かる.



問題概要

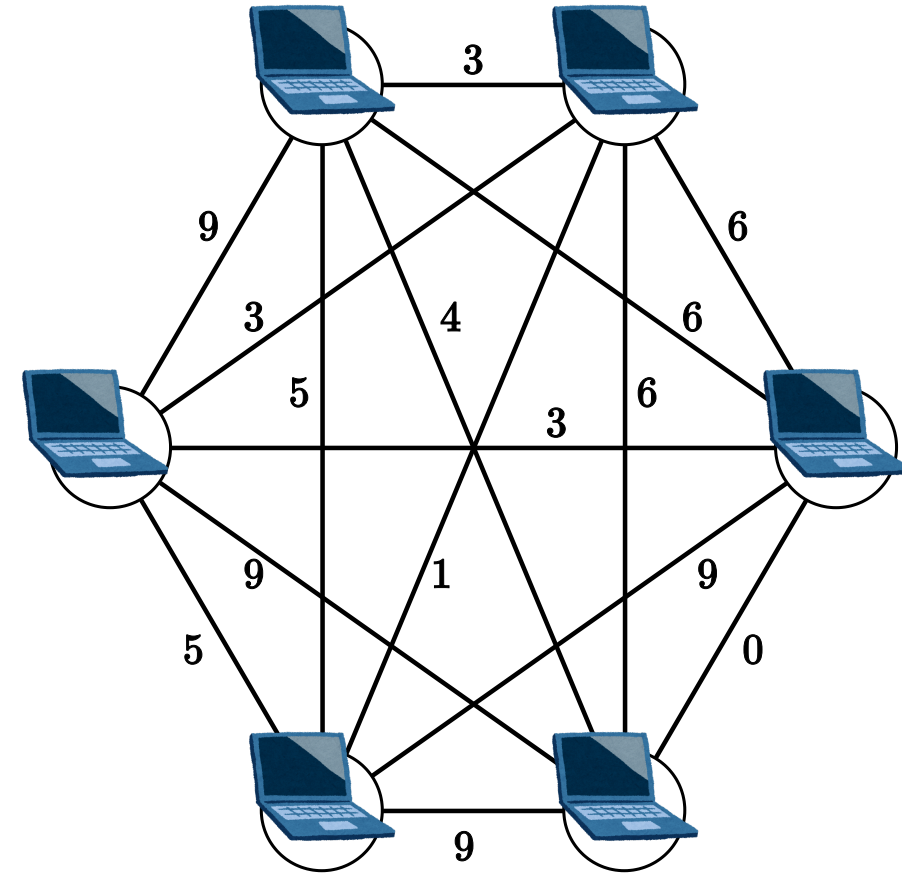
- N 頂点の重み付き完全無向グラフがあるが、重みは隠されている.
- 各頂点は、その頂点に隣接する辺の重みが分かる.
- 全頂点が協力して **分散計算** を行い、最小全域木の重み X を求める.



分散計算の方法

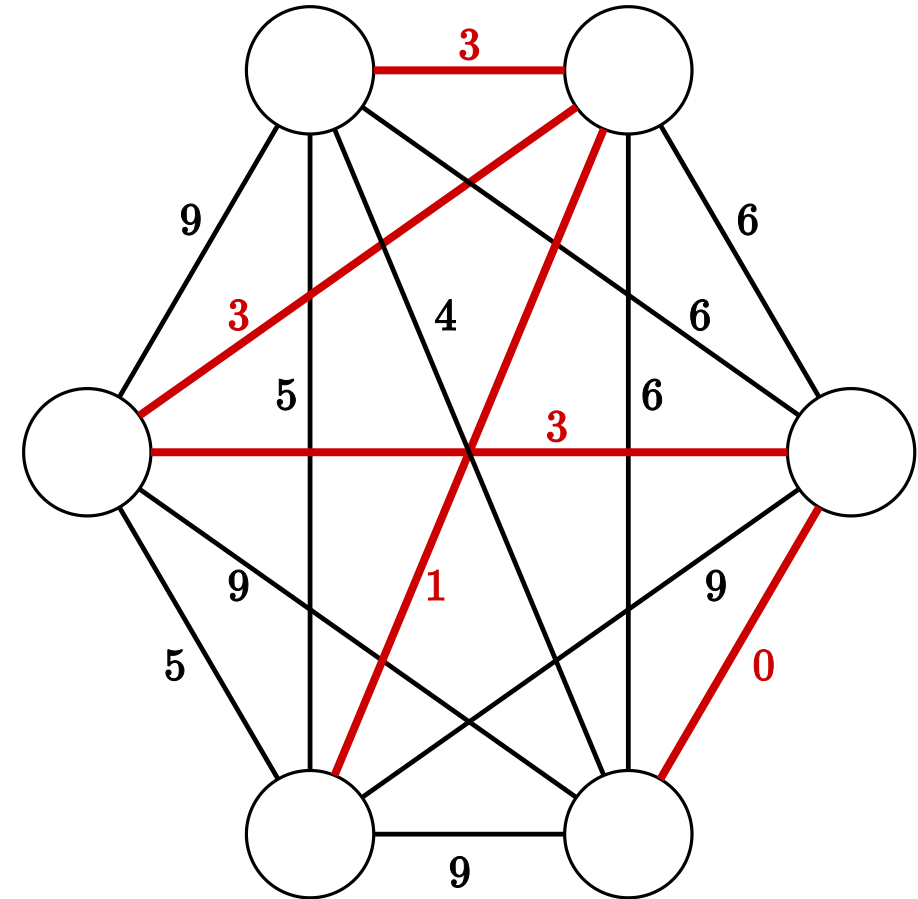
以下の **ラウンド** を繰り返す.

1. 前のラウンドの各頂点から 64 bit の情報を受け取る
2. 最小全域木の重み X が求められたなら, それを回答する.
3. そうでなければ, 次のラウンドの各頂点へ 64 bit の情報を送信する.
(次のラウンドの自分自身へも 64 bit の情報しか引き継がない!)



問題概要

- N 頂点の重み付き完全無向グラフがあるが、重みは隠されている.
- 各頂点は、その頂点に隣接する辺の重みが分かる.
- 全頂点が協力して **分散計算** を行い、最小全域木の重み X を求める.
- 何ラウンドかかるか？
- $N \leq 256$, 辺重み $A_{i,j}$ は 48 bit



小課題

	配点	$N \leq$	$A_{i,j}$ は	満点には
1	5 点	64	1 bit	6 ラウンド
2	10 点	256	1 bit	6 ラウンド
3	15 点	64	48 bit	80 ラウンド
4	40 点	256	20 bit	6 ラウンド
5	15 点	256	40 bit	6 ラウンド
6	15 点	256	48 bit	6 ラウンド

小課題内の点数 (小課題 3 を除く)

- 6 ラウンド → 100%
- 7 ラウンド → 80%
- 8 ラウンド → 60%
- 9 ラウンド → 40%
- 10 ラウンド → 30%

小課題

	配点	$N \leq$	$A_{i,j}$ は	満点には
1	5 点	64	1 bit	6 ラウンド
2	10 点	256	1 bit	6 ラウンド
3	15 点	64	48 bit	80 ラウンド
4	40 点	256	20 bit	6 ラウンド
5	15 点	256	40 bit	6 ラウンド
6	15 点	256	48 bit	6 ラウンド

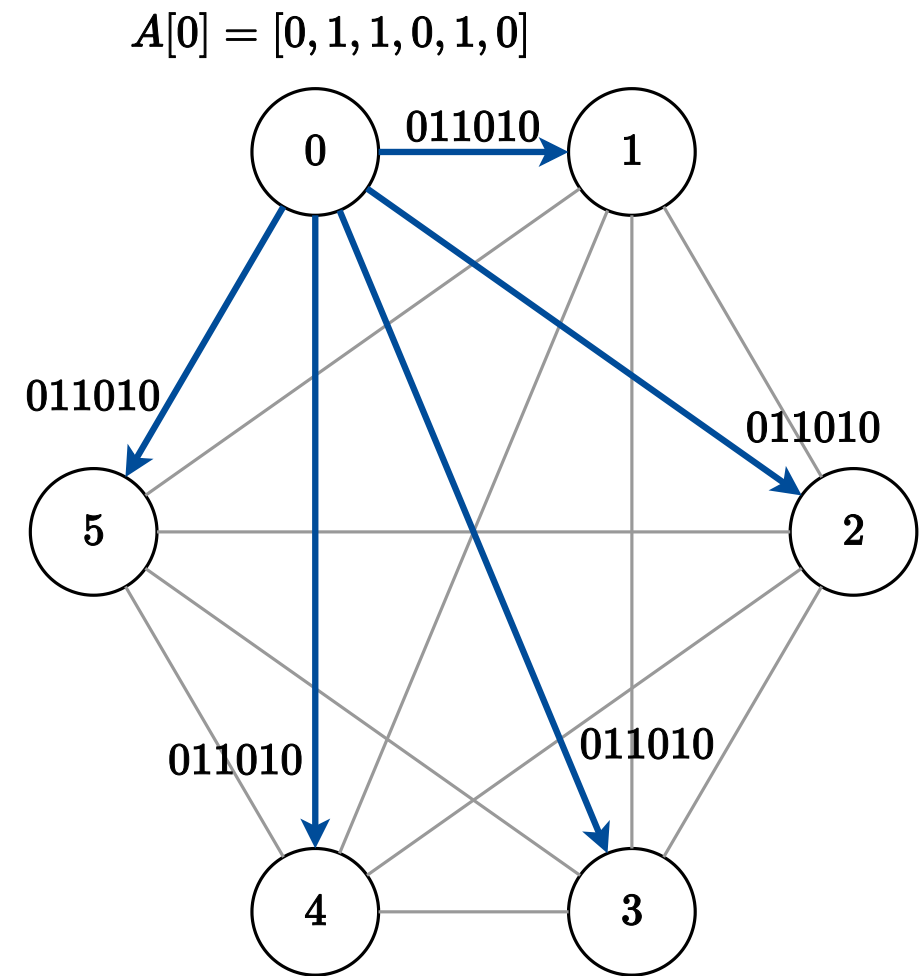
小課題内の点数 (小課題 3 を除く)

- 6 ラウンド → 100%
- 7 ラウンド → 80%
- 8 ラウンド → 60%
- 9 ラウンド → 40%
- 10 ラウンド → 30%

6 ラウンド？ 🤔 本当にそんなことが可能なのか？ 🤔 🤔

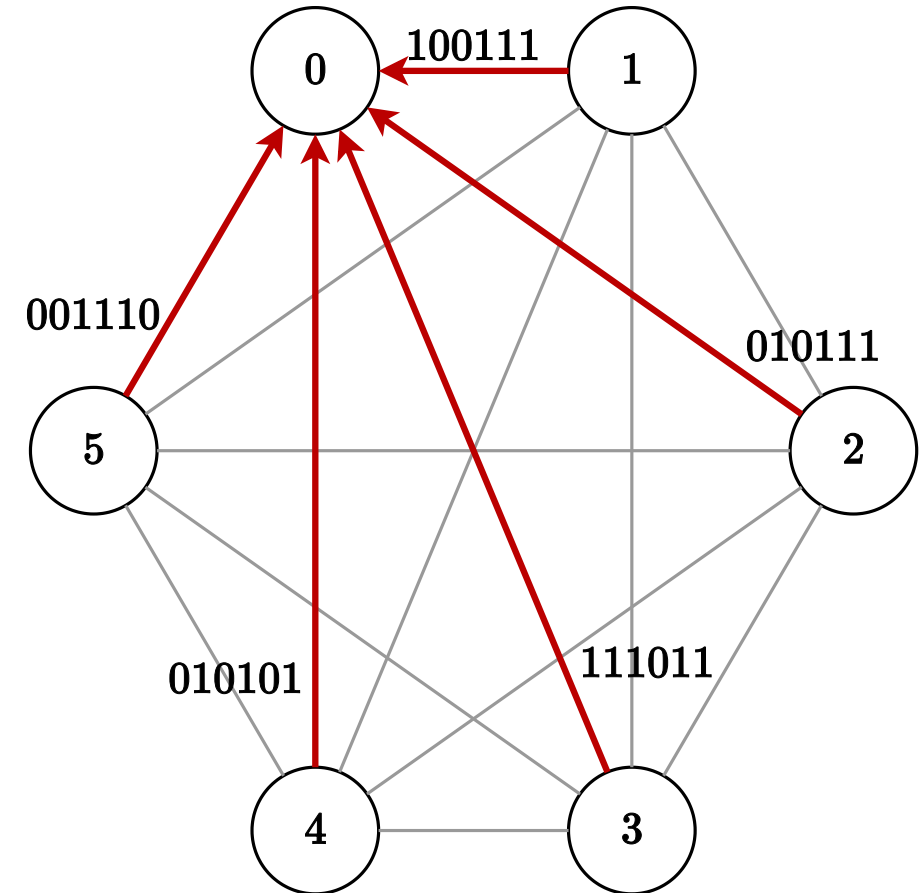
小課題 1: $N \leq 64$, $A_{i,j}$ は 1 bit

- 0 ラウンド目で, 各頂点 i は, 行 $A[i]$ の情報を 64 bit に詰めてすべての頂点に放送することができる!
 - 全頂点に送ることを **放送** と呼ぶことにします.



小課題 1: $N \leq 64$, $A_{i,j}$ は 1 bit

- 0 ラウンド目で, 各頂点 i は, 行 $A[i]$ の情報を 64 bit に詰めてすべての頂点に放送することができる!
 - 全頂点に送ることを **放送** と呼ぶことにします.
 - 1 ラウンド目ですべての辺の情報が集まるので, 最小全域木を計算する.
- 2 ラウンドでできた!



小課題 2: $N \leq 256$, $A_{i,j}$ は 1 bit

- 行 $A[i]$ の情報を 64 bit に詰められない... → **分けて送ろう！**
- 分けて送ろうにも，送られた情報をすべて覚えておく
ことができない...

どうする？

小課題 2: $N \leq 256$, $A_{i,j}$ は 1 bit

- 行 $A[i]$ の情報を 64 bit に詰められない... → **分けて送ろう！**
- 分けて送ろうにも，送られた情報をすべて覚えておく
ことができない

→ 送られた情報のうち，重み 0 の辺をすべて採用して，**現在の
連結成分の状態を覚えておけば良い！**

連結成分の状態を覚えておく

連結成分の状態を覚えておくには...

→ **UnionFind** を再現しよう！

UnionFind を再現しよう！

- $\text{leader}[i] :=$ 頂点 i を含む連結成分の代表元 (例えば, 連結成分の中で最小の頂点)

の配列が分かっているならば良い.

ラウンドの終わりに頂点 i が $\text{leader}[i]$ を放送すれば, 全頂点に配列 leader が渡る！

UnionFind を再現しよう！

- $\text{leader}[i] :=$ 頂点 i を含む連結成分の代表元 の配列が分かっている良い.

ラウンドの終わりに頂点 i が $\text{leader}[i]$ を放送すれば、全頂点に配列 leader が渡る！

注意点：正しい $\text{leader}[i]$ を放送するためには、

- 全員が同じように辺の情報を受け取り、
- 全員が同じように辺を採用し、
- 全員が同じように leader を更新する

必要がある！

小課題 2: $N \leq 256$, $A_{i,j}$ は 1 bit

- 各頂点 i は行 $A[i]$ の情報を 5 回に分けて 52 bit ずつ放送する.
- また, 毎回 $\text{leader}[i]$ を放送する (8 bit).
- 全員が同じように辺の情報を受け取り,
- 重みが 0 である辺を (サイクルができなければ) すべて採用し,
- 全員が同じように leader を更新する.
- 最終的な連結成分数から答えを逆算

5 回の放送 → 6 ラウンドでできた！

テクニック (1) : 採用された辺の情報を持たせる

実は,

- $\text{leader}[i] :=$ 頂点 i を含む連結成分の代表元 (例えば, 連結成分の中で最小の頂点)

の配列に, 「どの辺が採用されたか？」を持たせることができる！

テクニック (1) : 採用された辺の情報を持たせる

現在採用されている辺によって作られる森を T とする.

- $\text{leader}[i] :=$ 頂点 i を含む連結成分の代表元 (例えば, 連結成分の中で最小の頂点)

の代わりに,

- $\text{parent}[i] := T$ 上で頂点 i から $\text{leader}[i]$ 方向へ 1 つ進んだ頂点を持つ (T を有向森として表現している).

parent から leader は復元できるので, 情報が追加されている! 😊

小課題 3: $N \leq 64$, $A_{i,j}$ は 48 bit, 80 ラウンドで満点

最小全域木のアルゴリズムといえは... ?

小課題3: $N \leq 64$, $A_{i,j}$ は 48 bit, 80 ラウンドで満点

最小全域木のアルゴリズムといえば...?

- クラスカル法 (いつもの)
- プリム法 (ダイクストラ法みたいな)
- ブルーフカ法 (?)

小課題 3: $N \leq 64$, $A_{i,j}$ は 48 bit, 80 ラウンドで満点

最小全域木のアルゴリズムといえば... ?

- クラスカル法
- **プリム法** ← 小課題 3 の正解
- ブルーフカ法

クラスカル法でも可能 (ブルーフカ法に似るので省略)

最小全域木の性質

定理

頂点集合を V とする.

任意の $S \subset V$ に対して, $T := V \setminus S$ とする.

$S - T$ 間の最小重みの辺は, 最小全域木に採用して良い.

プリム法とは？

以下を $N - 1$ 回繰り返す：

- 頂点 0 と連結な頂点の集合を S ，そうでない頂点の集合を T とする．はじめ， $S = \{0\}, T = \{1, 2, \dots, N - 1\}$ である．
- $S - T$ 間の辺 $u - v$ であって，重みが最小のものを選ぶ．
- 辺 $u - v$ を採用し， v を T から S に移動させる．

小課題 3: プリム法

- $i \in T$ である各頂点 i は, $i - S$ 間の最小重みを放送する (48 bit).
→ この重みが最小である i を S に加える.
- 上記を放送するために, S, T の要素が分かる必要がある
→ 各頂点は $i \in S$ または $i \in T$ の 1 bit を放送する.
- 採用された辺重みを合計する必要がある
→ 頂点 0 は, 採用された辺重みの合計を放送する (56 bit).

N ラウンドでできた!

ブルーフカ法とは？

以下を高々 $\text{floor}(\log_2(N))$ 回繰り返す：

1. 各連結成分について、その連結成分の内外をまたぐ辺のうち、重みが最小のものを求める.
2. それらを (サイクルにならない限り) 採用する.

1 フェーズにおいて、各連結成分は少なくとも1つの連結成分と連結になる.

→ k フェーズ行ったとき、各連結成分のサイズは 2^k 以上！

💡 テクニック (2): ブルーフカ法豆知識

- 各連結成分のサイズが x 以上である, かつ
 - $N \leq 2x - 1$ である
- ⇒ グラフは連結である

余計に 1 フェーズやらないよう注意！

ブルーフカ法とは？

以下を高々 $\log_2(N)$ 回繰り返す：

1. 各連結成分について，その連結成分の内外をまたぐ辺のうち，重みが最小のものを求める．
2. それらを (サイクルにならない限り) 採用する．

辺を並列に探すことができる

→ 分散計算に適している！

ブルーフカ法 : 9 ラウンド

- 各頂点 i は, $\text{leader}[i]$ を放送する (8 bit).
- 各頂点 i は, $i - (i \text{ と連結でない頂点})$ 間の (最小重み, 行き先) を放送する (56 bit).

どうやって採用された辺の重みを合計する？

ブルーフカ法 + 集計: 10 ラウンド

- 各頂点 i は, **parent** $[i]$ を放送する (8 bit).
- 各頂点 i は, $i - (i$ と連結でない頂点) 間の (最小重み, 行き先) を放送する (56 bit).

8 回の放送で最小全域木を求める → 追加の 1 回の放送で,
最小全域木に採用された辺の重みを放送する (頂点 i は $A_{i,\text{parent}[i]}$
を放送)

放送 9 回 → 10 ラウンドでできた!

ブルーフカ法 : 9 ラウンド

- 各頂点 i は, $\text{leader}[i]$ を放送する (8 bit).
- 各頂点 i は, $i - (i \text{ と連結でない頂点})$ 間の (最小重み, 行き先) を放送する (56 bit).

送る必要のない情報はないか？

テクニック (3) : 自分自身に送る情報を変更

- 各頂点 i は, $\text{leader}[i]$ を放送する (8 bit).
- 各頂点 i は, $i - (i \text{ と連結でない頂点})$ 間の (最小重み, 行き先) を放送する (56 bit).
 - **自分自身にこの情報を送る必要はない !**

テクニック (3) : 自分自身に送る情報を変更

- 各頂点 i は, $\text{leader}[i]$ を放送する (8 bit).
- 各頂点 i は, $i - (i \text{ と連結でない頂点})$ 間の (最小重み, 行き先) を放送する (56 bit).
 - 自分自身へは, 「採用された辺の重みの合計」を送信する (56 bit)

放送 8 回 $\rightarrow$ 9 ラウンドでできた！

一応こんな方法も...？

- 各頂点 i は, $\text{leader}[i]$ を放送する (8 bit).
 - **種類が 8 bit もない！** 連結成分の番号を $0, 1, 2, \dots$ に振り直しても問題ない！
- 各頂点 i は, $i - (i \text{ と連結でない頂点})$ 間の (最小重み, 行き先) を放送する (56 bit).

一応こんな方法も...？

- 各頂点 i は, $\text{leader}[i]$ を放送する ($\text{ceil}(\log_2 N) - 1$ bit).
- 各頂点 i は, $i - (i \text{ と連結でない頂点})$ 間の (最小重み, 行き先) を放送する ($47 + \text{ceil}(\log_2 N)$ bit).
- $x := 18 - 2\text{ceil}(\log_2 N)$ とする. 各頂点 i は, 「採用された辺の重みの合計」の 2^x 進法で i 桁目を放送する (x bit).
 - $N \geq 4$ であれば足りる！

放送 8 回 $\rightarrow$ 9 ラウンドでできた！

テクニック (4) : 初手 3 倍

さっき 10 ラウンドになってしまったブルーフ法ですが... ?

- 各頂点 i は, **parent** $[i]$ を放送する (8 bit).
- 各頂点 i は, $i - (i \text{ と連結でない頂点})$ 間の (最小重み, 行き先) を放送する (56 bit).
- 8 回の放送で最小全域木を求める $\rightarrow$ 追加の 1 回の放送で, 最小全域木に採用された辺の重みを放送する

送る必要のない情報はないか ?

テクニック (4) : 初手 3 倍

0 ラウンド目では... ?

- 各頂点 i は, `parent` $[i]$ を放送する (8 bit).
- 各頂点 i は, $i - (i \text{ と連結でない頂点})$ 間の (最小重み, 行き先) を放送する (56 bit).

テクニック (4) : 初手 3 倍

0 ラウンド目では... ?

- ~~各頂点 i は, $\text{parent}[i]$ を放送する (8 bit).~~
- 各頂点 i は, $i - (i$ と連結でない頂点) 間の (~~最小重み~~, 行き先) を放送する (~~56 bit~~ 8 bit).

8 bit しかない !

テクニック (4) : 初手 3 倍

0 ラウンド目では, もう 1 辺送れる !

- 各頂点 i は, $i - (i$ と連結でない頂点) 間の 1st min の行き先を放送する (8 bit).
- 各頂点 i は, $i - (i$ と連結でない頂点) 間の 2nd min の (重み, 行き先) を放送する (56 bit).

テクニック (4) : 初手 3 倍

定理

頂点集合を V とする.

任意の $S \subset V$ に対して, $T := V \setminus S$ とする.

$S - T$ 間の最小重みの辺は, 最小全域木に採用して良い.

1st min の辺が双方向に向いていたら, その辺を採用した後, その連結成分から出る最小重みの辺は「2nd min のうち重みの小さい方」である. この辺も採用できる!

→ 連結成分のサイズが 3 以上に!

ブルーフカ法 + 初手 3 倍 + 集計 : 9 ラウンド

0 ラウンド目 : 1st min と 2nd min を送る

1 ラウンド目以降 :

- 各頂点 i は, **parent** $[i]$ を放送する (8 bit).
- 各頂点 i は, $i - (i$ と連結でない頂点) 間の (最小重み, 行き先) を放送する (56 bit).

$3 \times 2^6 = 192$ なので, 7 回の放送で最小全域木を求める → 追加の 1 回の放送で, 最小全域木に採用された辺の重みを放送する

9 ラウンドでできた !

小課題 4: 3 倍ブルーフカ法

小課題 4 の制約 「 $A_{i,j}$ は 20 bit」 をうまく使えないか？

→ (最小重み, 行き先) を 2 本放送することができる！

小課題 4: 3 倍ブルーフカ法

- 各頂点 i は, $\text{leader}[i]$ を放送する (8 bit).
- 各頂点 i は, $i - (i \text{ と連結でない頂点})$ 間の 1st min と 2nd min の (重み, 行き先) を放送する ($28 + 28$ bit).
- 各連結成分ごとに, その内外を結ぶ辺の 1st min と 2nd min を求める
- 1st min の辺が双方向に向いていたら, その辺を採用した後, その連結成分から出る最小重みの辺は「2nd min のうち重みの小さい方」である. この辺も採用できる!

$3^5 = 243$ なので, 放送 5 回 $\rightarrow$ 6 ラウンドでできた! 100

ブルーフカ法 : 9 ラウンド (再掲)

- 各頂点 i は, $\text{leader}[i]$ を放送する (8 bit).
- 各頂点 i は, $i - (i \text{ と連結でない頂点})$ 間の (最小重み, 行き先) を放送する (56 bit).
 - 自分自身へは, 「採用された辺の重みの合計」を送信する (56 bit)

放送 8 回 → 9 ラウンドでできた！

どこに無駄があるか？ 🤔

ブルーフカ法: 9 ラウンド (再掲)

連結成分のサイズ:

$$1 \rightarrow 2 \rightarrow 4 \rightarrow 8 \rightarrow 16 \rightarrow 32 \rightarrow 64 \rightarrow 128 \rightarrow 256$$

採用される辺の本数:

$$1 \xrightarrow{128} 2 \xrightarrow{64} 4 \xrightarrow{32} 8 \xrightarrow{16} 16 \xrightarrow{8} 32 \xrightarrow{4} 64 \xrightarrow{2} 128 \xrightarrow{1} 256$$

ブルーフカ法: 9 ラウンド (再掲)

連結成分のサイズ:

$1 \rightarrow 2 \rightarrow 4 \rightarrow 8 \rightarrow 16 \rightarrow 32 \rightarrow 64 \rightarrow 128 \rightarrow 256$

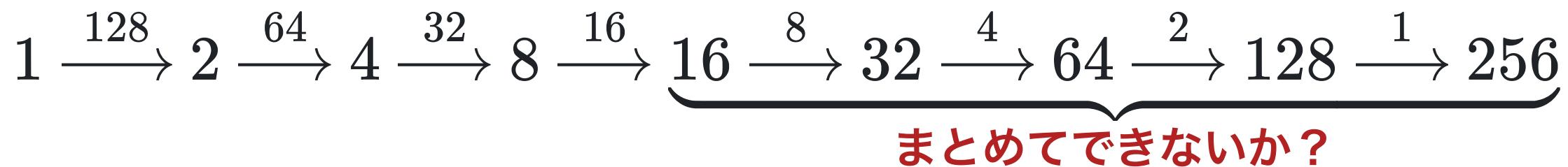
採用される辺の本数:

$1 \xrightarrow{128} 2 \xrightarrow{64} 4 \xrightarrow{32} 8 \xrightarrow{16} 16 \xrightarrow{8} 32 \xrightarrow{4} 64 \xrightarrow{2} 128 \xrightarrow{1} 256$

まとめてできないか？

テクニック (5) : 2 ラウンドかけて頂点を縮約

採用される辺の本数:



16 頂点の連結成分が 16 個あるとき、**各連結成分間の最小重みの情報を 256 頂点で分担して放送できそう！**

ブルーフカ法 + 縮約 : 7 ラウンド

ブルーフカ法で 4 回放送し, (各連結成分のサイズ) ≥ 16 にしておく.

目標 : 連結成分 i の k 番目の頂点が, (連結成分 i) $-$ (連結成分 k)
間の最小重みを放送する.

ブルーフカ法 + 縮約 : 7 ラウンド

ブルーフカ法で 4 回放送し, (各連結成分のサイズ) ≥ 16 にしておく.

目標 : 連結成分 i の k 番目の頂点が, (連結成分 i) $-$ (連結成分 k) 間の最小重みを放送する.

↑

そのために, (連結成分 i の k 番目の頂点) は, (連結成分 k) の各頂点 j から, $j -$ (連結成分 i) 間の最小重みを受け取る.

どうやって採用された辺の重みを合計する ?

ブルーフカ法 + 縮約 : 7 ラウンド

ブルーフカ法で 4 回放送し, (各連結成分のサイズ) ≥ 16 にしておく.

目標 : 連結成分 i の k 番目の頂点が, (連結成分 i) $-$ (連結成分 k) 間の最小重みを放送する. **$i = k$ なら放送する必要はない !**

↑

そのために, (連結成分 i の k 番目の頂点) は, (連結成分 k) の各頂点 j から, $j -$ (連結成分 i) 間の最小重みを受け取る.

ここに「採用された辺の重みの合計」を入れよう !

ブルーフカ法 + 縮約 : 7 ラウンド

ブルーフカ法で 4 回放送し, (各連結成分のサイズ) ≥ 16 にしておく.

目標 : 連結成分 i の k 番目の頂点は, $i = k$ なら, 「採用された辺の重みの合計」を放送する

↑

そのために, (連結成分 i の k 番目の頂点) は, いずれかの頂点から「採用された辺の重みの合計」を受け取る.

これで辺の重みを合計できる !

ブルーフカ法 + 縮約 : 7 ラウンド

ブルーフカ法で 4 回放送し, (各連結成分のサイズ) ≥ 16 にしておく.

	$i \neq k$	$i = k$
連結成分 i の k 番目の頂点が,	(連結成分 i) – (連結成分 k) 間の最小重みを放送	「採用された辺の重みの合計」を放送
そのために, (連結成分 i の k 番目の頂点) は,	(連結成分 k) の各頂点 j から, j – (連結成分 i) 間の最小重みを受け取る	いずれかの頂点から 「採用された辺の重みの合計」を受け取る

6 回放送 → 7 ラウンドでできた !

ブルーフカ法 + 縮約 : 7 ラウンド

ブルーフカ法で 4 回放送し, (各連結成分のサイズ) ≥ 16 にしておく.

	$i \neq k$	$i = k$
連結成分 i の k 番目の頂点が,	(連結成分 i) – (連結成分 k) 間の最小重みを放送	「採用された辺の重みの合計」を放送
そのために, (連結成分 i の k 番目の頂点) は,	(連結成分 k) の各頂点 j から, j – (連結成分 i) 間の最小重みを受け取る	いずれかの頂点から 「採用された辺の重みの合計」を受け取る

6 回放送 → 7 ラウンドでできた !

本当に 6 ラウンドにできるんですか? 🤔

ブルーフ手法 + 縮約 + 初手 3 倍 : 6 ラウンド

実は初手 3 倍が使える！

- 各頂点 i は, $i - (i$ と連結でない頂点) 間の 1st min の行き先を放送する (8 bit).
- 各頂点 i は, $i - (i$ と連結でない頂点) 間の 2nd min の (重み, 行き先) を放送する (56 bit).
- 1st min の辺が双方向に向いていたら, その辺を採用した後, その連結成分から出る最小重みの辺は「2nd min のうち重みの小さい方」である. この辺も採用できる！

どうやって, 採用された 1st min の辺の重みを合計する？

ブルーフカ法 + 縮約 + 初手 3 倍 : 6 ラウンド

- 各頂点 i は, $i - (i$ と連結でない頂点) 間の 1st min の行き先を放送する (8 bit).
- 各頂点 i は, $i - (i$ と連結でない頂点) 間の 2nd min の (重み, 行き先) を放送する (56 bit).
- 1st min の辺が双方向に向いていたら, その辺を採用した後, その連結成分から出る最小重みの辺は「2nd min のうち重みの小さい方」である. この辺も採用できる!

- 採用された 1st min の辺の重みは, その辺の両端の頂点しか分からない
→ どちらかに, 採用された 1st min の辺の重みの合計を保持してもらう!
- 採用された 2nd min の辺の重みは誰でも分かる
→ 頂点 0 に合計を保持してもらう!

ブルーフカ法 + 縮約 + 初手 3 倍 : 6 ラウンド

- 初手 3 倍
 - 採用された 1st min の辺の重みは, その辺の一端が保持する
 - 採用された 2nd min の辺の重みは, 頂点 0 が保持する
- ブルーフカ法 2 回
 - 採用された辺の重みは, 頂点 0 が保持する
- **どうやって各頂点が保持している辺重みを合計する?**
- 縮約
 - **連結成分のサイズより連結成分数が多い!**

ブルーフカ法 + 縮約 + 初手 3 倍 : 6 ラウンド

どうやって各頂点が保持している辺重みを合計する？

	$i \neq k$	$i = k$
連結成分 i の k 番目の頂点が,	(連結成分 i) – (連結成分 k) 間の最小重みを放送	「採用された辺の重みの合計」を放送
そのために, (連結成分 i の k 番目の頂点) は,	(連結成分 k) の各頂点 j から, j – (連結成分 i) 間の最小重みを受け取る	いずれかの頂点から 「採用された辺の重みの合計」を受け取る

ブルーフカ法 + 縮約 + 初手 3 倍 : 6 ラウンド

どうやって各頂点が保持している辺重みを合計する？

	$i \neq k$	$i = k$
連結成分 i の k 番目の頂点が,	(連結成分 i) – (連結成分 k) 間の最小重みを放送	「採用された辺の重みの合計」を放送
そのために, (連結成分 i の k 番目の頂点) は,	(連結成分 k) の各頂点 j から, j – (連結成分 i) 間の最小重みを受け取る	各頂点から 「保持している辺重み」 を受け取る

ブルーフカ法 + 縮約 + 初手 3 倍 : 6 ラウンド

サイズ 12 の連結成分が最大 21 個あって、すべての情報を送れない... ?

	$i \neq k$	$i = k$
連結成分 i の k 番目の頂点が,	(連結成分 i) – (連結成分 k) 間の最小重みを放送	「採用された辺の重みの合計」を放送
そのために, (連結成分 i の k 番目の頂点) は,	(連結成分 k) の各頂点 j から, j – (連結成分 i) 間の最小重みを受け取る	各頂点から 「保持している辺重み」を受け取る

ブルーフカ法 + 縮約 + 初手 3 倍 : 6 ラウンド

サイズ 12 の連結成分が最大 21 個あって、すべての情報を送れない... ?

→ $\binom{21}{2} = 210 < 256$ だから頂点数は足りているはず !

	$i \neq k$	$i = k$
連結成分 i の k 番目の頂点が,	(連結成分 i) – (連結成分 k) 間の最小重みを放送	「採用された辺の重みの合計」を放送
そのために, (連結成分 i の k 番目の頂点) は,	(連結成分 k) の各頂点 j から, j – (連結成分 i) 間の最小重みを受け取る	各頂点から 「保持している辺重み」を受け取る

ブルーフ法 + 縮約 + 初手 3 倍 : 6 ラウンド

双方向に辺を送らないように, 連結成分 i は連結成分 $i \rightarrow i + \{1, 2, \dots, 10\} \pmod{N}$ の辺を放送!

	$k \neq 0$	$k = 0$
連結成分 i の k 番目の頂点が,	(連結成分 i) - (連結成分 $i + k$) 間の最小重みを放送	「採用された辺の重みの合計」を放送
そのために, (連結成分 i の k 番目の頂点) は,	(連結成分 $i + k$) の各頂点 j から, $j -$ (連結成分 i) 間の最小重みを受け取る	各頂点から 「保持している 辺重み」を受け取る

ブルーフカ法 + 縮約 + 初手 3 倍 : 6 ラウンド

- 初手 3 倍
 - 採用された 1st min の辺の重みは, その辺の一端が保持する
 - 採用された 2nd min の辺の重みは, 頂点 0 が保持する
- ブルーフカ法 2 回
 - 採用された辺の重みは, 頂点 0 が保持する
- 縮約 (2 ラウンド)
 - 連結成分 i は連結成分 $i \rightarrow i + \{1, 2, \dots, 10\} \pmod{N}$ の辺を放送

放送 5 回 = 6 ラウンドでできる！  100

おまけ : $O(\log \log N)$ ラウンド

実はちょっとの変更で $O(\log \log N)$ ラウンドになっている.

おまけ : $O(\log \log N)$ ラウンド

実はちょっとの変更で $O(\log \log N)$ ラウンドになっている.

	$k \neq 0$	$k = 0$
連結成分 i の k 番目の頂点が,	縮約後の, 連結成分 i から 出る k -th min の (重み, 行き先) を放送	「採用された辺の重みの 合計」を放送
そのために, (連結成分 i の k 番目の頂点) は,	各頂点 j から, $j -$ (連結成分 i) 間の最小重みを受け取る	各頂点から「保持している 辺重み」を受け取る

連結成分のサイズが x 以上なら, $(x - 1)$ -th min まで放送することができ, 連結成分のサイズを x 倍にできる !

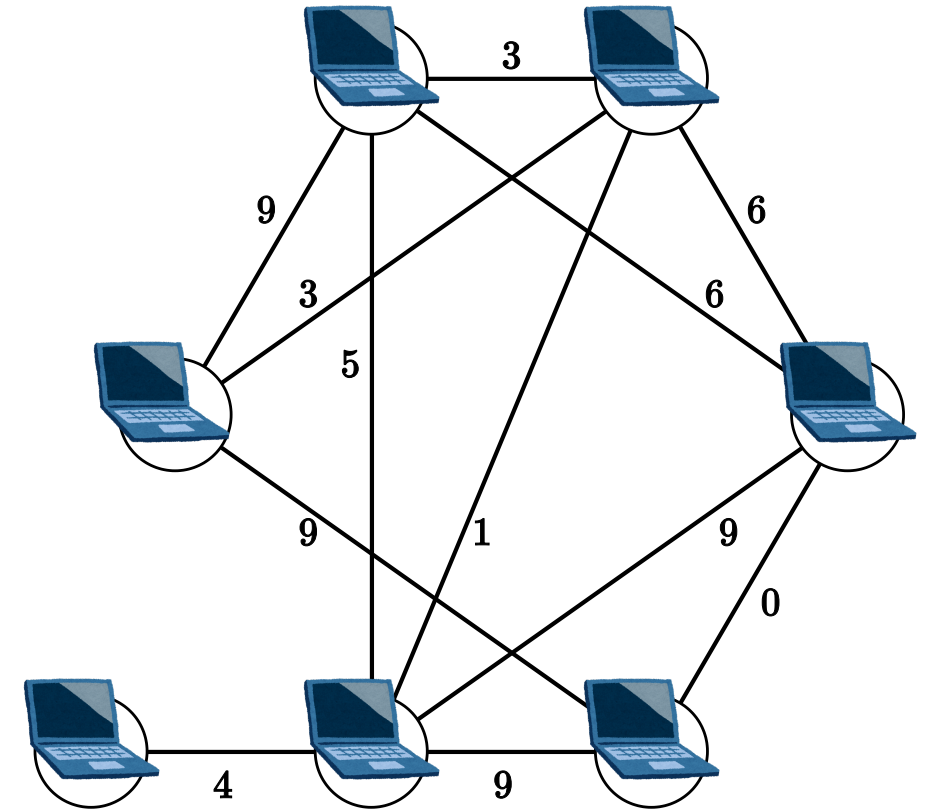
おまけ: $O(1)$ ラウンド

- 各頂点はいくらかでも情報を記憶できる
- 各辺は $O(\log N)$ bit の情報を送信できる

としたとき, 定数ラウンドで解けるらしい 🤔🤯 [Nowicki, 2021]

おまけ: CONGEST モデル

- 入力の無向グラフ = 計算ネットワークであるようなモデル
 - 頂点 = 計算機
 - 辺 = 通信路
- 各頂点は各ラウンドで多項式時間の計算ができ、いくらでも記憶することができる.
- 各辺は各ラウンドで $O(\log N)$ bit の通信を行うことができる.



参考資料: ネットワーク上の分散グラフアルゴリズムと最適化 – COSS2017

最小全域木が $O((\text{直径} + \sqrt{N}) \log N)$ ラウンドで解けるらしい [Elkin, 2020]

得点分布

得点	人数	累積
66	1	1
58	1	2
20	2	4
6	1	5
5	10	15
0	15	30



JOI/JOIG 2026 ファイナルステージ Day2 カジノ (Casino) 解説

解説: 蜂矢 倫久 (Mitsubachi)

Step 0

問題概要

0

問題概要

- 8 行 8 列のマス目があります
- ① A, B からなる L 文字の文字列 S が Azzurro に伝えられます
- ② Azzurro は各マス进行青か赤で塗ります
- ③ Chiaro がマス目の左上から右下までのパスを取って色を反転します
- ④ Bordeaux は長さ L と(反転後の)マス目を見て S を当てます

- 長さ L が長いほど点数が高い
- $L = 51$ で満点獲得

L の値	JOI	JOIG
$1 \leq L \leq 28$	2L 点 (2 ~ 56)	2.5L 点 (2 ~ 70)
$29 \leq L \leq 39$	L + 28 点 (57 ~ 67)	1.5L + 29 点 (72 ~ 87)
$40 \leq L \leq 50$	67 + 3(L - 40) 点 (67 ~ 97)	L + 49 点 (89 ~ 99)
$L = 51$	100 点	100 点

0

記法について

- 色について, 青 を 0 で, 赤 を 1 で表記します
- 青と 0, 赤と 1 を同一視するよということ

Step 1

多数派に着目

1

藍二乗（ヨルシカ）

- **変わらない風景** 浅い正午
- 高架下、藍二乗、寝転ぶまま
- 白紙の人生に拍手の音が一つ鳴っている
- 空っぽな自分を今日も歌っていた

1

藍二乗（ヨルシカ）

- **変わらない風景** 浅い正午
 - 高架下、藍二乗、寝転ぶまま
 - 白紙の人生に拍手の音が一つ鳴っている
 - 空っぽな自分を今日も歌っていた
-
- 操作をしても変わらないものがないか？

1

藍二乗（ヨルシカ）

- **変わらない風景** 浅い正午
 - 高架下、藍二乗、寝転ぶまま
 - 白紙の人生に拍手の音が一つ鳴っている
 - 空っぽな自分を今日も歌っていた
-
- 操作をしても変わらないものがないか？
 - よくあるのは**多数派**！

1

$$L = 1$$

- 例えば $L = 1$ を考える
- S は A か B のどちらか
- A なら全部青で, B なら全部赤で塗ってしまう
- Bordeaux の見るマス目は S が A ならほぼ青, B ならほぼ赤なので成功できる
- 2 点 (JOI) / 2 点 (JOIG)

1 $L = 1$ からの改善

- これは全体の多数派が変わらないよということだった
- 同じように多数派が変わらない場所に区切れないか考えてみる
- 多数派が変わりにくくするにはどうする？
- まず、最初は全部同じ色に塗ることが良さそう
- Chiaro により変わる場所ができるだけ少ないといいかも

1

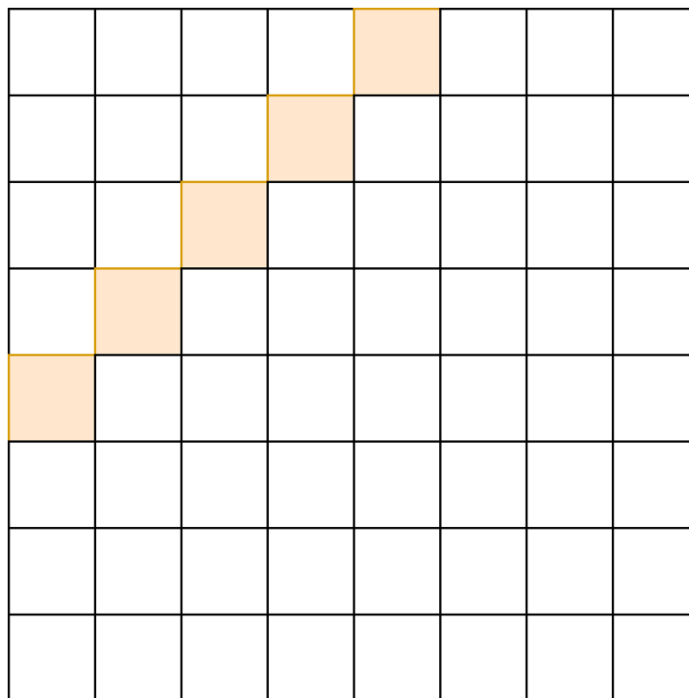
多数派を同じにしたい

- Chiaro により変わる場所ができるだけ少なくするには？
- どのパスを選んでも区切った場所を通る回数を少なくしたい
- パスは右下に向かってクネクネする
- ということは右上に向かって区切った場所を取ると良さそう

1

多数派を同じにしたい

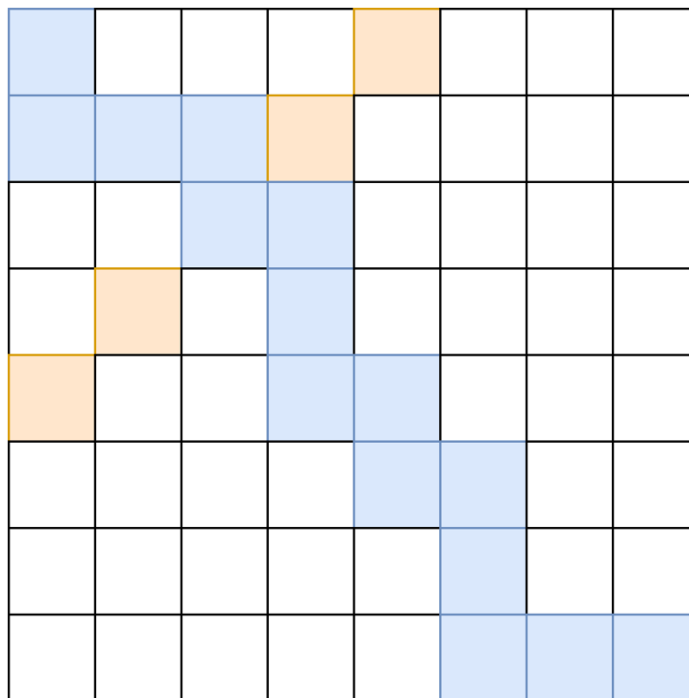
- ということは右上に向かって区切った場所を取ると良さそう
- 例えばこんな感じ



1

多数派を同じにしたい

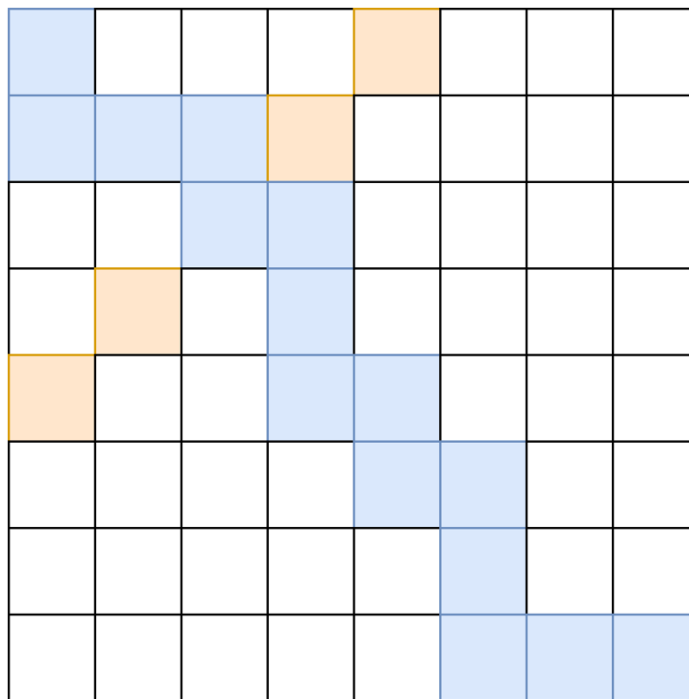
- このオレンジの 5 マスのうち通るのは 1 マス以下
- つまりこの多数派は変わらない！



1

多数派を同じにしたい

- このオレンジの 5 マスのうち通るのは 1 マス以下
- つまりこの多数派は変わらない！ 1 bit 伝えられる



1

多数派を同じにしたい

- これを対角線ごとにやると対角線 1 つで 1 bit
- 左上 3 マスと右下 3 マスは難しそう 合計で 11 bit
- 22 点 (JOI) / 27 点 (JOIG)

		1	2	3	4	5	6
	1	2	3	4	5	6	7
1	2	3	4	5	6	7	8
2	3	4	5	6	7	8	9
3	4	5	6	7	8	9	10
4	5	6	7	8	9	10	11
5	6	7	8	9	10	11	
6	7	8	9	10	11		

1

多数派

- この解法でキーになるのは、1つ変わっても多数派はそのままということ
- 加えて、Chiaro のパス上のマスにおいて、**行番号 + 列番号は 1 ずつ増える**ということ
- グリッドの 45 度回転というのと近いかも
- これは 3 個でできる
- 3 個ずつ区切れないか？
- 実はできる

1

3 つに改善

- 3 個ずつ区切ると 19 bit 達成できる
- 38 点 (JOI) / 47 点 (JOIG)

		1	3	4	6	9	11
	1	2	4	6	8	11	...
1	2	4	6	8	11	...	...
2	3	5	8	10	...	...	...
3	5	7	10	...	...	...	19
5	7	10	...	...	...	...	
7	9	...	...	...	...	19	
9	...	...	...	...	19		

1

ムダを探る

- 左上と右下で何もしていない！
- 何かできないか？

		1	3	4	6	9	11
	1	2	4	6	8	11	...
1	2	4	6	8	11	...	...
2	3	5	8	10	...	...	...
3	5	7	10	...	...	...	19
5	7	10	...	...	...	...	
7	9	...	...	...	...	19	
9	...	...	...	...	19		

1

ムダを探る

- $(0, 0)$ と $(7, 7)$ は絶対に通る
- なので Chiaro が操作する前にどうだったかが分かる
- 2 bit 追加で送ることができる 合計で 21 bit
- 42 点 (JOI) / 52 点 (JOIG)

20		1	3	4	6	9	11
	1	2	4	6	8	11	...
1	2	4	6	8	11	...	...
2	3	5	8	10	...	...	...
3	5	7	10	...	...	...	19
5	7	10	...	...	...	...	
7	9	...	...	...	...	19	
9	...	...	...	...	19		21

1

ムダを探る

- 20 の右下 2 マスおよび 21 の左上 2 マスでも何か伝えたい
- 2 マスのうちちょうど 1 つを通る
- じゃあ 1 つを 0 で固定してもう 1 つに情報を書けばいい？

20	22	1	3	4	6	9	11
	1	2	4	6	8	11	...
1	2	4	6	8	11	...	...
2	3	5	8	10	...	...	...
3	5	7	10	...	...	...	19
5	7	10	...	...	...	...	
7	9	...	...	...	...	19	23
9	...	...	...	...	19		21

1

ムダを探る

- じゃあ 1 つを 0 で固定してもう 1 つに情報を書けばいい？
- 下のように Azzurro は書く

20	22	1	3	4	6	9	11
	1	2	4	6	8	11	...
1	2	4	6	8	11	...	...
2	3	5	8	10	...	...	...
3	5	7	10	...	...	...	19
5	7	10	...	...	...	...	
7	9	...	...	...	...	19	23
9	...	...	...	...	19		21

1

ムダを探る

- Bordeaux は下のものが送られてきたとする
- $(1, 0)$ が青 $\rightarrow$ パスは $(0, 1)$ を通って 22 を反転させた
- $(7, 6)$ が赤 $\rightarrow$ パスは $(7, 6)$ を通って 23 を反転させていない

20	22	1	3	4	6	9	11
	1	2	4	6	8	11	...
1	2	4	6	8	11	...	...
2	3	5	8	10	...	...	...
3	5	7	10	...	...	...	19
5	7	10	...	...	...	...	
7	9	...	...	...	...	19	23
9	...	...	...	...	19		21

1

ムダを探る

- これで OK
- 2 bit 追加で送ることができる 合計で 23 bit
- 46 点 (JOI) / 57 点 (JOIG)

20	22	1	3	4	6	9	11
	1	2	4	6	8	11	...
1	2	4	6	8	11	...	...
2	3	5	8	10	...	...	...
3	5	7	10	...	...	...	19
5	7	10	...	...	...	...	
7	9	...	...	...	...	19	23
9	...	...	...	...	19		21

Step 2

複数回書く

2

だから僕は音楽を辞めた（ヨルシカ）

- 僕だって信念があった
- 今じゃ塵みたいな想いだ
- **何度でも君を書いた**
- 売れることこそがどうでもよかったんだ
- 本当だ 本当なんだ 昔はそうだった

2

だから僕は音楽を辞めた（ヨルシカ）

- 僕だって信念があった
 - 今じゃ塵みたいな想いだ
 - **何度でも君を書いた**
 - 売れることこそがどうでもよかったんだ
 - 本当だ 本当なんだ 昔はそうだった
-
- 同じことを何回か書いて、どれかから復元できないか？
 - 多数派と同じ考えとも捉えることができる

2

複数回書く

- 同じことを何回か書いて、どれかから復元できないか？
- 多数派と同じ考えとも捉えることができる
- 先ほどは 3 回同じことを書いた なので $64 / 3$ bit ぐらい
- 2 回でできないか？

2 $L = 16$

- 左下と右上の 4 行 4 列の部分に同じものを書いてみよう
- 周りは青で埋めておく

				1	2	3	4
				5	6	7	8
				9	10	11	12
				13	14	15	16
1	2	3	4				
5	6	7	8				
9	10	11	12				
13	14	15	16				

2 L = 16

- Bordeaux はこんな感じの盤面を見る
- 右上は通っていないことがわかる

				1	2	3	4
				5	6	7	8
				9	10	11	12
				13	14	15	16
1	2	3	4				
5	6	7	8				
9	10	11	12				
13	14	15	16				

2 L = 16

- こんな感じのときはややこしい
- (3, 4) (= 右上の 13) か (4, 3) (= 左下の 4) のどちらだろう？

				1	2	3	4
				5	6	7	8
				9	10	11	12
				13	14	15	16
1	2	3	4				
5	6	7	8				
9	10	11	12				
13	14	15	16				

2 L = 16

- (3, 4) (= 右上の 13) か (4, 3) (= 左下の 4) のどちらだろう？
- 4, 13 のどちらが左下と右上で異なるかを見ればわかる！
- どんなパスでも左下と右上のうち通っていない方がわかる

				1	2	3	4
				5	6	7	8
				9	10	11	12
				13	14	15	16
1	2	3	4				
5	6	7	8				
9	10	11	12				
13	14	15	16				

2 L = 16

- どんなパスでも左下と右上のうち通っていない方がわかる
- 16 bit 伝えられる
- 32 点 (JOI) / 40 点 (JOIG)

				1	2	3	4
				5	6	7	8
				9	10	11	12
				13	14	15	16
1	2	3	4				
5	6	7	8				
9	10	11	12				
13	14	15	16				

2 L = 22

- Step 1 の改善を入れるとこんな感じにできる
- 22 bit 伝えられる
- 44 点 (JOI) / 55 点 (JOIG)

19	21	17		1	2	3	4
	17			5	6	7	8
17				9	10	11	12
				13	14	15	16
1	2	3	4				
5	6	7	8				18
9	10	11	12			18	22
13	14	15	16		18		20

2

さらに強い形

- さっきのは左上から右下までの対角線で対称という感じだった
- よりそれを強くしてみるとこんな感じ

22		1	2	4	7	11	16
	23		3	5	8	12	17
1		24		6	9	13	18
2	3		25		10	14	19
4	5	6		26		15	20
7	8	9	10		27		21
11	12	13	14	15		28	
16	17	18	19	20	21		29

2

さらに強い形

- Bordeaux の復元を考える
- 下のときは間の 23 を通ったんだなとわかる

22		1	2	4	7	11	16
	23		3	5	8	12	17
1		24		6	9	13	18
2	3		25		10	14	19
4	5	6		26		15	20
7	8	9	10		27		21
11	12	13	14	15		28	
16	17	18	19	20	21		29

2

さらに強い形

- Bordeaux の復元を考える
- 下のときは間の 23 を通ったんだなとわかる

22		1	2	4	7	11	16
	23		3	5	8	12	17
1		24		6	9	13	18
2	3		25		10	14	19
4	5	6		26		15	20
7	8	9	10		27		21
11	12	13	14	15		28	
16	17	18	19	20	21		29

2

さらに強い形

- Bordeaux の復元を考える
- 下のときは 1 から 6 は左下を参考にすれば良い

22		1	2	4	7	11	16
	23		3	5	8	12	17
1		24		6	9	13	18
2	3		25		10	14	19
4	5	6		26		15	20
7	8	9	10		27		21
11	12	13	14	15		28	
16	17	18	19	20	21		29

2

さらに強い形

- Bordeaux の復元を考える
- 下のときは $(0, 2)$ (= 右上の 1) を通ったか $(1, 1)$ (= 23) を通ったか分からなさそう

22		1	2	4	7	11	16
	23		3	5	8	12	17
1		24		6	9	13	18
2	3		25		10	14	19
4	5	6		26		15	20
7	8	9	10		27		21
11	12	13	14	15		28	
16	17	18	19	20	21		29

2

さらに強い形

- 下のときは $(0, 2)$ (= 右上の 1) を通ったか $(1, 1)$ (= 23) を通ったか分からなさそう
- 右上の 1 と左下の 1 が同じか見れば分かる

22		1	2	4	7	11	16
	23		3	5	8	12	17
1		24		6	9	13	18
2	3		25		10	14	19
4	5	6		26		15	20
7	8	9	10		27		21
11	12	13	14	15		28	
16	17	18	19	20	21		29

2

さらに強い形

- これで，対角線については通ったか / 通っていないか分かる
- それ以外もどっちを見れば正しいか分かる
- 29 bit 伝えられる
- 57 点 (JOI) / 72 点 (JOIG)

22		1	2	4	7	11	16
	23		3	5	8	12	17
1		24		6	9	13	18
2	3		25		10	14	19
4	5	6		26		15	20
7	8	9	10		27		21
11	12	13	14	15		28	
16	17	18	19	20	21		29

2

この手法の限界

- Step 1 の **3** マスに同じことを書くを **2** マスに同じことを書くに改善した
- おそらくこの手法の限界は $64 / 2 = 32$ bit ぐらいだろう
- 満点の $L = 51$ にはだいぶ遠い

2

よく考え直してみる

- Azzurro と Bordeaux が正しい戦略を持っているとする
- Bordeaux は Azzurro の見た S が理解できた
- ということは Azzurro が書いた盤面も分かる
- 差分を取ると Bordeaux は Chiaro の選んだパスが分かる

2

よく考え直してみる

- Azzurro と Bordeaux が正しい戦略を持っているとする
- 差分を取ると Bordeaux は Chiaro の選んだパスが分かる
- 逆に, Bordeaux がパスを知ることができれば ... ?
- Azzurro の書いた盤面がわかって, 逆算で S を求められそう

2

今後の方針

- Bordeaux は Azzurro の選んだ S を特定するにあたり，最初に **Chiaro の選んだパス** を特定することを目指そう！

Step 3

パスを特定して，満点へ

3 Step 1 の復習

- 最後にこんな盤面が出てきた

20	22	1	3	4	6	9	11
	1	2	4	6	8	11	...
1	2	4	6	8	11	...	...
2	3	5	8	10	...	...	...
3	5	7	10	...	...	...	19
5	7	10	...	...	...	...	
7	9	...	...	...	...	19	23
9	...	...	...	...	19		21

3

Step 1 の復習

- この盤面を使えば、パスが $(0, 0)$ の次にどっちに行ったかが分かる
- $(0, 1)$ と $(1, 0)$ のどっちを通ったかな？ということ

20	22	1	3	4	6	9	11
	1	2	4	6	8	11	...
1	2	4	6	8	11	...	...
2	3	5	8	10	...	...	...
3	5	7	10	...	...	...	19
5	7	10	...	...	...	...	
7	9	...	...	...	...	19	23
9	...	...	...	...	19		21

3 Step 1 の復習

- $(0, 1)$ と $(1, 0)$ のどっちを通ったかが分かる
- その次に下と右のどっちに進んだかを知りたい
- 今回なら 1 の 3 マスを見ればいいが、他の方法はないか？

20	22	1	3	4	6	9	11
	1	2	4	6	8	11	...
1	2	4	6	8	11	...	...
2	3	5	8	10	...	...	...
3	5	7	10	...	...	...	19
5	7	10	...	...	...	...	
7	9	...	...	...	...	19	23
9	...	...	...	...	19		21

3

一般化

- 一般化する
- パスがマス (r, c) を通りましたと分かっている
- 次に進んだのは $(r + 1, c)$ か $(r, c + 1)$ のどちらか
- 下に進んだ or 右に進んだとも解釈できる
- これを多くの (r, c) で分かるようにできないか？

3

対角線に着目

- $(r + 1, c)$ と $(r, c + 1)$ は同じ対角線上で隣り合う 2 マス
- 隣り合う $\rightarrow$ 対角線上の偶奇が異なる
- ある対角線の偶数番目や奇数番目を集めて何かできないか？

3

対角線に着目

- $(r + 1, c)$ と $(r, c + 1)$ は同じ対角線上で隣り合う 2 マス
- 隣り合う $\rightarrow$ 対角線上の偶奇が異なる
- ある対角線の偶数番目や奇数番目を集めて何かできないか？
- 実はできる！
- **チェックサム** という概念を使う

3

チェックサム

- $(r + 1, c)$ と $(r, c + 1)$ は同じ対角線上で隣り合う 2 マス
- 隣り合う $\rightarrow$ 対角線上の偶奇が異なる
- ある対角線の偶数番目や奇数番目を集めて何かできないか？
- 実はできる！
- **チェックサム** という概念を使う

3

チェックサム

- ある対角線の偶数番目や奇数番目を集めて何かできないか？
- **チェックサム** という概念を使う
- 例えば下の 3 の対角線に着目

		1	2	3	4	5	6
	1	2	3	4	5	6	7
1	2	3	4	5	6	7	8
2	3	4	5	6	7	8	9
3	4	5	6	7	8	9	10
4	5	6	7	8	9	10	11
5	6	7	8	9	10	11	
6	7	8	9	10	11		

3

チェックサム

- 3 のマスに書かれた 1 の数を偶数にする
- 3' についても同様
- XOR を知っていれば, XOR を 0 にするともみなせる

		1	2	3	4	5	6
	1	2	3'	4	5	6	7
1	2	3	4	5	6	7	8
2	3'	4	5	6	7	8	9
3	4	5	6	7	8	9	10
4	5	6	7	8	9	10	11
5	6	7	8	9	10	11	
6	7	8	9	10	11		

3

チェックサム

- 3 と 3' から 1 個ずつとってきて，そこで調整すればいい
- 他の 3 と 3' は自由に書くことができる

		1	2	3	4	5	6
	1	2	3'	4	5	6	7
1	2	3	4	5	6	7	8
2	3'	4	5	6	7	8	9
3	4	5	6	7	8	9	10
4	5	6	7	8	9	10	11
5	6	7	8	9	10	11	
6	7	8	9	10	11		

3

チェックサム

- もしパスが 3 のマスを通ったら？ → Bordeaux のマスにおいて 3 のマスに書かれた 1 の数は奇数になり， 3' においては偶数になる
- 3' の場合も逆も然り → 1 の数が**奇数**の方を通ったといえる！

		1	2	3	4	5	6
	1	2	3'	4	5	6	7
1	2	3	4	5	6	7	8
2	3'	4	5	6	7	8	9
3	4	5	6	7	8	9	10
4	5	6	7	8	9	10	11
5	6	7	8	9	10	11	
6	7	8	9	10	11		

3

チェックサム

- 結局 3 を通ったか 3' を通ったかは特定できる
- 2 の対角線においてどこを通ったか分かっているとする
- 例えば (1, 2) としよう

		1	2	3	4	5	6
	1	2	3'	4	5	6	7
1	2	3	4	5	6	7	8
2	3'	4	5	6	7	8	9
3	4	5	6	7	8	9	10
4	5	6	7	8	9	10	11
5	6	7	8	9	10	11	
6	7	8	9	10	11		

3

チェックサム

- 例えば (1, 2) としよう 次は (2, 2) か (1, 3) の 2 択
- ちょうど 1 つは 3 のマスで, もう 1 つは 3' のマス
- 3 を通ったか 3' を通ったかは特定できていた

		1	2	3	4	5	6
	1	2	3'	4	5	6	7
1	2	3	4	5	6	7	8
2	3'	4	5	6	7	8	9
3	4	5	6	7	8	9	10
4	5	6	7	8	9	10	11
5	6	7	8	9	10	11	
6	7	8	9	10	11		

3

チェックサム

- ちょうど 1 つは 3 のマスで，もう 1 つは 3' のマス
- 3 を通ったか 3' を通ったかは特定できていた
- → パスは次にどのマスを通ったか特定できる！

		1	2	3	4	5	6
	1	2	3'	4	5	6	7
1	2	3	4	5	6	7	8
2	3'	4	5	6	7	8	9
3	4	5	6	7	8	9	10
4	5	6	7	8	9	10	11
5	6	7	8	9	10	11	
6	7	8	9	10	11		

3

チェックサム

- これを全ての対角線について行う
- 右上の 6 行 6 列は自由で，濃い色のところで調整するイメージ
- Bordeaux は 1 の対角線から順に行えばパスの特定ができる

		1	2'	3	4'	5	6'
	1'	2	3'	4	5'	6	7
1	2'	3	4'	5	6'	7'	8'
2	3'	4	5'	6	7	8	9
3	4'	5	6'	7'	8'	9'	10'
4	5'	6	7	8	9	10	11
5	6'	7'	8'	9'	10'	11'	
6	7	8	9	10	11		

3

チェックサム

- パスが分かったので Bordeaux は Azzurro が右上の 6 行 6 列に何を書いたかわかる
- 36 bit 伝えられる
- 64 点 (JOI) / 83 点 (JOIG)

		1	2'	3	4'	5	6'
	1'	2	3'	4	5'	6	7
1	2'	3	4'	5	6'	7'	8'
2	3'	4	5'	6	7	8	9
3	4'	5	6'	7'	8'	9'	10'
4	5'	6	7	8	9	10	11
5	6'	7'	8'	9'	10'	11'	
6	7	8	9	10	11		

3

微改善

- 左上と右下の 3 マスは Step 1 の手法で 2 bit ずつ伝えられる
- 合計 40 bit 伝えられる
- 67 点 (JOI) / 89 点 (JOIG)

		1	2'	3	4'	5	6'
	1'	2	3'	4	5'	6	7
1	2'	3	4'	5	6'	7'	8'
2	3'	4	5'	6	7	8	9
3	4'	5	6'	7'	8'	9'	10'
4	5'	6	7	8	9	10	11
5	6'	7'	8'	9'	10'	11'	
6	7	8	9	10	11		

3

そして満点へ

- 実は $1', 2', \dots$ の濃いマスにも自由に書ける
- $1', 2', \dots$ のマスに書かれた 1 の数の情報が消えたけど ... ?

		1	2'	3	4'	5	6'
	1'	2	3'	4	5'	6	7
1	2'	3	4'	5	6'	7'	8'
2	3'	4	5'	6	7	8	9
3	4'	5	6'	7'	8'	9'	10'
4	5'	6	7	8	9	10	11
5	6'	7'	8'	9'	10'	11'	
6	7	8	9	10	11		

3

そして満点へ

- もしパスが 3 のマスを通ったら？ → Bordeaux のマスにおいて 3 のマスに書かれた 1 の数は**奇数**になる
- もしパスが 3' のマスを通ったら？ → Bordeaux のマスにおいて 3 のマスに書かれた 1 の数は**偶数**になる

		1	2'	3	4'	5	6'
	1'	2	3'	4	5'	6	7
1	2'	3	4'	5	6'	7'	8'
2	3'	4	5'	6	7	8	9
3	4'	5	6'	7'	8'	9'	10'
4	5'	6	7	8	9	10	11
5	6'	7'	8'	9'	10'	11'	
6	7	8	9	10	11		

3

そして満点へ

- 結局この形でも 3 か 3' を通ったか分かる！
- なので同様にパスの特定ができて、解ける！

		1	2'	3	4'	5	6'
	1'	2	3'	4	5'	6	7
1	2'	3	4'	5	6'	7'	8'
2	3'	4	5'	6	7	8	9
3	4'	5	6'	7'	8'	9'	10'
4	5'	6	7	8	9	10	11
5	6'	7'	8'	9'	10'	11'	
6	7	8	9	10	11		

3

そして満点へ

- 色が薄い分の 47 bit + 左上と右下の 4 bit = 51 bit 伝えられる
- 100 点 (JOI) / 100 点 (JOIG)
- 満点！

		1	2'	3	4'	5	6'
	1'	2	3'	4	5'	6	7
1	2'	3	4'	5	6'	7'	8'
2	3'	4	5'	6	7	8	9
3	4'	5	6'	7'	8'	9'	10'
4	5'	6	7	8	9	10	11
5	6'	7'	8'	9'	10'	11'	
6	7	8	9	10	11		

3

チェックサムの補足

- チェックサムについて
- 現実社会でもよく使われています
- 例えばファイルの一致確認（ハッシュと言ったほうがいいかも）

3

チェックサムの補足

- 他にはファミコンや DS 時代のゲームのカセットなんかにも使われています
- ゲームのデータをの表す 0 や 1 について総和を取って保存しておくことでセーブデータが破損しているかチェックする仕組み
- FF6 だと総和を 2 byte 分 (0 ~ 65535) で保存している
- 空のデータは中身が全部 0 なのでチェックサムも 0

3

チェックサムの補足

- FF6 だと総和を 2 byte 分 (0 ~ 65535) で保存している
- 空のデータは中身が全部 0 なのでチェックサムも 0
- なので、セーブしたときに偶然 0 になると破損扱いになる
- セーブするたびに $1 / 65536$ の確率でデータが消失！

Step 4

おまけ & 得点分布

4

おまけ

- ところでこのスライドのタイトル画像はなにか覚えていますか

4

おまけ

- 麻雀です
- これは何点でしょうか（ツモ上がり，一発なし）



4

おまけ

- これは **5200** 点です
- ところで $L \geq 52$ というのは解けるのでしょうか



4

おまけ

- まず, Chiaro の選ぶパスは ${}_{14}C_7 = 3432$ 通りあります
- Azzurro が塗りうる盤面を X 通りとします
- Azzurro が塗りうる盤面を 2 つ選んだとき, Bordeaux が見る盤面は必ず異なる必要があります
- すると $3432X \leq 2^{64}$ である必要があります

4

おまけ

- すると $3432X \leq 2^{64}$ である必要があります
- $2^{11} = 2048 < 3432$ に着目すると
- $X < 2^{53}$ という評価ができます
- したがって, $L \geq 53$ では解けないということが言えます
- $L = 51$ では解けることは先ほどまで解説しました

4

おまけ

- すると $3432X \leq 2^{64}$ である必要があります
- $2^{11} = 2048 < 3432$ に着目すると
- $X < 2^{53}$ という評価ができます
- したがって, $L \geq 53$ では解けないということが言えます
- $L = 51$ では解けることは先ほどまで解説しました
- 間の **$L = 52$** は？

4

おまけ

- 間の **$L = 52$** は？
- 私は解説を書きながら考えたりコードをいくつか書いてみましたが、分かりませんでした
- 不可能か可能かも分かっていません
- また、今回の思考を考えると N 行 N 列なら $L = (N - 1)^2 + 2$ が可能ということにはなっています
- N を増やせばより強い評価があるかも分かっていません

4

おまけ

- また、今回の思考を考えると N 行 N 列なら $L = (N - 1)^2 + 2$ が可能ということにはなっています
- N を増やせばより強い評価があるかも知っていません
- 例えば $L = N^2 - 2N + O(\log N)$ の戦略があるかもしれません

4

おまけ

- このような話が好きな方は**情報量**や**情報理論**などを勉強すると楽しいかもしれません

4

得点分布 (JOI)

- 理想:
 - 100 点 : 30 人
-
- 選抜という意味では差がつかないな

4 得点分布 (JOIG)

- 理想:
 - 100 点 : 15 人
-
- 選抜という意味では差がつかないな

JOI ツアー 2 (joitour)

2026/03/22 春 Day 2

解説：保坂 結 (hos.lyric)

問題概要

- N 頂点の木
 - 頂点に値段 $A_u \in [1, N]$ が書いてある
- M 本のパス
- Q 通りの予算 $B_q \in [1, 2N]$
- 各予算について、以下のような選び方を数えよ
 - パスを選ぶ
 - $\rightarrow$ パス上の異なる 2 頂点 u, v を選ぶ
 - $\rightarrow$ 値段の和がちょうど予算になっている ($A_u + A_v = B_q$)
- $N \leq 100\,000$, $M \leq 200\,000$, $Q \leq 2\,000$

小課題 1 ($N, M, Q \leq 100$)

- 全探索
 - M 本のパスについて, 通る頂点を列挙
 - 頂点の 2 つ組を列挙
 - 予算を添え字とする配列に記録
 - $O(N^2M + Q)$ 時間

小課題 2, 3 ($N \leq 5\,000$)

- 頂点の 2 つ組を全列挙できる
- 2 頂点 u, v を両方通るパスを高速に数えられればよい
 - パスグラフのとき
 - $u < v$ として, 左端 $\in [1, u]$, 右端 $\in [v, N]$ という条件
 - 2 次元累積和
 - 一般の木するとき
 - Euler tour で長方形クエリ ≤ 2 回にできる
- $O(N^2 + M + Q)$ 時間 $\cdot O(N^2 + M + Q)$ 空間

小課題 4, 5 ($Q = 1$)

- 予算 B が決まっている
- ひとまず, パス 1 本あたり $O(N)$ で解くには?

パス上の各頂点 u に対し:

```
answer += count[B - A[u]]
```

```
count[A[u]] += 1
```

- 1 頂点追加したとき答えの差分を $O(1)$ 時間で計算できる
- 1 頂点削除もできる (逆に操作するだけ)

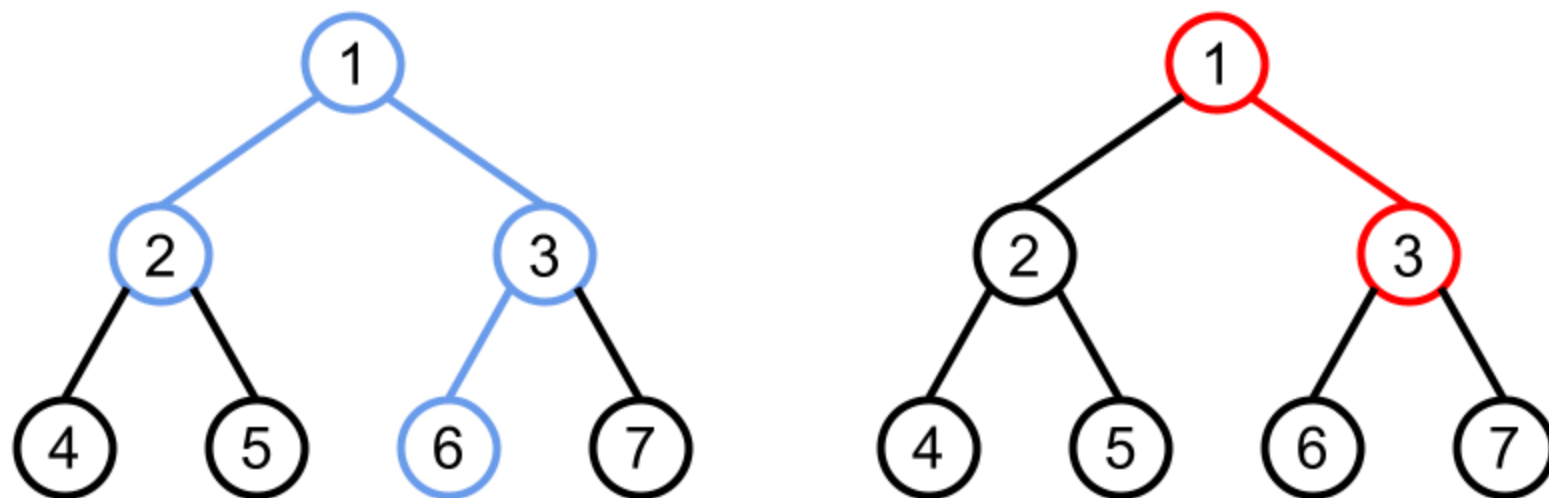
小課題 4 ($Q = 1$, パスグラフ)

- 列なら **Mo** のアルゴリズムが適用可能
 - 列をブロックサイズ L ごとに分ける
 - M 個の区間を (左端の属するブロック, 右端) でソート
 - 区間の伸縮回数が左 $O(ML)$ ・ 右 $O(N^2/L)$
 - $L = \Theta(N/\sqrt{M})$ とすると伸縮回数が $O(N\sqrt{M})$
- 「現在の区間に対する答え」を持って更新していくことで、各区間についての答えが求まり、それを合計して出力
- $O(N\sqrt{M} + M)$ 時間 ・ $O(N + M)$ 空間

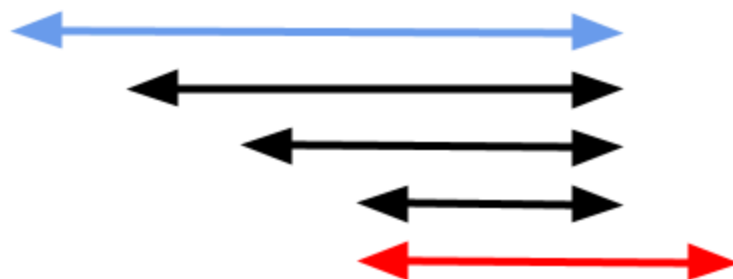
小課題 5 ($Q = 1$)

- 木のパスにも Mo が応用可能
- **Euler tour** (頂点列) $u_0, \dots, u_{2N-2}$ をとる (u_i と u_{i+1} が隣接)
- パス $s-t$ を, $(u_l, u_r) = (s, t)$ または $(u_l, u_r) = (t, s)$ である区間 $[l, r]$ に対応させる
 - 1 通りとは限らないがどれでもよい
- 区間の伸縮 $\longleftrightarrow$ パスの端点を隣に動かす $\longleftrightarrow$ パスの伸縮
 - 区間を伸ばしてもパスが縮むこと (やその逆) はある
- 対応させた区間を Mo の順にソートすれば, パスの伸縮回数が $O(N\sqrt{M})$ になる

小課題 5 ($Q = 1$)

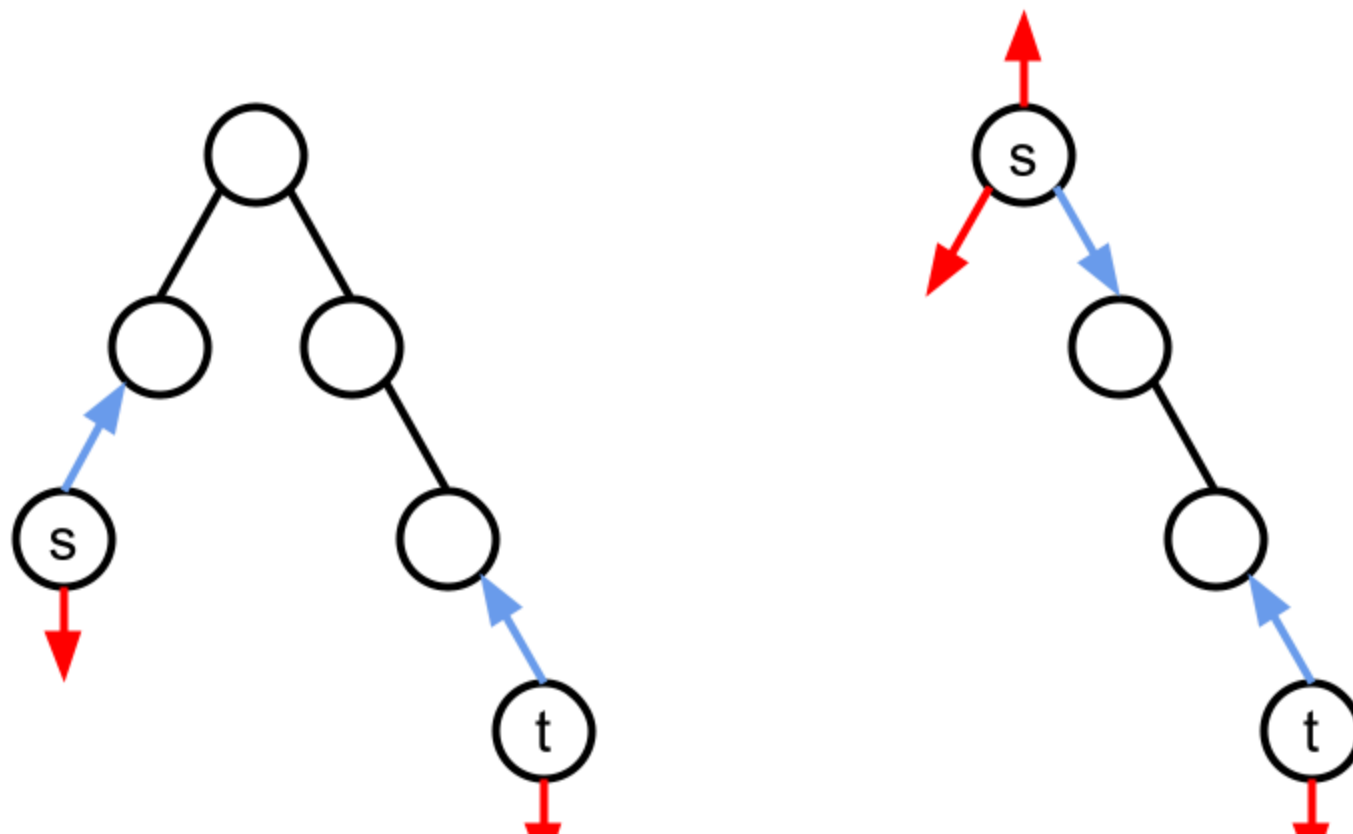


1 2 4 2 5 2 1 3 6 3 7 3 1



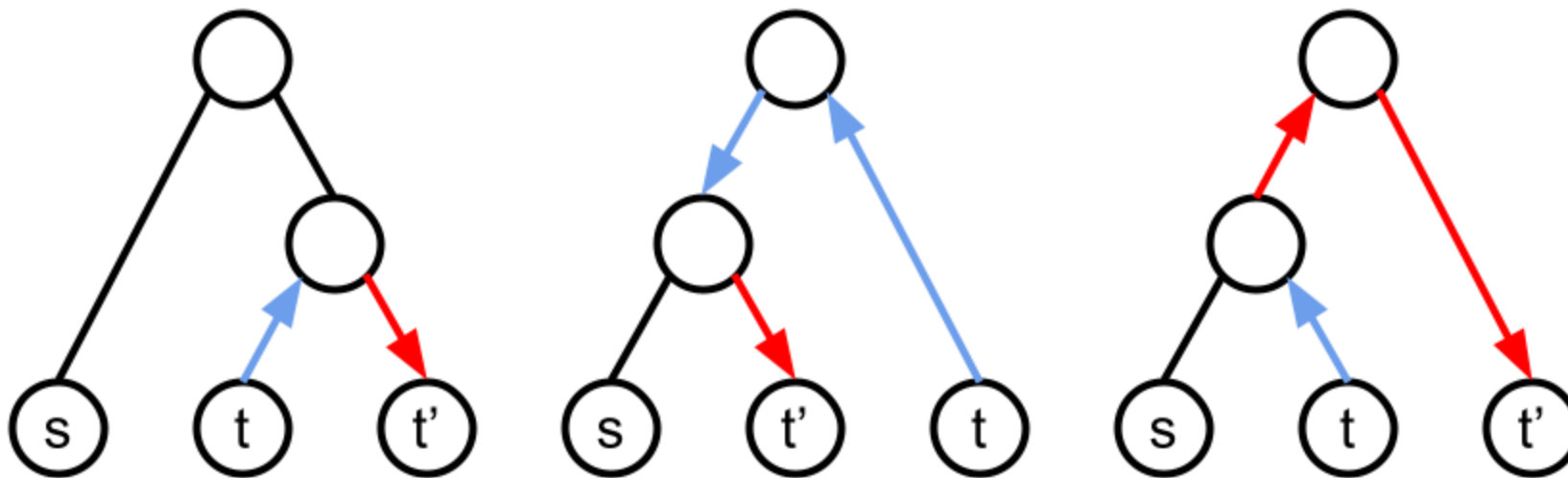
小課題 5 ($Q = 1$)

- パスの伸縮を管理する方法 (その 1)
 - 先祖・子孫関係にあるかを $O(1)$ 時間で判定できるとよい
 - Euler tour で最初と最後の登場を記録するとできる



小課題 5 ($Q = 1$)

- パスの伸縮を管理する方法 (その 2)
 - Euler tour 上の区間を 1 ずつ伸縮させなくてもよい
 - パスを $s-t$ から $s-t'$ にするとき, t から t' への最短路に沿って動かす
 - $\text{LCA}(s, t), \text{LCA}(s, t'), \text{LCA}(t, t')$ の一致関係で場合分け



考察整理タイム

別バージョンの問題

- 入力
 - N 頂点の木, 頂点重み A_u
 - M 本のパス
 - 重み $W[1], \dots, W[2N]$, そのうち非 0 なのは Q か所 $W[B_q]$
- 出力
 - パスを選んでパス上の異なる 2 頂点 u, v を選ぶ方法について, $W[A_u + A_v]$ の合計 (値 1 個だけ)

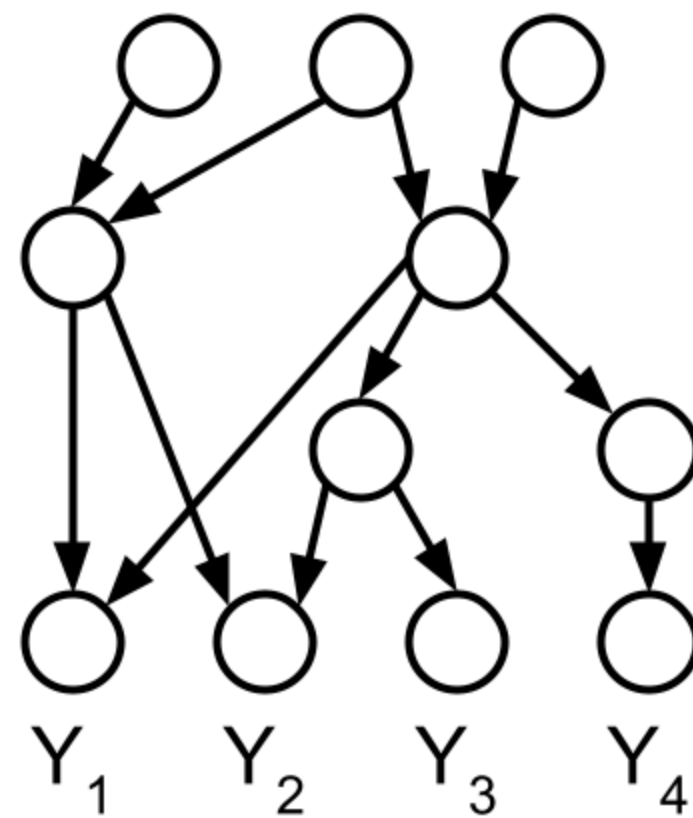
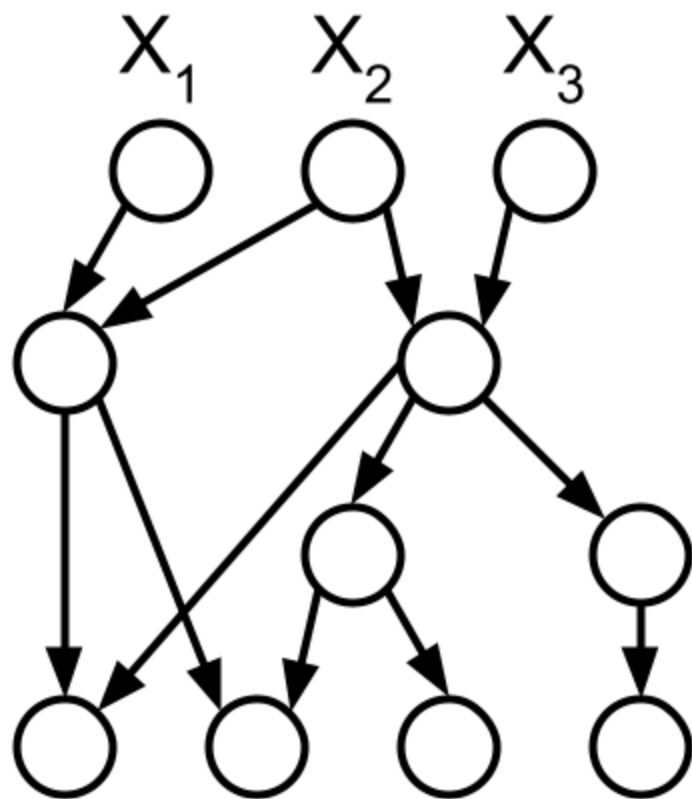
考察整理タイム

- 何がうれしいの？
- **転置原理**によって、**別バージョンの問題**が解ければ、元の問題の(計算量がほぼ同じ) 解法が機械的に得られることが示せる
- 「本質的な難しさ」みたいなものは変わらないですが、本問では、人によっては**別バージョンの問題**のほうが少し考えやすいと思います
- この問題の満点解法を理解するのに必須ではありませんが、考察テクニックのひとつとして紹介します

転置原理

- 辺重み付き DAG が与えられる
- 頂点 $S_1, \dots, S_P$ は入次数 0 (始点)
- 頂点 $T_1, \dots, T_Q$ は出次数 0 (終点)
- S_p から T_q へのパスの重みの合計を $C_{p,q}$ とする
 - パスの重み：通る辺の重みの積
- 問 1： $X_1, \dots, X_P$ が与えられる． 各 q に対し $\sum_{p=1}^P X_p C_{p,q}$ を求めよ
- 問 2： $Y_1, \dots, Y_Q$ が与えられる． 各 p に対し $\sum_{q=1}^Q C_{p,q} Y_q$ を求めよ

転置原理



転置原理

- **転置原理**：問 1 と問 2 の関係にある問題はほぼ同じ計算量で解ける
 - 片方の問題だけ空間計算量が節約できてしまうことはある
- 問 1 の解法：始点側から DP
- 問 2 の解法：終点側から DP
- 解法の変換
 - 同じ変数を用意する
 - 入力以外を 0 で初期化する
 - 手順を逆向きに見ていく
 - 変数 f , g と係数 c に対して, $g += c * f$ を $f += c * g$ に変換

転置原理

- 典型的な適用例： $\text{ans}_1, \dots, \text{ans}_Q$ を求めよ，という形の問題
- 重み X が与えられて， $X \cdot \text{ans}_1, \dots, X \cdot \text{ans}_Q$ を求めよ，という問題とみなす
- **転置すると**：重み $Y_1, \dots, Y_Q$ が与えられて， $\sum_{q=1}^Q \text{ans}_q \cdot Y_q$ を求めよ
- 実際にアルゴリズムを変換するには
 - `g += c * f` の形のステップに分解して逆に辿って変換する (大変)
 - 何段階かに区切って，↑のイメージを持つことで触る変数や計算量を意識しながら，意味を考えて解く (おすすめ)

転置原理

- ある意味絶対に「必要」にならないテクニック
- 問題を変換して、簡単に思えることもそうでないこともあります
- しかし解けない問題に帰着してしまうことがないので、割と考え得
- 多項式関連の問題や線型代数を学ぶと深く理解できると思いますが、発想自体は情報オリンピックの範囲でも有用です
- 参考資料
 - [Alin Bostan, Grégoire Lecerf, and Éric Schost. "Tellegen's principle into practice."](#)
 - [転置原理まとめ | Mathenachia](#)
 - "転置原理" で web 検索

考察整理タイム

転置した問題 (再掲)

- 入力
 - N 頂点の木, 頂点重み A_u
 - M 本のパス
 - 重み $W[1], \dots, W[2N]$, そのうち非 0 なのは Q か所 $W[B_q]$
- 出力
 - パスを選んでパス上の異なる 2 頂点 u, v を選ぶ方法について, $W[A_u + A_v]$ の合計 (値 1 個だけ)

考察整理タイム

転置した問題

- 頂点集合 U, V に対し，値 $U * V$ を次で定める
 - $u \in U$ と $v \in V$ の組に対する $W[A_u + A_v]$ の合計
- 性質
 - $(U \pm U') * V = (U * V) \pm (U' * V)$
 - $U * (V \pm V') = (U * V) \pm (U * V')$
 - 集合の $\pm$ が適切に定義できるとき

小課題 6 ($M \leq 1\,000$, パスグラフ) 転置

- 区間 $[l, r)$ に対する答えを以下の和に分解 (小課題 4, 5 でやった発想)
 - $[l, l) * \{l\}$
 - $[l, l + 1) * \{l + 1\}$
 - $\vdots$
 - $[l, r - 1) * \{r - 1\}$
- 以降頂点の添え字を 0 から始めます
- $[l, r) * \{v\} = [0, r) * \{v\} - [0, l) * \{v\}$ を用いて, $[0, u) * \{v\}$ という形のもの $O(NM)$ 個になる

小課題 6 ($M \leq 1\,000$, パスグラフ) 転置

- 分解した $[0, u) * \{v\}$ たちを u の昇順に見ていくことにする
- 目標
 - 各 u に対して, $O(Q)$ 時間くらいかけて何か準備していい
 - 各 u に対していろいろな v が来たとき, $O(1)$ 時間くらいで答えたい
- 「 $A_v = a$ のときの $[0, u) * \{v\}$ 」を各 $a \in [1, N]$ について持っておけたらうれしい
- それは $[0, u)$ のときと $[0, u + 1)$ のときでの変化が少ない
 - $W[A_u + a]$ だけ増えるので, Q か所だけ更新
- $O(NM + NQ)$ 時間 $\cdot$ $O(NM)$ 空間 (MLE)

小課題 6 ($M \leq 1\,000$, パスグラフ) 転置

- $[0, u) * \{v\}$ に分解した結果をすべて明示的に持つと MLE
 - $[l, l) * \{l\} = [0, l) * \{l\} - [0, l) * \{l\}$
 - $[l, l + 1) * \{l + 1\} = [0, l + 1) * \{l + 1\} - [0, l) * \{l + 1\}$
 - $\vdots$
 - $[l, r - 1) * \{r - 1\} = [0, r - 1) * \{r - 1\} - [0, l) * \{r - 1\}$
- まとめて持とう
 - $[0, u) * \{u\}$ 型は N 種類しかないので個数をカウント
 - 残りは $[0, l) * [l, r)$ という形でまとめて記録
- $O(N + M + Q)$ 空間

考察整理タイム

元の問題

- 頂点集合 U, V に対し，長さ Q の列 $U * V$ を次で定める
 - q 項目は， $u \in U$ と $v \in V$ の組であって $A_u + A_v = B_q$ なものの個数
- 性質
 - $(U \pm U') * V = (U * V) \pm (U' * V)$
 - $U * (V \pm V') = (U * V) \pm (U * V')$
 - 集合の $\pm$ が適切に定義できるとき
 - 右辺の $\pm$ は項ごと

小課題 6 ($M \leq 1\,000$, パスグラフ)

- (機械的に転置をとれる人は転置をとって終わりです)
- 転置した問題での方針
 - $[0, u) * \{v\}$ を u の昇順に見る
 - 各 u に対して, $O(Q)$ 時間かけて準備する
 - 各 u に対していろいろな v が来たとき, $O(1)$ 時間で答える
- 元の問題の方針
 - $[0, u) * \{v\}$ を u の降順に見る
 - 各 u に対して, $O(Q)$ 時間くらいかけて何か計算
 - 各 u に対していろいろな v が来たとき, $O(1)$ 時間くらいで何か準備

小課題 6 ($M \leq 1\,000$, パスグラフ)

- 答えを $O(NM)$ 個の $[0, u) * \{v\}$ たちに分解する
 - $[l, l) * \{l\}, [l, l+1) * \{l+1\}, \dots, [l, r-1) * \{r-1\}$
- 頂点 u' が左側で関わる答えを $O(Q)$ 時間で計算するためにほしいものとは
 - 分解した $[0, u) * \{v\}$ たちのうち, $u' \in [0, u)$ (つまり $u' \leq u$) なものについての, A_v の分布 (長さ N の列)
- u' から $u' - 1$ へ変わるときの分布の差は, いろいろな v が来て 1 箇所足すだけなので, 1 個あたり $O(1)$ 時間
- $O(NM + NQ)$ 時間 $\cdot$ $O(N + M + Q)$ 空間

小課題 10 (パスグラフ)

- ある程度少ない個数の $[0, u) * \{v\}$ に分解すれば解けるということ
- Mo の区間の伸縮とは，まさに $[l, r) * \{r\}$ や $[l, r) * \{l - 1\}$ の増減
- $k - 1$ 番目の区間から k 番目の区間への伸縮中に登場したものは，係数 $M + 1 - k$ で答えに寄与する ($1 \leq k \leq M$)
 - 係数付きの $[0, u) * \{v\}$ が $O(N\sqrt{M})$ 個になる
- メモリ節約も同様
 - $[0, u) * \{u\}$ 型や $[0, u] * \{u\}$ 型を分けておく
 - 残りを，区間 * 区間 の形にまとめる
 - この右側の区間の長さの和が $O(N\sqrt{M})$ で抑えられている
- $O(N\sqrt{M} + M + NQ)$ 時間・ $O(N + M + Q)$ 空間

満点

- M_0 のパスの伸縮によって、パス * 頂点 という形に分解される
- 列の場合の $[0, u)$ に対応するものは、根付き木にして (頂点 0 を根とする)、根からのパス (根を 0 とする)
- パス $s-t$ は、 $\text{LCA}(s, t) = l$ として、 $[0, s] + [0, t] - [0, l] - [0, l]$ のように分解できる
 - 根からのパスを区間のように表記している
- 分解の情報を $O(N + M)$ 空間で持つのも同様
 - $[0, u] * \{u\}$ のようなものを分けておく
 - 残りを、根からのパス * パス の形にまとめる

満点 転置

- 「 $A_v = a$ のときの $[0, u] * \{v\}$ 」を各 $a \in [1, N]$ について持っておけたらうれしい
- 分解した $[0, u] * \{v\}$ たちを u について DFS しながら見ていく
 - u に潜るとき, Q か所増やす
 - u から上がる時, Q か所減らす
- $O(N\sqrt{M} + M + NQ)$ 時間 $\cdot$ $O(N + M + Q)$ 空間

満点

- u が関わる答えを $O(Q)$ 時間で計算するためにほしいものとは
 - 分解した $[0, u'] * \{v\}$ たちのうち, $u \in [0, u')$ (つまり u' が u の部分木内) なものについての, A_v の分布 (長さ N の列)
- $[0, u'] * \{v\}$ を u' について DFS 順に追加していくと, 部分木は区間
 - u に潜るとき, 答えから引く
 - u から上がる時, 答えに足す
- $O(N\sqrt{M} + M + NQ)$ 時間 $\cdot$ $O(N + M + Q)$ 空間
 - LCA の計算で $O(N \log(N))$ や $O(M \log(N))$ 使っても OK

その他の小課題

- 小課題 7 ($M \leq 1\,000$)
 - Mo をせず，各パスを空集合から伸ばして作る
 - パスの伸縮での場合分けがなくなりちょっと楽
- 小課題 8, 9 ($N, M \leq 50\,000$)
 - メモリ節約をせず，区間 * 点 への分解を明示的にすべて持つ
 - 満点解法だが定数倍が悪い場合
 - Mo のブロックサイズの決め方にも注意

得点分布

得点	人数	累積
24	1	1
17	1	2
13	2	4
12	1	5
9	1	6
7	6	12
4	1	13
3	6	19
0	11	30

Teleporter 解説

解説担当者：blackyuki

Subtask 0

問題概要

問題概要

- 半开区間 $[S_1, T_1)$, $[S_2, T_2)$, ..., $[S_M, T_M)$ がある。
- 区間を破壊することで、区間スケジューリングの答えが K 以下となるようにしたい。
- 区間 i はコスト C_i で破壊できる。
- コストの総和の最小値を求めてください。

0

問題概要

制約

- $N, M \leq 10^5$
- $1 \leq S_i < T_i \leq N$
- $1 \leq C_i \leq 10^9$

Subtask 1

$K=1$

1

K=1

- 「区間スケジューリングの答えが 1 」を同値に言い換える。
 - $\rightarrow \max S_i < \min T_i$
- $j = 1, 2, \dots, N-1$ について、 $S_i \leq j < T_i$ となる i の C_i の総和を計算すれば良い。
- 累積和で全体 $O(N+M)$ 。
- 5点がもらえます。

- (別解) : 双対をとる
 - 「区間スケジューリングの答えが K 」は次と同値
 - 「全ての区間を貫通させるには K 本の串が必要」
 - $K=1$ の場合、先ほどの解法に一致する

Subtask 2

$N, M \leq 20$

2

$N, M \leq 20$

- bit 全探索をします。
- 3 点がもらえます。
- うれしいね。

Subtask 3

$N, M \leq 500$

3

$N, M \leq 500$

- 区間スケジューリング問題の貪欲アルゴリズムを思い出す
 - T_i の昇順（タイブ레이크は S_i の昇順）で区間をソートし貪欲に選んでいく
- 便宜上、次の2つの区間を番兵として追加する
 - $(S_0, T_0, C_0) = (-1, 0, \text{inf})$
 - $(S_{\{M+1\}}, T_{\{M+1\}}, C_{\{M+1\}}) = (N, N+1, \text{inf})$

3

$N, M \leq 500$

- 貪欲に取っていった結果区間 $0=i_1 < i_2 < \dots < i_k=M+1$ が
選ばれるための条件を考える
- 区間 i_1 を選んでから区間 i_2 を選ぶまでに、本来選ぶべきだった区間は全て削除する必要がある
 - 具体的には、 $i_1 < j < i_2$ かつ $S_j \geq T_{\{i_1\}}$ を満たす区間 j

3

$N, M \leq 500$

- 次のような dp を考える
 - $dp[k][i]$: 区間 i まで見て、区間 i を含む k 個の区間を選んだ場合のコストの総和の最小値
 - $dp[k][i] \rightarrow dp[k+1][j]$ の遷移コストは 2 次元累積和で $O(1)$ 取得可能
- 計算量は $O(KM^2)$

3

$N, M \leq 500$

- (別解)：双対をとる。
 - 番兵として串 0 と串 $N+1$ は必ず刺すことにする。
 - 串 $0 = i_1 < i_2 < \dots < i_k = N+1$ を選んだ時、削除すべき区間のコストの最小値を求めたい。
 - i_1 から i_2 に遷移する時に加算する必要のあるコスト：
 - $i_1 < S_j < T_j \leq i_2$ を満たす区間 j のコストの総和

3

$N, M \leq 500$

- (別解)：双対をとる。
 - 本解と似たような dp が組める。
 - 二次元累積和を用いて $O(1)$ 遷移が可能。
 - 計算量は $O(KN^2)$ 。

Subtask 4

$N, M \leq 4000$

4

$N, M \leq 4000$

- 以下、双対をとったバージョンで説明します。
 - 双対を取らない場合もそんなに変わらないはずです。
- DP を高速化したいです。
- $dp[k][i_1] \rightarrow dp[k][i_2]$ の遷移コストを $F[i_1][i_2]$ とします。
 - $F[i_1][i_2] = (\text{ } i_1 < S_j < T_j \leq i_2 \text{ を満たす区間 } j \text{ のコストの総和})$
- このコスト関数 F は **Monge** です。

4

$N, M \leq 4000$

- Monge とは ... ?

Monge の定義

実数からなる $N \times M$ 行列について考えます。

$N \times M$ 行列 A が Monge であるとは、以下の条件が成り立つことをいう。

- $1 \leq i_1 < i_2 \leq N$ および $1 \leq j_1 < j_2 \leq M$ を満たすすべての整数の組 (i_1, i_2, j_1, j_2) について、 $A_{i_1, j_1} + A_{i_2, j_2} \leq A_{i_1, j_2} + A_{i_2, j_1}$ である。

引用： <https://hackmd.io/@tatyam-prime/monge1>

4

$N, M \leq 4000$

- 証明

- 一つの区間が F に与える寄与が Monge なので、
それらの和をとっても Monge になります。

4

$N, M \leq 4000$

- Monge 性の利用による DP 高速化

小課題3

- Monotone Minima なので分割統治で $O(N \log N)$
- 全体 $O(N^2 \log N)$ で定数は軽い
- 分からない場合は「Monge DP 高速化」等で検索してください

引用 : <https://www2.ioi-jp.org/camp/2023/2023-sp-tasks/contest3/chorus-review.pdf>

4

$N, M \leq 4000$

- Monge 性の利用による DP 高速化
 - $dp[k+1][r]$ の最小値が $dp[k][l]$ 由来であることを $f(k,r)=l$ と書くことにすると、 $f(k,1) \leq f(k,2) \leq \dots$ となる。
 - monotone minima ができる。
 - 計算量は $O(KN \log N)$

4

$N, M \leq 4000$

- (別解) Monge 性を使わずに、遅延セグ木を用いた inline DP をやっても良い
 - 定数倍はだいぶ厳しい (monotone minima の定数倍はすごく軽い)
 - 頑張る必要がある
 - 遅延セグ木を使わない解法も
 - 詳細は subtask 6 で

Subtask 5,6

$N, M \leq 40000$
 $N, M \leq 100000$

5

$N, M \leq 40000$ (sub5), 100000 (sub6)

- Alien DP

- N 頂点のグラフがあり、頂点 i から頂点 j へ ($i < j$) に移動するコストが $A_{i,j}$ であるとして、 A が Monge であるとき、単一始点最短路が $O(N)$ で求められます。
 - 辺をちょうど d 回通る場合の $s - t$ 最短路が $O(N \log A_{\max})$ で求められます。

引用：<https://hackmd.io/@tatyam-prime/monge1>

5

$N, M \leq 40000$ (sub5), 100000 (sub6)

- Alien DP

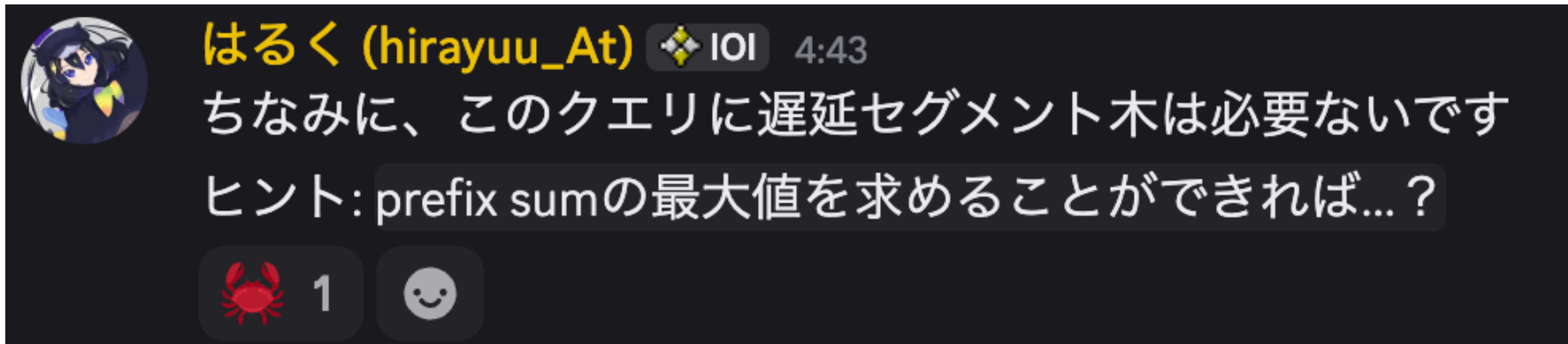
- 各辺のコストに X 加算した時の最短距離が
高速に求められれば良い
- 選択肢1：区間加算・区間 max 取得の遅延セグ木
- 選択肢2：starry sky tree
- 選択肢3：std::set で隣接点の差分を管理 + 不要な点を削除
 - スライド最小値のようなイメージ

5

$N, M \leq 40000$ (sub5), 100000 (sub6)

- Alien DP

- 選択肢 4 : 普通のセグ木で差分を管理し prefix sum の max



- 昨日の問題をちゃんと復習しましたか？という問題でしたね

5

$N, M \leq 40000$ (sub5), 100000 (sub6)

- 計算量は $O(N \log N \log(N \max C))$
 - 定数倍が良ければ満点が得られる
 - 余計な $\log$ がついたり定数倍が悪ければ sub5 が得られる

得点分布

6

得点分布

- 84点：1人
- 60点：3人
- 37点：7人
- 32点：5人
- 8点：7人
- 3点：4人
- 0点：3人

JOI 2025/2026 ファイナルステージ

Day 3 「三角形降雨」

解説担当：西本将樹 (maspy)

問題概要

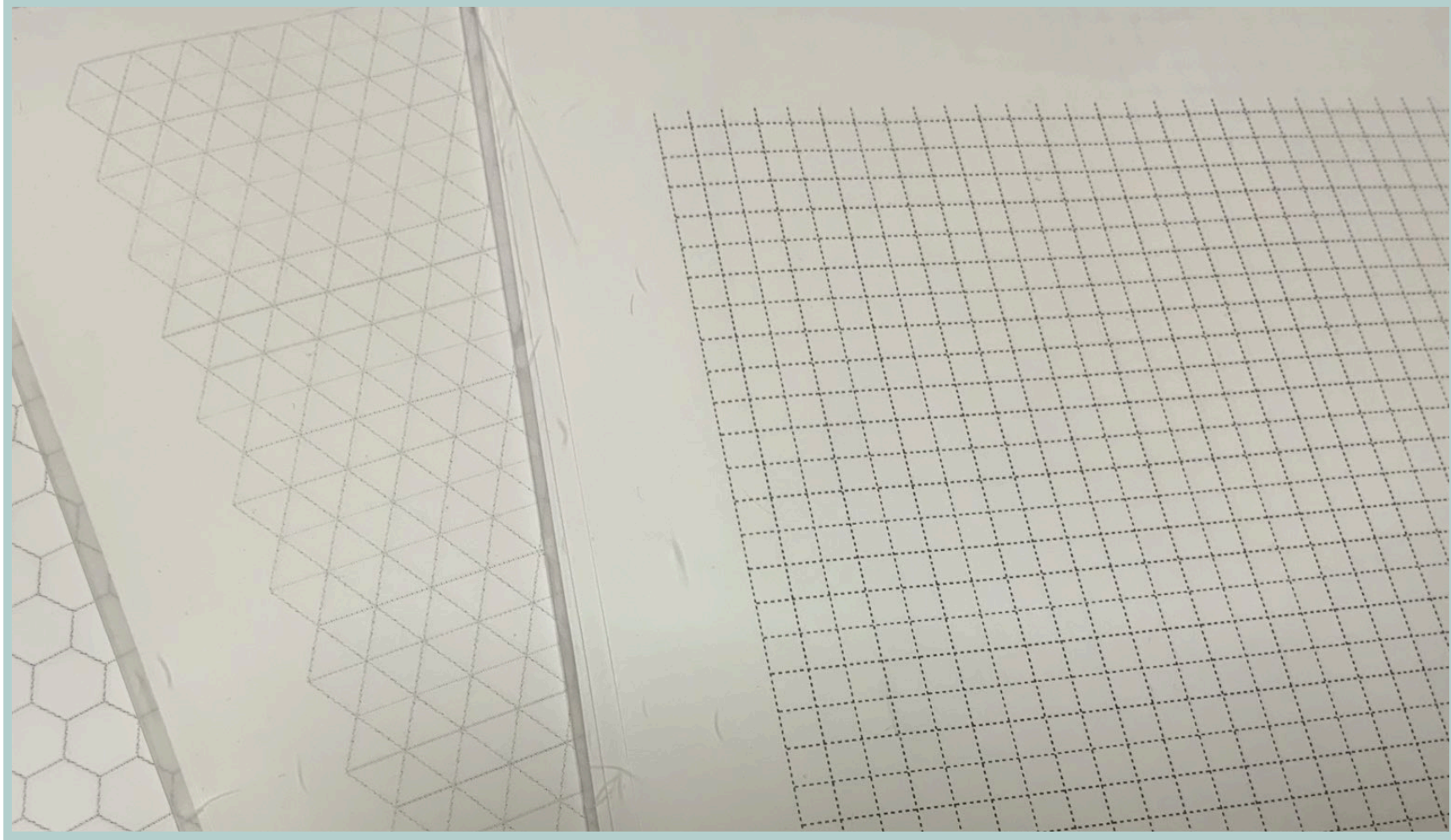
- 正三角形グリッドがある（無限に続くとしてよい）
- 正三角形領域へのインクリメントが来る．
- k 以上となった部分の面積を求める ($k = 1, 2, 3, 4, 5$)．

問題文の図を参照．

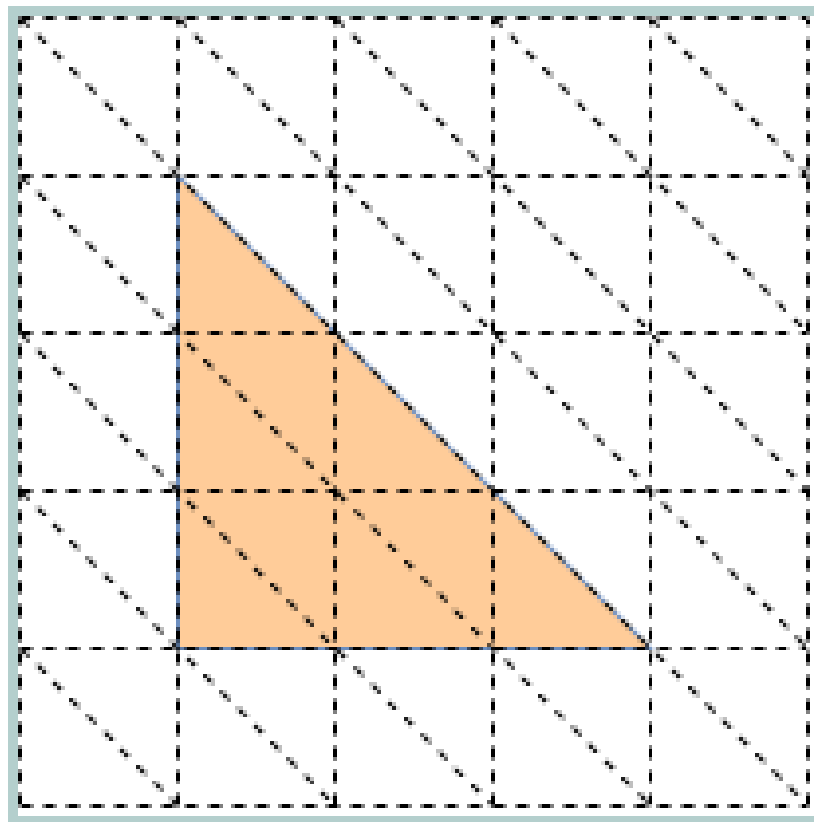
正三角形グリッドについて

- 持ち込み自由コンテスト：専用の用紙を持っておくと役に立つことがあります。
- 持ち込みなしコンテスト：作図が下手な人は、**線形変換**により、斜交座標を通常の直交座標に置き換えてしまうことも有効だと思います。
- ただし、鏡像原理などが見えにくくなるリスクはあります。

さまざまなグリッド用紙



直交座標への置き換え



解説もこの形式でやります.

小課題 1 ($N = 2$)

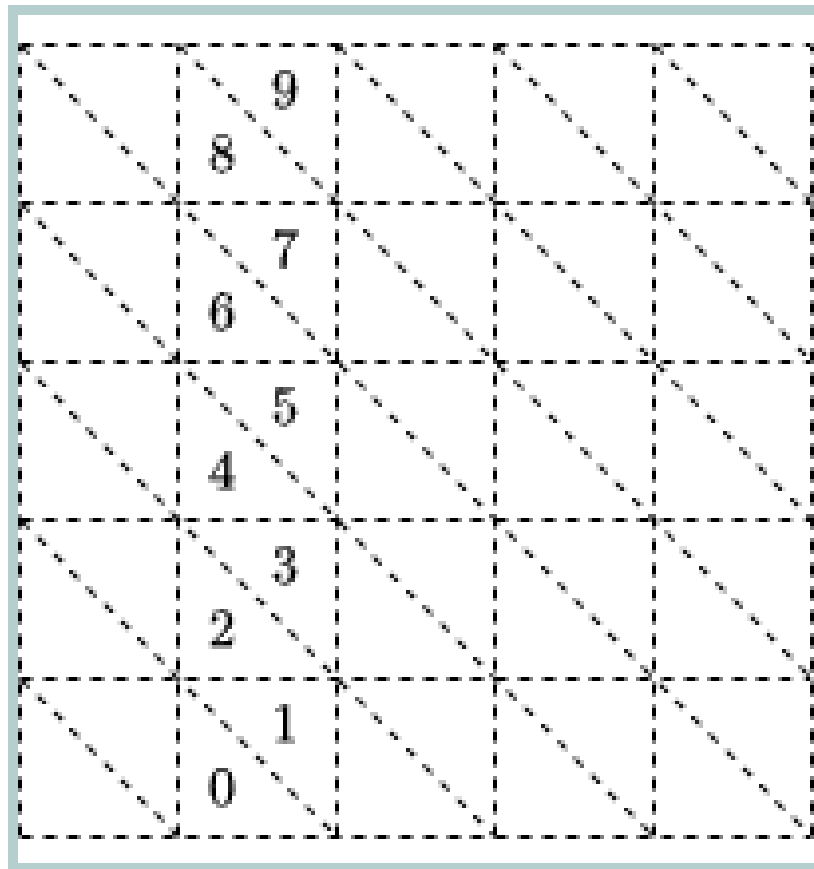
- 共通部分の面積を求めるという幾何問題です.
- 正三角形は $a_i \leq x, b_i \leq y, x + y \leq c_i$ の形.
- その共通部分も, $a \leq x, b \leq y, x + y \leq c$ の形.
- 平行移動すると, $0 \leq x, 0 \leq y, x + y \leq c$ の形.
- c の符号で場合分けして終了.

後続の小課題群のランダムテスト等の助けになりにくいので, 小課題 4 あたりまで解ける見込みがあるならスキップするのもよいでしょう.

小課題 2 ($L \leq 100, N \leq 100$)

- 愚直解.
- T_i 内のすべての区画の座標を列挙して、足すだけです.
- 「区画の座標」をどうするか？
 - 方法 1：上向きの三角形と下向きの三角形に分けて処理する.
 - 方法 2：両方の向きの三角形全体にいい感じの番号を付ける.

小課題 2 ($L \leq 100, N \leq 100$)



- 方法 2：両方の向きの三角形全体にいい感じの番号を付ける.

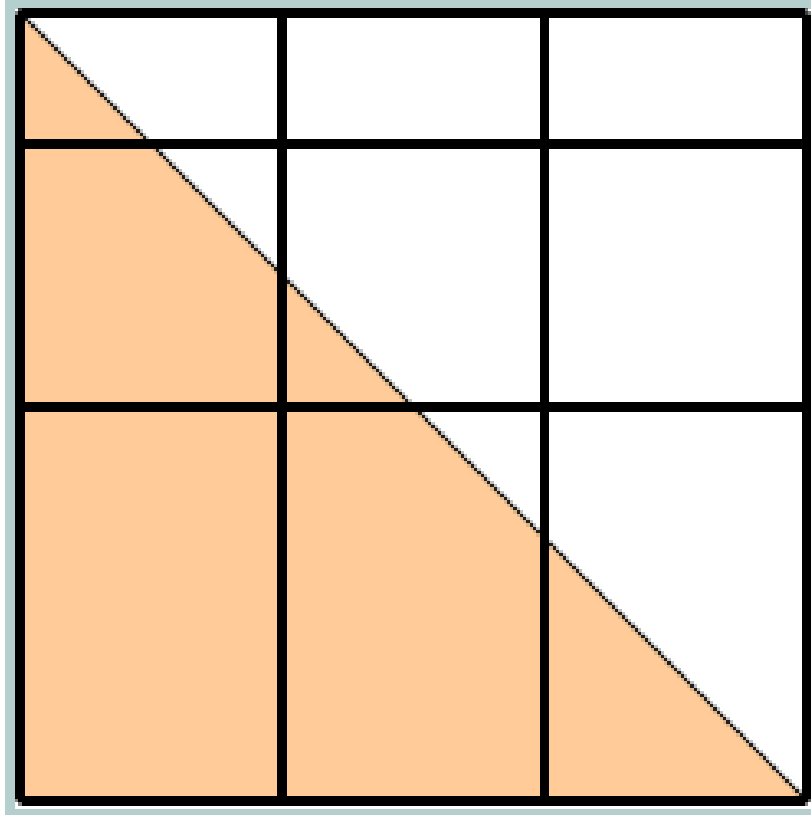
小課題 3 ($L \leq 1000$)

- 区画の個数は $O(L^2)$ なので、すべての区画に対する頻度を求める解法を考えられます.
- 累積和 (imos 法) などを用いて高速に足せばよいです.
- $O(NL + L^2)$ でも十分高速です. x は愚直にループして, y 方向には区間加算とすると簡単です.
- 複数方向の累積和を頑張ることで $O(N + L^2)$ にもできます. 今回の出題では不要な処理です.

小課題 4 ($N \leq 2000$)

- 座標圧縮を考えましょう.
- 「交点に現れる座標」をすべてとろうとすると、座標の種類が大きくなってしまいます.
- 単に、 a_i の集合や b_i の集合で区切ってみます.

小課題 4 ($N \leq 2000$)



小課題 4 ($N \leq 2000$)

- $O(N^2)$ 個の長方形領域に分割.
- T_i は, いくつかの長方形を完全に覆い, いくつかの長方形を部分的に覆う.
- 前者のパターンは小課題 3 と同様にできる.
- 後者のパターンは T_i ごとに $O(N)$ 個. これを適切に処理すればよい. ちょっと面倒にはなると思います.

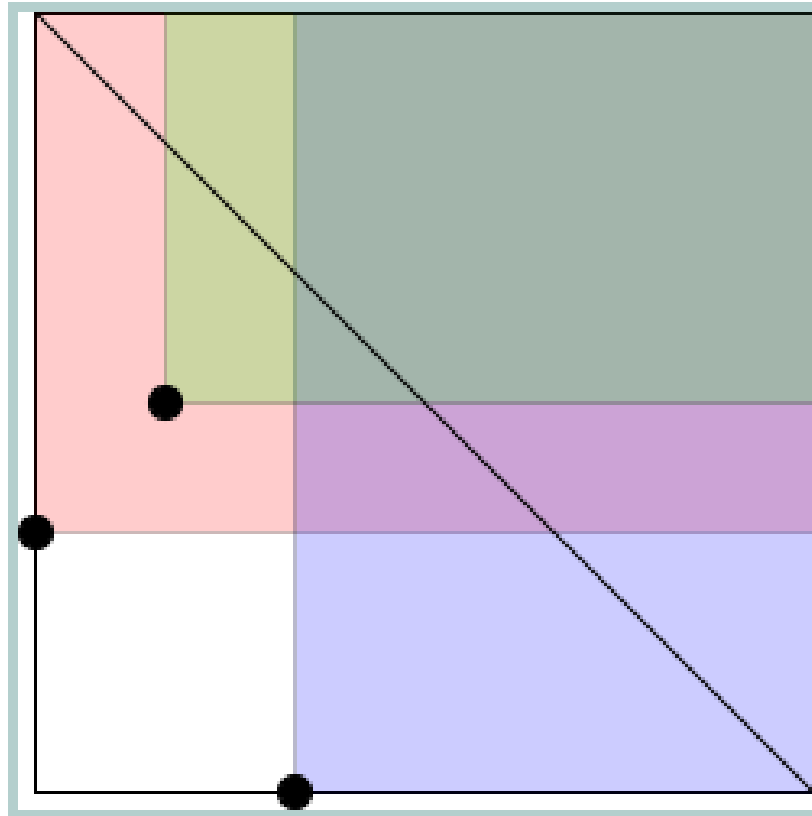
小課題 5, 小課題 6 ($X_i = 0$)

- 回転して, 制約を $X_i + Y_i + Z_i = L$ として解説します.
- T_i は領域 $[a_i, \infty) \times [b_i, \infty) = \{(x, y) \mid a_i \leq x, b_i \leq y\}$ のうちで $x + y \leq L$ を満たす部分. と考えることができます.

小課題 5, 小課題 6 ($X_i = 0$)

- まずは, $x + y \leq L$ という条件を無視した上で, 領域 $[a_i, \infty) \times [b_i, \infty)$ が k 個以上重なる部分を求めます.
- $O(NK)$ 個の矩形領域が登場.
- それぞれについて, その中で $x + y \leq L$ を満たす部分の面積を求めて, 答に加算すればよいです.

小課題 5, 小課題 6 ($X_i = 0$)



小課題 7, 小課題 8, 小課題 9

- T_i は $a_i \leq x, b_i \leq y, x + y \leq c_i$ という領域です.
- このうち $x + y \leq c_i$ という条件を無視すれば, 「 k 個以上重なっている領域」は簡単に表せることが分かりました.
- この考え方をもとに, $x + y = c$ を scan line として処理するというタイプの解法を考えます.

小課題 7, 小課題 8, 小課題 9

次のような解法が可能です.

- T_i を c_i について降順ソートして, ひとつずつ追加していきます.
- 各時点での答を保持します. (答は, $c_i \leq x + y$ の部分の面積だけを考慮する**ではありません**).
- T_i の追加のたびに, 答を差分更新します.

小課題 7, 小課題 8, 小課題 9

- T_i の追加のたびに, 答を差分更新します.
- この際, $x + y \leq c_i$ である部分での重複個数だけが問題です.
- したがってこの目的のためには, T_i を領域 $[a_i, \infty) \times [b_i, \infty)$ に置き換えてしまってもよいです.

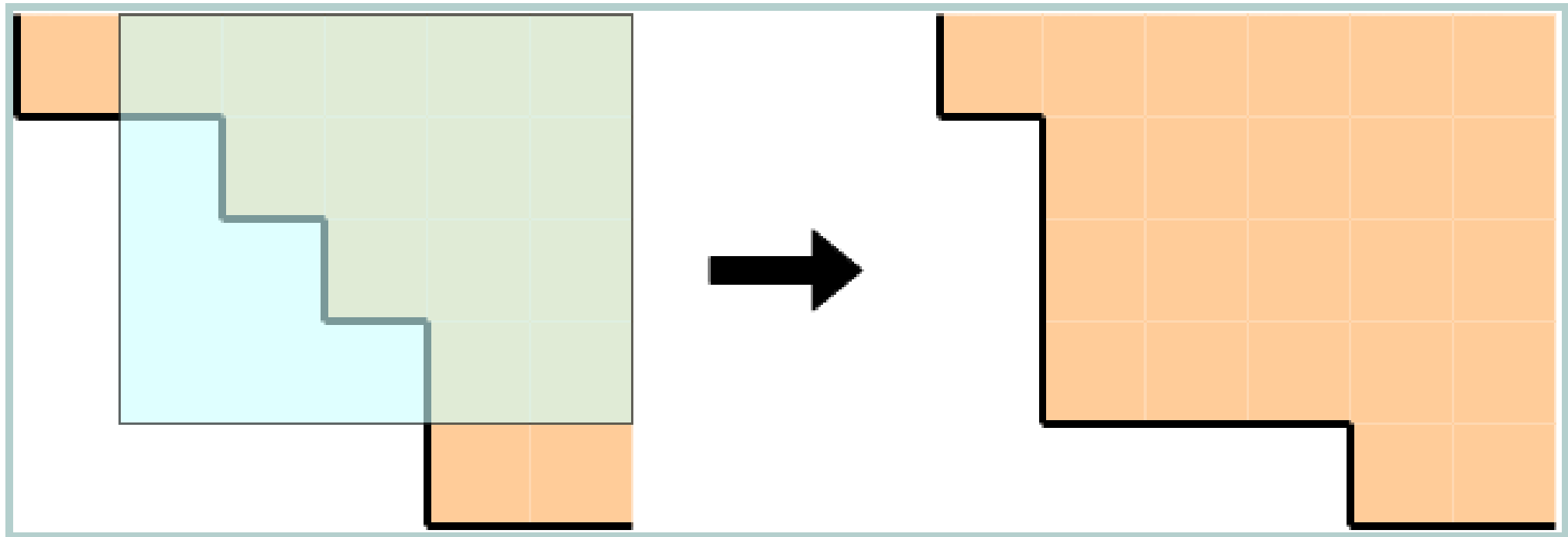
小課題 7, 小課題 8, 小課題 9

- 結局, 次をやればよいということになります.
- 長方形 $[a_i, \infty) \times [b_i, \infty)$ の多重集合を適切に管理する.
 - 追加するたびに, $0, 1, \dots, K$ 個が重なっている部分がどう変化したかを取得できるようにする.
- 個数が変化した部分について, $x + y \leq c_i$ を満たす部分の面積を求めて, 答を差分更新する.

小課題 7 ($K = 1$)

- 小課題 7 では、長方形 $[a_i, \infty) \times [b_i, \infty)$ の和集合を管理すればよいです.
- 列の問題に言い換えると次のようになります.
 - 列 $(h_0, h_1, h_2, \dots)$ がある. すべて ∞ で初期化されている.
 - $h_i, h_{i+1}, h_{i+2}, \dots$ に対して $h_j := \min(h_j, x)$ というタイプのクエリが来る.
 - クエリのたびに h がどう変わったかを報告する.
- h は単調減少列になっていて、これを定値区間列として管理すると、全体で $O(N)$ 回の区間変更という形になります.

小課題 7 ($K = 1$)

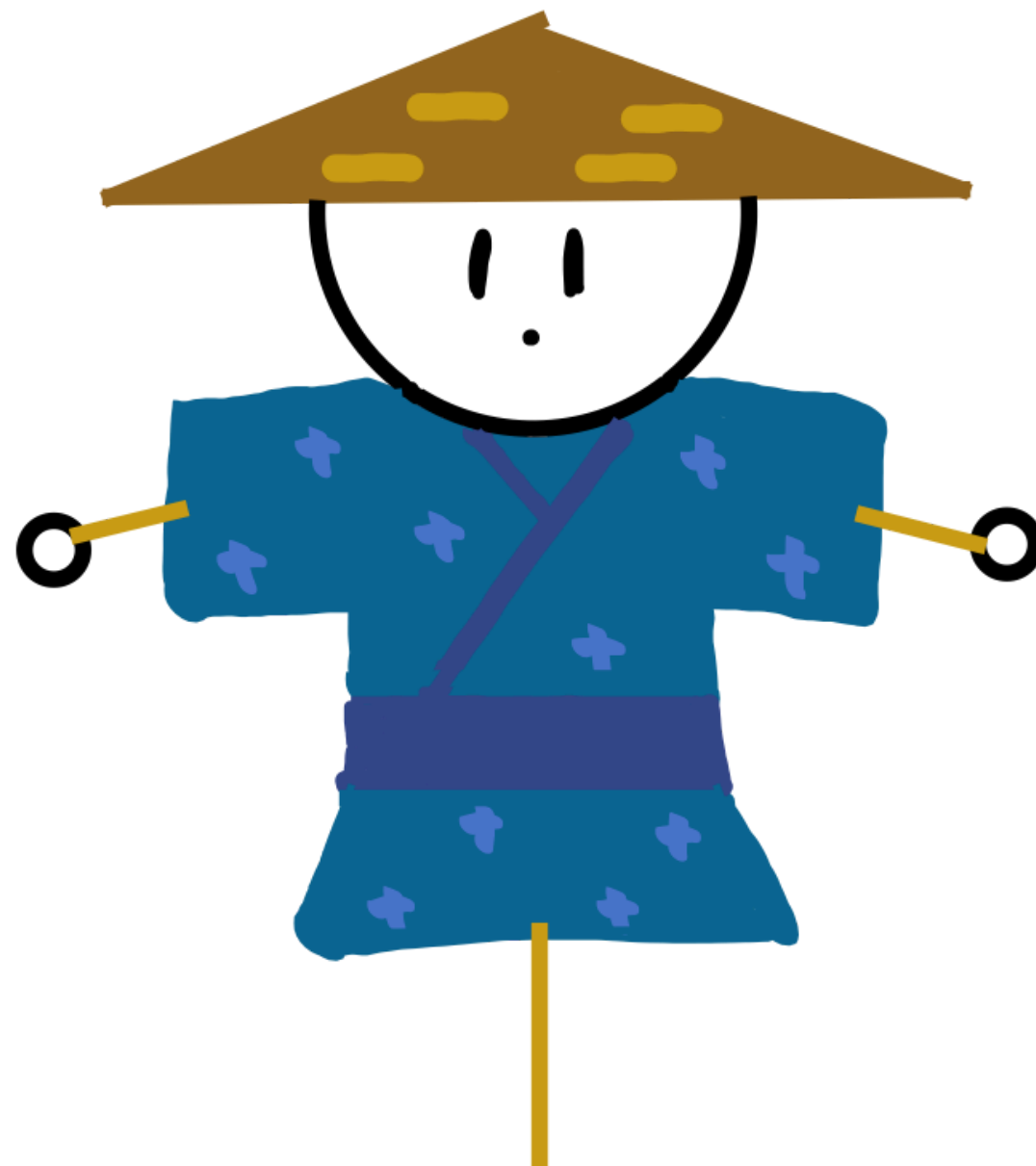


小課題 8, 小課題 9 (満点解法)

- 実装は難化しますが, $K = 5$ になってもやることは変わりません.
- $k = 1, 2, \dots, K$ に対して「 k 個以上重なっている部分」は, やはり単調減少列により表されます. これらを定値区間列として管理して, それぞれ $O(N)$ 回の区間変更という形で処理すればよいです.
- 定数倍が悪くなければ満点, 悪ければ小課題 8 ということになると思います.
- 定数倍に苦戦した場合: set の代わりに, 座標圧縮して 64 分木を用いるなど.

JOI 2025/2026 ファイ ナルステージ Day 3 かかし 2 (Scarecrow 2) 解説

解説担当 : iro_



問題概要

- xy 座標平面上に N 体のかかしがいる。
それぞれのコストは C_i であり、
 - $T_i = 1$ のとき、かかしは点 (X_i, Y_i) より $x \leq X_i$ の領域を守る
 - $T_i = 2$ のとき、かかしは点 (X_i, Y_i) より $x \geq X_i$ の領域を守る
 - $T_i = 3$ のとき、かかしは点 (X_i, Y_i) より $y \leq Y_i$ の領域を守る
 - $T_i = 4$ のとき、かかしは点 (X_i, Y_i) より $y \geq Y_i$ の領域を守る
- すべての点を K 体以上のかかしで守るために必要な合計コストの最小値を求めよ。

配点 (JOI)

番号	配点	制約	計算量の例
1.	-- 点	$K = 1, T = 2$	
2.	4 点	$K = 1$	
3.	6 点	$K \leq 2$	
4.	-- 点	$N \leq 15$	
5.	11 点	$N \leq 300$	$O(NK^2)$
6.	27 点	$N \leq 6\,000$	$O(NK)$
7.	19 点	$N \leq 75\,000$	
8.	33 点	$N \leq 200\,000$	$O((N + K) \log N)$

配点 (JOIG)

番号	配点	制約	計算量の例
1.	5 点	$K = 1, T = 2$	
2.	7 点	$K = 1$	
3.	7 点	$K \leq 2$	
4.	22 点	$N \leq 15$	
5.	35 点	$N \leq 300$	$O(NK^2)$
6.	24 点	$N \leq 6\,000$	$O(NK)$
7.	-- 点	$N \leq 75\,000$	
8.	-- 点	$N \leq 200\,000$	$O((N + K) \log N)$

小課題 1 ($K = 1, T \leq 2$)

小課題 1 ($K = 1, T \leq 2$)



- $T_i = 1$ のとき、かかしは点 (X_i, Y_i) より $x \leq X_i$ の領域を守る
- $T_i = 2$ のとき、かかしは点 (X_i, Y_i) より $x \geq X_i$ の領域を守る

$\implies Y_i$ 座標は関係ない

ある地点 (x, y) を守るためには、

- $T_i = 1$ のかかし $x \leq X_i$ が存在するか
- $T_i = 2$ のかかし $x \geq X_i$ が存在するか

特に

点 $(-\infty, 0)$ を守ることができるのは $T_i = 1$ のかかし

点 $(\infty, 0)$ を守ることができるのは $T_i = 2$ のかかし

$\implies T_i = 1$ のかかしと $T_i = 2$ のかかしは少なくとも一体ずつ必要

ある i, j について $T_i = T_j = 1$ かつ $X_i \leq X_j$ であるとき

i 番目のかかしが守る領域は j 番目のかかしが守る領域に完全に含まれているため、 i 番目のかかしは必要ない

$\implies T_i = 1$ を満たすかかしは複数体必要ない

よって

条件を満たす (i, j) 組は

$$T_i = 1 \text{ かつ } T_j = 2 \text{ かつ } X_j \leq X_i$$

このとき、求める合計コストの最小値は

$$\min(C_i + C_j)$$

小課題 2 ($K = 1$)



- $T_i = 3$ のとき、かかしは点 (X_i, Y_i) より $y \leq Y_i$ の領域を守る。
- $T_i = 4$ のとき、かかしは点 (X_i, Y_i) より $y \geq Y_i$ の領域を守る。

$\implies X_i$ 座標は関係ない。

点 $(-\infty, -\infty)$ を守ることができるのは $T_i = 1$ または $T_i = 3$ のかかし

点 $(-\infty, \infty)$ を守ることができるのは $T_i = 1$ または $T_i = 4$ のかかし

点 $(\infty, -\infty)$ を守ることができるのは $T_i = 2$ または $T_i = 3$ のかかし

点 (∞, ∞) を守ることができるのは $T_i = 2$ または $T_i = 4$ のかかし

小課題 1 と同じように考察すると 計画を 2 つ実行し、実行する計画を 計画 i 、計画 j とすると、 (i, j) は

- $T_i = 1$ かつ $T_j = 2$ かつ $X_j \leq X_i$
- $T_i = 3$ かつ $T_j = 4$ かつ $Y_j \leq Y_i$

のどちらかを満たせばよいと分かる

よって答えは満たす (i, j) の組は

$$T_i = 1 \text{ かつ } T_j = 2 \text{ かつ } X_j \leq X_i$$

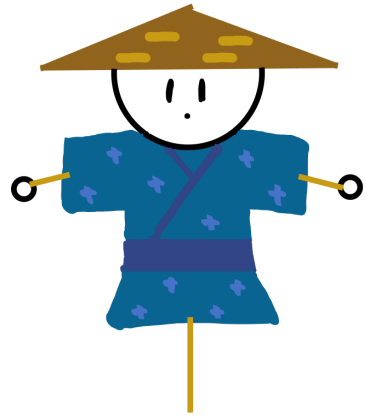
または

$$T_i = 3 \text{ かつ } T_j = 4 \text{ かつ } Y_j \leq Y_i$$

合計コストの最小値は

$$\min(C_i + C_j)$$

小課題 3 ($K \leq 2$)



結局すべての点を 1 体以上のかかしで守るためには計画を 2 つ実行すればよい

実行する計画を 計画 i 、計画 j とすると、 (i, j) は

- $T_i = 1$ かつ $T_j = 2$ かつ $X_j \leq X_i$ を満たす
- $T_i = 3$ かつ $T_j = 4$ かつ $Y_j \leq Y_i$ を満たす

のどちらかを満たせばよいと分かる

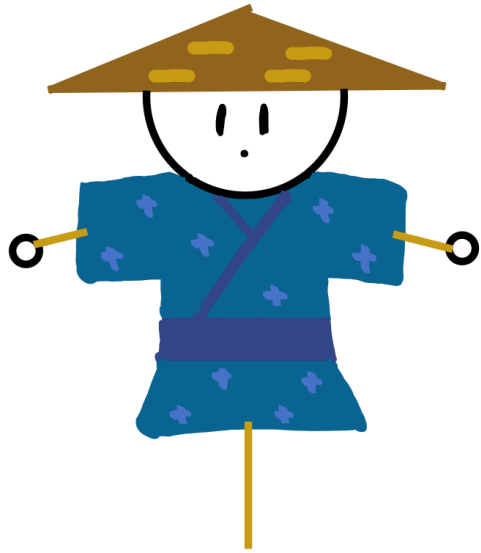
実装では $T_i = 1$ のかかし、 $T_i = 2$ のかかしを走査し合計コストが小さい 2 組を求めたらよい

実装では 2 組挙げる時に重複しないように気を付けましょう

これを $T_i = 3, 4$ のかかしについても同様に行う

求めた 4 組のうち合計コストが小さい 2 組を選ぶと答えが求まる

小課題 4 ($N \leq 15$)



$T_i = 1, 2$ と $T_i = 3, 4$ は独立なので今後は分けて考える

$T_i = 1, 2$ のみですべての点を k 体以上のかかしで守るとき、
 $T_i = 1$ の計画を t 個、 $T_i = 2$ の計画を t 個実行するのが最適

仮に $T_i = 1$ のかかしが k 個未満であれば、 $(-\infty, y)$ の点は k 個にカバーされない
仮に $T_i = 1$ のかかしが $k + 1$ 個以上であれば、その中で X_i の大きい k 個を残しても「条件が満たされているかどうか」は変わらない

$T_i = 2$ についても同様

しかし、この k 個のかかしの並びも重要

左から順に 111...1222...2 と並んでいたら、誰も真ん中を守らない

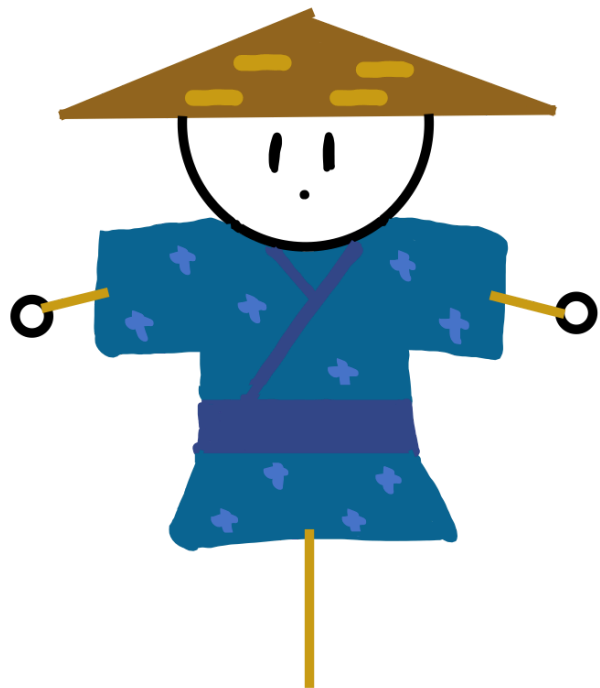
具体的には、どの prefix についても $T_i = 2$ の数が $T_i = 1$ の数以上である

計画の数が少ないのでどの計画を実行するか全探索し、 $O(N)$ かけて判定する
計算量は $O(2^N N)$ 時間

かかしがすべての点を守るためのまとめ

- $T_i = 1, 2$ を用いて k 体以上のかかしで守るとき、 $T_i = 3, 4$ ではを $K - k$ 体以上のかかしで守ればよい
- $T_i = 1$ の計画を t 個、 $T_i = 2$ の計画を t 個実行するのが最適
- どの prefix についても $T_i = 2$ の数が $T_i = 1$ の数以上である

小課題 5 ($N \leq 300$)



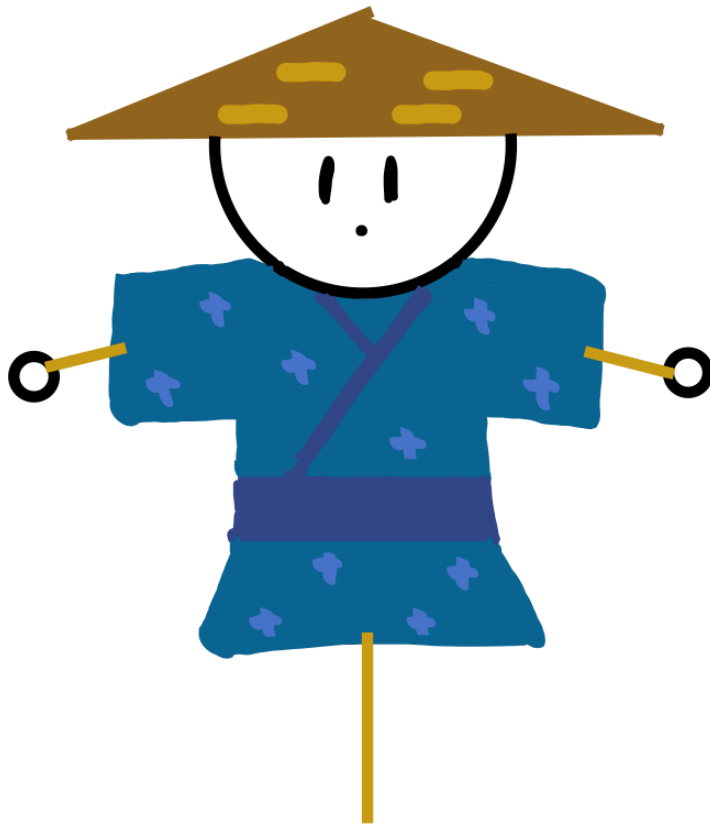
小課題 4 の方針を高速化させたいので動的計画法に落とし込む

$dp[t][s] : T_i = 1$ の計画を t 個実行し $T_i = 2$ の計画を s 個実行したときの合計コストの最小値

X_i をソートし、左から順にみていく。

$O(NK^2)$ 時間で解ける

小課題 6 ($N \leq 6\,000$)



かかしがすべての点を守るための条件

- $T_i = 1$ の計画を t 個、 $T_i = 2$ の計画を t 個実行するのが最適
- どの prefix についても $T_i = 2$ の数が $T_i = 1$ の数以上である

分かりやすく

$T_i = 1$ の計画を t 個、 $T_i = 2$ の計画を s 個実行する
このとき、

- 座標が小さい順に数えると常に $t \leq s$
- 最後には $t = s$

見覚えのある条件ですね

これは「良い括弧列」である条件と同値

$T_i = 1$ を $)$ とし、 $T_i = 2$ を $($ とする

すべての点を守るかかしの最小数を 1 増やすことと括弧列に $()$ を追加することは同値

1212 は " $()()$ "

1122 は " $(())$ "

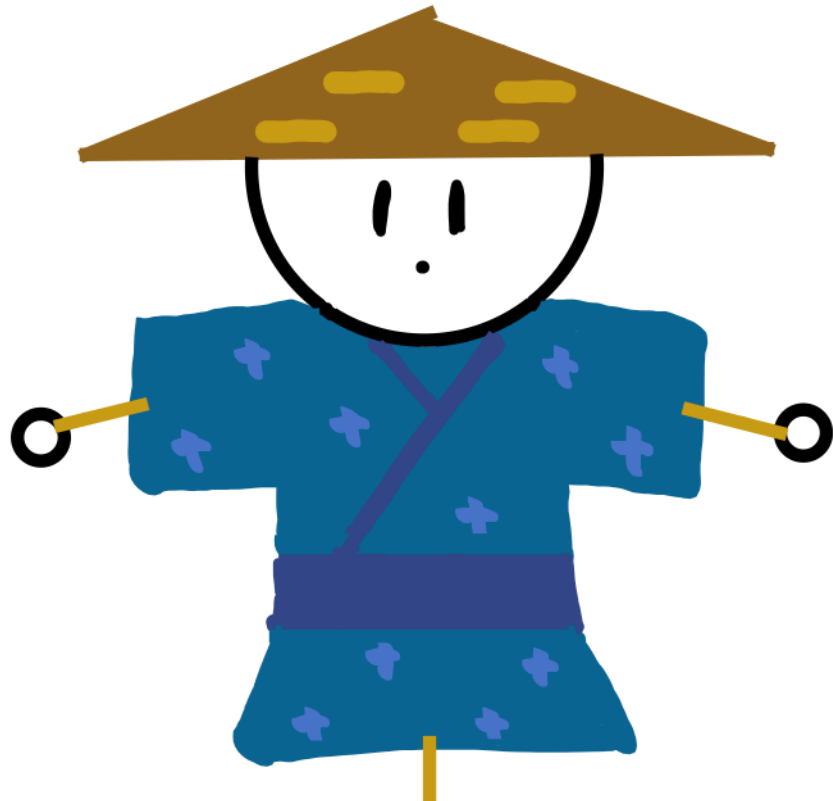
(を $+1$,) を -1 として 累積和を取ると、括弧列の条件を満たすかどうかは累積和の最小値が 0 以上であるかどうかで判定できる

新たに () を追加するときは制約はなく

) (を追加するときは、累積和が 0 のところを跨がなければよい

$O(NK)$ 時間

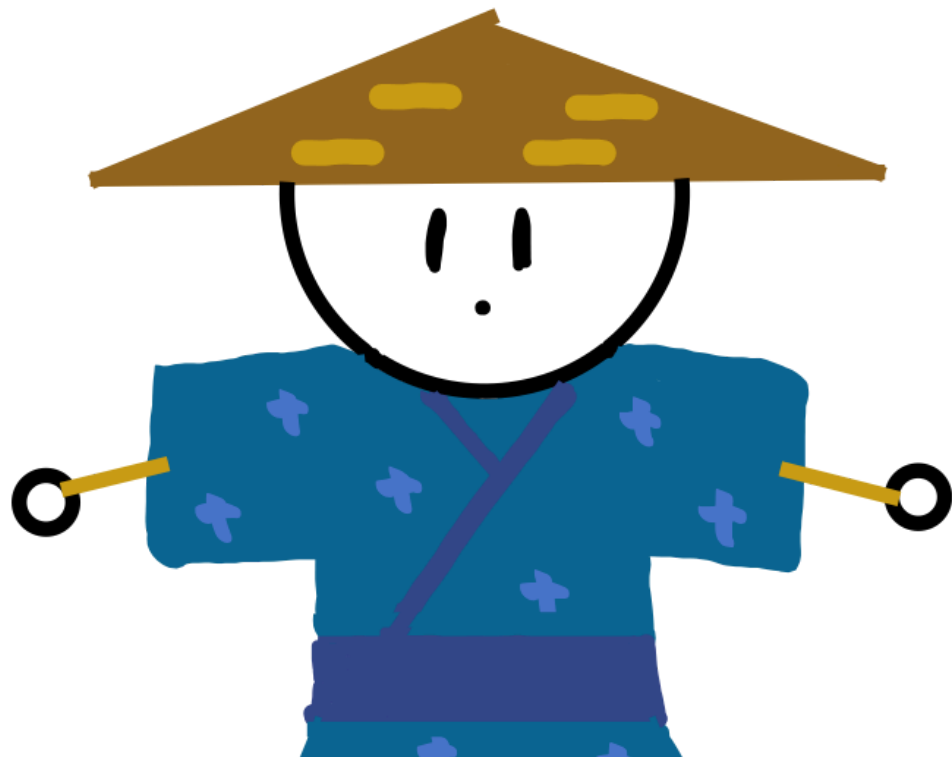
小課題 7 ($N \leq 75\,000$)



小課題 7 ($N \leq 75000$)

小課題 8 の $O((N + K) \log N)$ 時間に定数倍色々つけて

小課題 8 ($N \leq 200\,000$)



これを遅延セグメント木に乗せて高速させたい

括弧列であることと、開き括弧、閉じ括弧の完全マッチングが可能というのは同値
その貪欲は最小費用流が貪欲で求まることと対応する

フローの問題に定式化させます
path に流す = 括弧列に 2 個追加

- グラフ

頂点 $S, 1, 2, \dots, 2N, T$

$i \rightarrow i+1$ に容量 ∞ かつコスト 0 の辺がある

$i+1 \rightarrow i$ に容量 0 かつコスト 0 の辺（逆辺）がある

- 流す場所の探し方

($\rightarrow$) は無条件

) $\rightarrow$ (は、逆辺の流量が正になっているところだけたどれる

- 流し方

開きカッコから閉じカッコに流す

流す場所を見つけたあとの処理

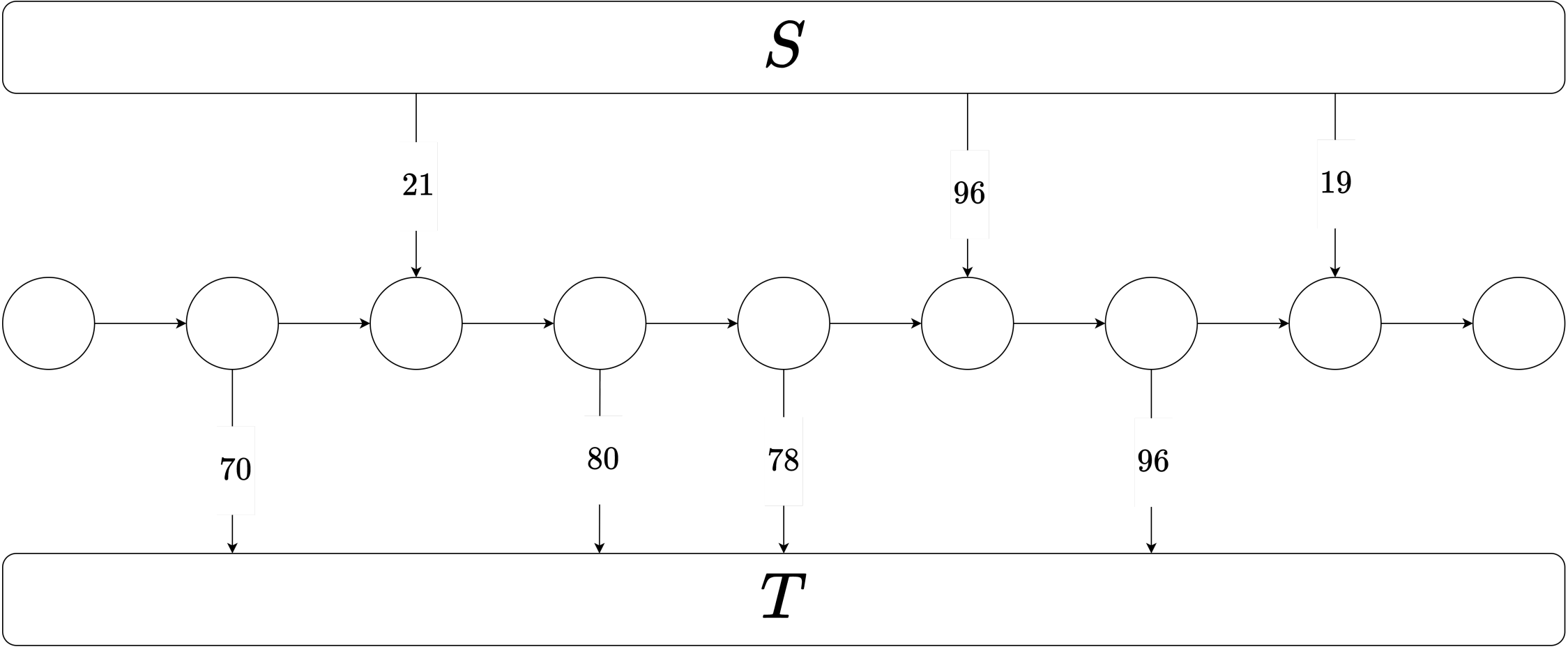
($\rightarrow$) なら逆辺の容量が +1 される

) $\rightarrow$ ((これは逆辺の容量が全部正のときだけ可能) なら逆辺の容量が -1 される

フローは最短路反復で解ける

(証明が気になる方は「最短路反復 最小費用流」で検索してください)

サンプル 1



実装では累積和($= C_i$)をマージするときに C_i が 1 以上であるかではなく、
 $C_i > \min C$ を持ちます

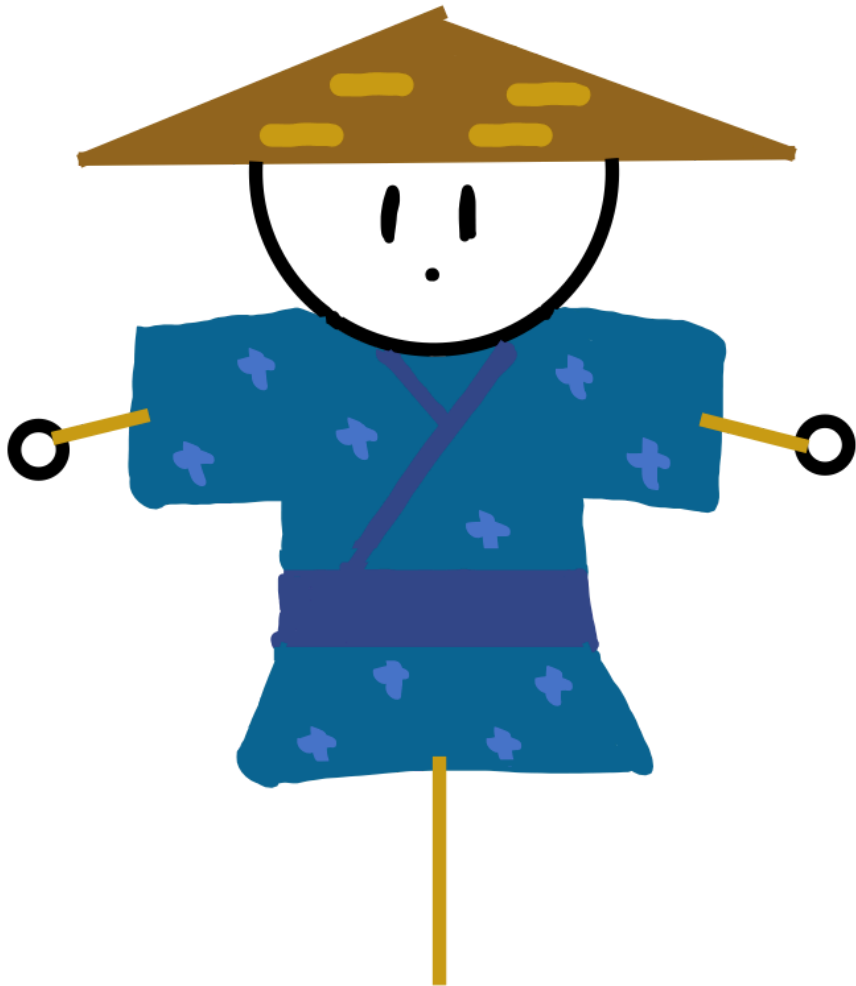
おまけ

$k = 1, 2, \dots, K$ についてすべての点を k 体以上のかかしで守ると考えると

1 次元において解が下に凸なので min-plus convolution できるので毎回最も小さい組を探せばよい

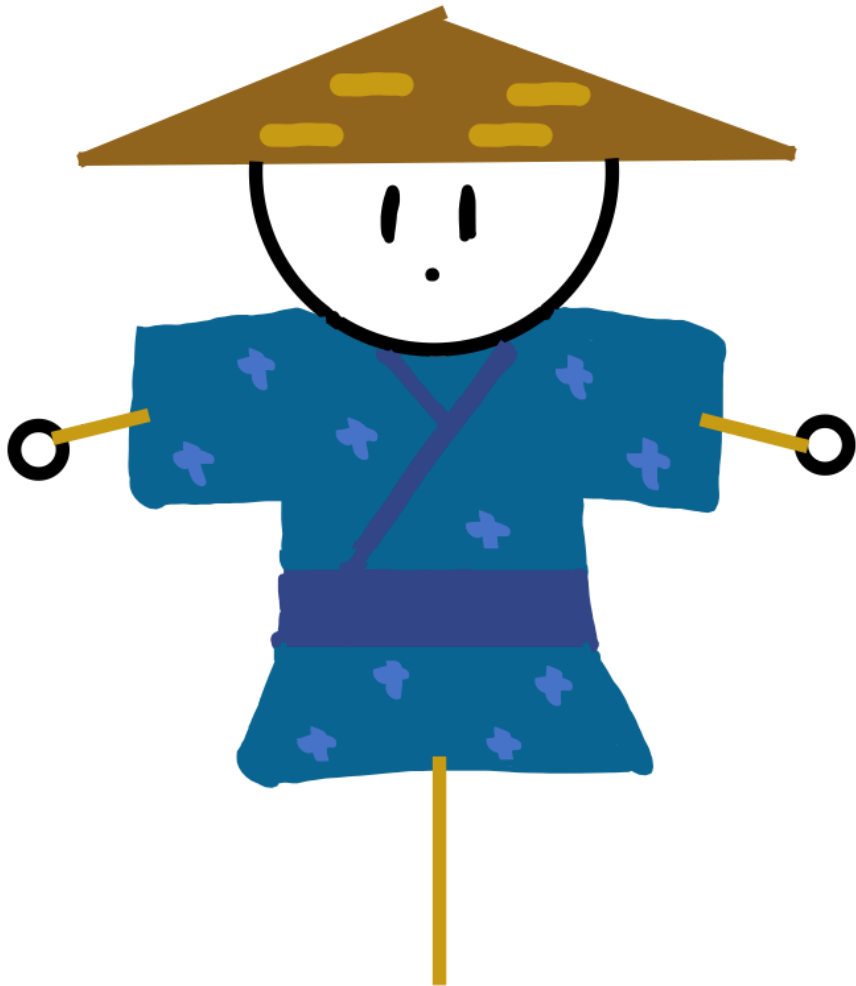
JOI 得点分布

得点	人数	累積
100	1	1
48	5	6
21	14	20
15	1	21
11	2	23
10	2	25
4	3	28
0	2	30



JOIG 得点分布

得点	人数	累積
100	0	0
41	1	1
34	2	3
19	1	4
12	10	14
5	1	15
0	0	15



A vibrant underwater scene featuring a large school of various fish swimming in clear blue water. The fish are of different sizes and species, including some with prominent stripes and others with solid colors. Sunlight filters through the water, creating a shimmering effect. The overall atmosphere is serene and natural.

Collecting Stamps 5 解説

解説担当者：blackyuki

Subtask 0

問題概要

問題概要

- N 頂点の木がある。
- 頂点の組 (s, g) は次の条件を満たす時良いペアと呼ぶ。
 - 木上のパスを $(s=i_0, i_1, i_2, \dots, i_d=g)$ とした時、
 $T_{\{i_j\}} \leq j$ を満たす $0 \leq j \leq d$ が存在し、かつ、 $d \leq D$
- $s=1, 2, \dots, N$ それぞれについて、 (s, g) が良いペアとなる g の個数を求めよ。

0

問題概要

制約

- $2 \leq N \leq 400000$
- $0 \leq D \leq N-1$
- $0 \leq T_i \leq N$

Subtask 1

$$D \leq 1$$

1

$D \leq 1$

- 頂点 s と頂点 g の距離を $\text{dist}(s,g)$ とする。
- $D \leq 1$ の時、 $\text{dist}(s,g) \leq D$ となる (s,g) の組は $O(N)$ 。
 - $\text{dist}(s,g) = 0$ は頂点数。
 - $\text{dist}(s,g) = 1$ は辺数の2倍。
- それぞれのペアについて愚直に判定すれば良い。
- 計算量は $O(N)$ 。

```
int N, D; cin >> N >> D;
vector<int> T(N), ans(N);
for(int i = 0; i < N; i++) {
    cin >> T[i];
    if(T[i] == 0) ans[i] ++;
}
for(int i = 0; i < N-1; i++) {
    int u, v; cin >> u >> v;
    u--; v--;
    if(D == 1) {
        if(T[u] <= 0 || T[v] <= 1) ans[u] ++;
        if(T[v] <= 0 || T[u] <= 1) ans[v] ++;
    }
}
for(int i = 0; i < N; i++) cout << ans[i] << '\n';
```

Subtask 2

パスグラフ、 $N \leq 3000$

2

パスグラフ、 $N \leq 3000$

- $s < g$ のケースだけ考える
 - $s = g$ は簡単、 $s > g$ は対称性により $s < g$ のケースに帰着できるので
- s から右方向に歩いて行った時、最初にスタンプを押せる街を m とすると、 (s, g) が良いペアとなる g は $g = m, m+1, \dots, s+D$ となる。
- 各 s について、 $O(N)$ かけて良い。
- 計算量は $O(N^2)$ 。

```
for(int s = 0; s < N; s++) {  
    int ans = 0;  
    for(int i = 0; i <= D && s + i < N; i++) if(T[s + i] <= i) {  
        ans += D - i + 1;  
        break;  
    }  
    for(int i = 0; i <= D && s - i >= 0; i++) if(T[s - i] <= i) {  
        ans += D - i + 1;  
        break;  
    }  
    if(T[s] == 0) ans--;  
    cout << ans << '\n';  
}
```

Subtask 3

$N \leq 3000$

3

$N \leq 3000$

- 小課題 2 と同様に、各 s ごとに $O(N)$ かけて良い。
 - DFS で木を探索する。
 - 以下の情報を持つ。
 - 現在の体力
 - スタンプを押したか (bool 値)
- 計算量は $O(N^2)$ 。

```
for(int s = 0; s < N; ++s) {  
    int ans = 0;  
    auto dfs = [&](auto &&self, int i, int p, int d, bool stamped) -> void {  
        if(T[i] <= d) stamped = true;  
        if(stamped) ans ++;  
        if(d < D) {  
            for(int to : G[i]) if(to != p) self(self, to, i, d+1, stamped);  
        }  
    }; dfs(dfs, s, -1, 0, false);  
    cout << ans << '\n';  
}
```


Subtask 4

パスグラフ

4

パスグラフ

- $s < g$ のケースのみ考える。
- 今後は逆に、各 g について、 s から g まで歩いた時に一度以上スタンプを押せるような最大の s を $dp[g]$ とする。
 - $dp[g] = \max(g - T[g], dp[g-1])$ で求められる。

4

パスグラフ

- g を固定した時、 (s, g) が良いペアとなる s は区間となる。
 - 具体的には $g-D, g-D+1, \dots, dp[g]$ となる。
 - imos 法で $ans[s]$ に区間加算すれば良い。
- $s > g$ についても同様にやる。
- 計算量は $O(N)$ 。

```

vector<int> sol(int N, int D, const vector<int>& T) {
    vector<int> ans(N), dp(N, -1);
    if(T[0] == 0) dp[0] = 0;
    for(int g = 1; g < N; g++) dp[g] = max(dp[g-1], g - T[g]);
    for(int g = 0; g < N; g++) if(g - D <= dp[g]){
        ans[max(0, g - D)] ++;
        if(dp[g] + 1 < N) ans[dp[g] + 1] --;
    }
    for(int i = 1; i < N; ++i) ans[i] += ans[i-1];
    return ans;
}

int main() {
    int N, D; cin >> N >> D;
    vector<int> T(N);
    for(int i = 0; i < N; i++) cin >> T[i];
    vector<int> ans = sol(N, D, T);
    reverse(T.begin(), T.end());
    vector<int> ans2 = sol(N, D, T);
    reverse(ans2.begin(), ans2.end());
    reverse(T.begin(), T.end());
    for(int i = 0; i < N; ++i) {
        ans[i] += ans2[i];
        if(T[i] == 0) ans[i]--;
        cout << ans[i] << '\n';
    }
}

```

Subtask 5,6

$D=N-1, N \leq 150000$

$N \leq 400000$

5

満点

- sub6 の解法を説明します。
 - 実装をサボったりすると sub5 だけ点が入るかもしれません。
- T_i が全部 0 の時を考える。
 - つまり、スタンプ台が最初から全て配置されている。
 - 各 s について、 s から距離 D 以下の頂点数を求めよ、という問題

5

満点

- 各 s について、 s から距離 D 以下の頂点数を求めよ、という問題
- 有名問題
- 重心分解で $O(N \log N)$
 - 重心を通るパスについて $O(N)$ で処理できれば、
再帰的に行うことで全体 $O(N \log N)$ 。

5

満点

- よって T_i が一般の場合も重心分解が必要になると予想される。
- 重心を通るパスだけ考えれば良い。
- それぞれの頂点について、以下の情報を計算しておく(全て dp で計算できる)。
 - 重心からの距離
 - その頂点を時刻 0 に出発し重心に向かう途中でスタンプを押せるか
 - 重心を時刻 t に出発しその頂点に向かう途中でスタンプを押せるような t の最小値

5

満点

- 全体計算量は $O(N\log N)$ 。
- 実装の細かい部分を詰めるのは難しいですが、頑張りましょう。
- 細かい部分をサボったり無駄な $\log$ をつけたりすると sub5 だけ点が入るかもしれません。

得点分布

6

得点分布 (JOI)

• 72 点 : 2人

• 69 点 : 1人

• 31 点 : 15人

• 28 点 : 2人

• 21 点 : 1人

• 20 点 : 1人

• 17 点 : 2人

• 10 点 : 1人

• 3 点 : 1人

• 0 点 : 4人

6

得点分布 (JOIG)

- 48 点 : 2人

- 31 点 : 2人

- 26 点 : 1人

- 14点 : 1人

- 5 点 : 4人

- 0 点 : 5人

JOI 2025/26 ファイナル Day 4

パン職人
(Baker)

解説：渡邊雄斗 (yuto1115)

問題概要

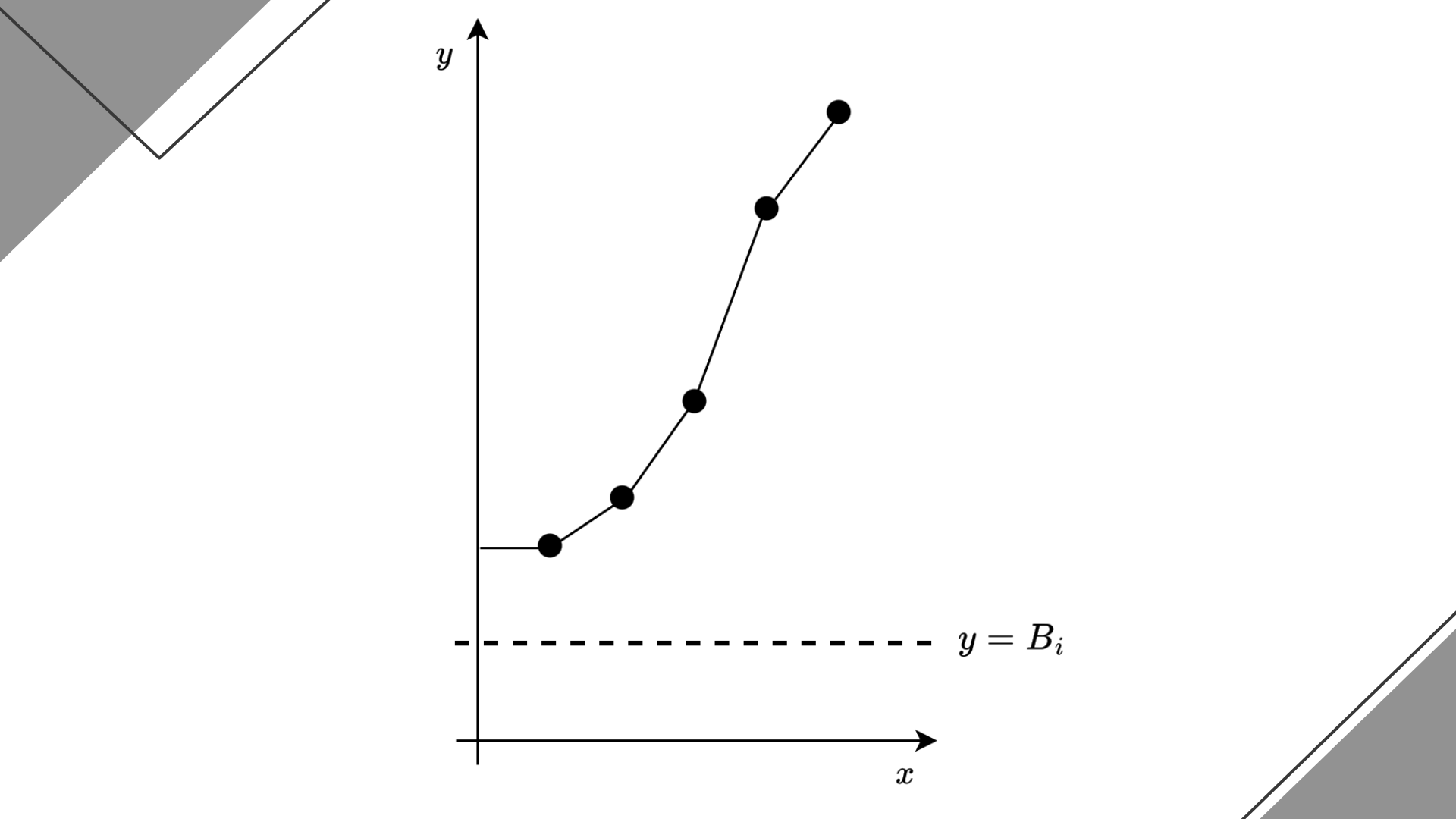
- M 人の客がいる 客 i は時刻 T_i にクロワッサンを注文し、
最大で時刻 $T_i + L$ まで待てる
- Q 個のクエリがある 各クエリでは A_q, B_q が与えられる
 - A_q 分でクロワッサンを作れる職人が 1 人、時刻 B_q に出勤
 - 出勤時刻までに入った注文のみ考慮する
 - 最大で何人の客の注文を捌けるか？
- 制約： $M \leq 2 \times 10^6$, $Q \leq 4 \times 10^5$

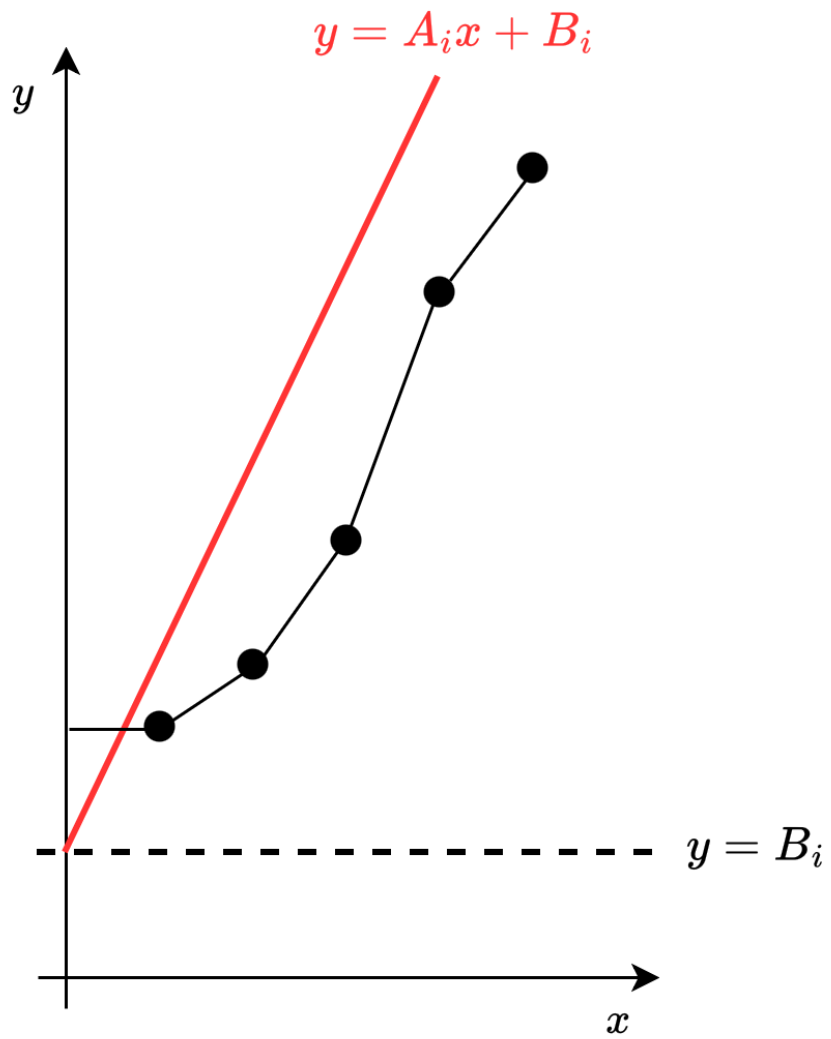
Subtask 1 : $M \leq 10, Q \leq 10^5$

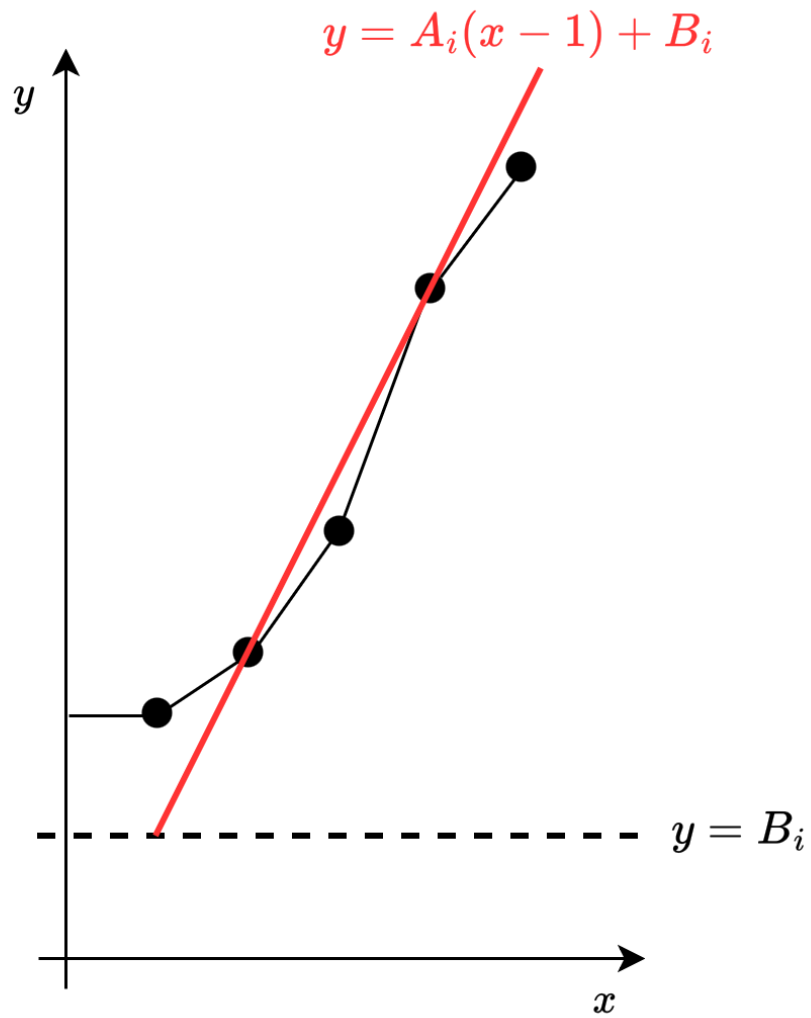
- 以下の値を導入します
 - $Y_i (1 \leq i \leq M) := T_i + L$: 客 i の締切時刻
 - $L_q, R_q (1 \leq q \leq Q)$: クエリ q が考慮する客の区間 $[L_q, R_q)$
 - $T_i \leq B_q \leq Y_i$ なる客が考慮される
- $k = 1, 2, \dots, R_q - L_q$ それぞれについて、 k 人の客を捌けるか判定したい
 - $R_q - k, R_q - k + 1, \dots, R_q - 1$ をこの順に貪欲に捌くのが最適
- $O(QM^2)$ 時間

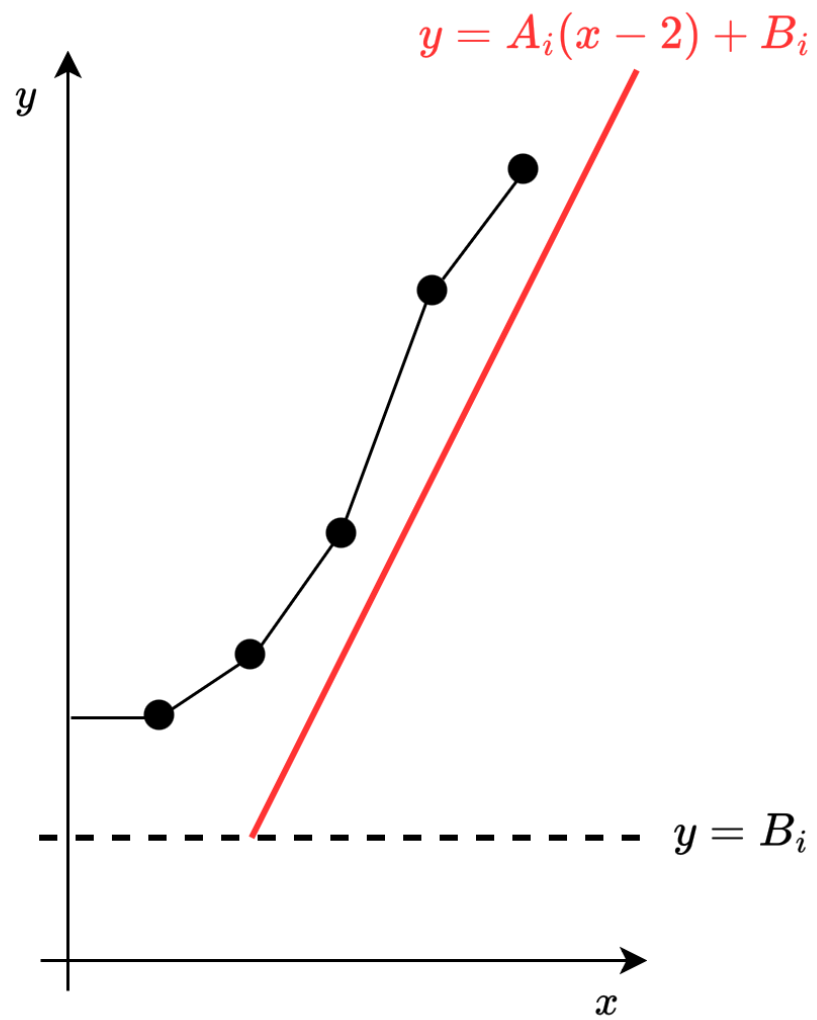
Subtask 2 : $M \leq 500, Q \leq 10^5$

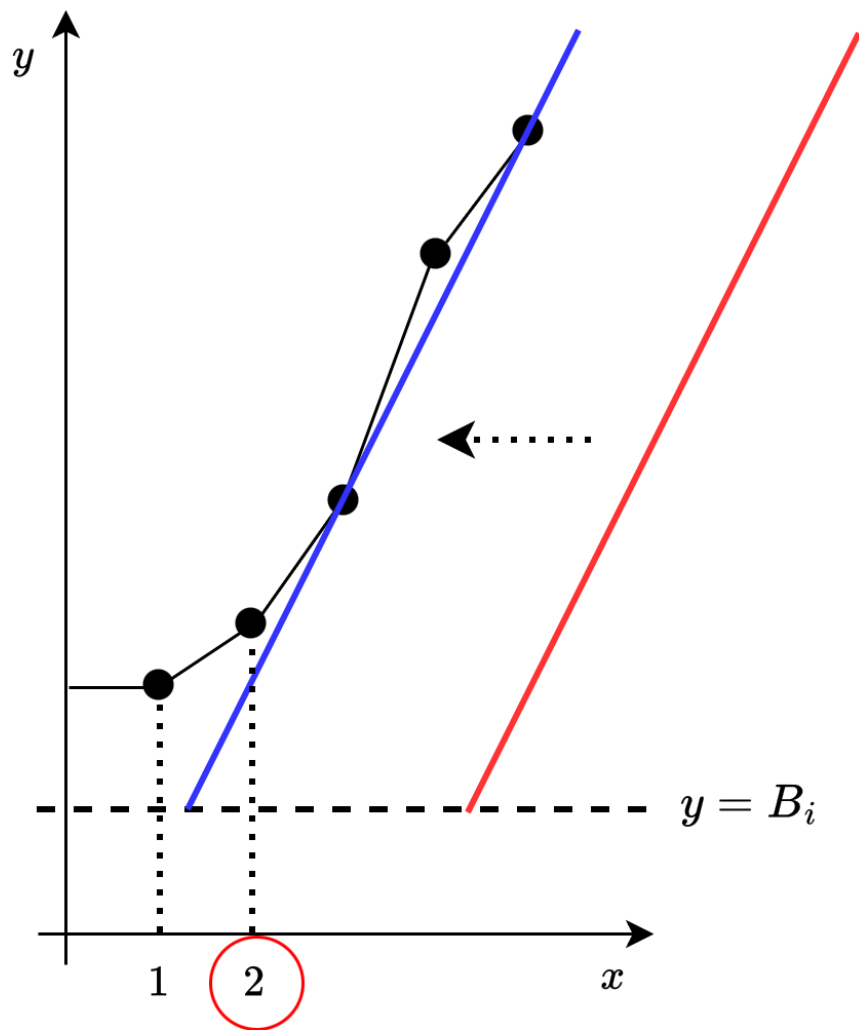
- 前から貪欲、Sub 1 + 二分探索など色々あるが、ここでは次につながる解法を紹介
- $k = 1, 2, \dots, R_q - L_q$ それぞれについて、 k 人の客を捌けるか判定したい
 - $R_q - k, R_q - k + 1, \dots, R_q - 1$ をこの順に貪欲に捌くのが最適
 - $\Leftrightarrow Y_i \geq B_q + A_q(i - R_q + k + 1) \quad (R_q - k \leq i < R_q)$
 - $\Leftrightarrow Y_i \geq B_q + A_q(i - R_q + k + 1) \quad (L_q \leq i < R_q)$
 - $\Leftrightarrow k \leq \frac{Y_i - B_q}{A_q} + R_q - 1 - i \quad (L_q \leq i < R_q)$
- 右辺の最小値の floor が答え $O(QM)$ 時間

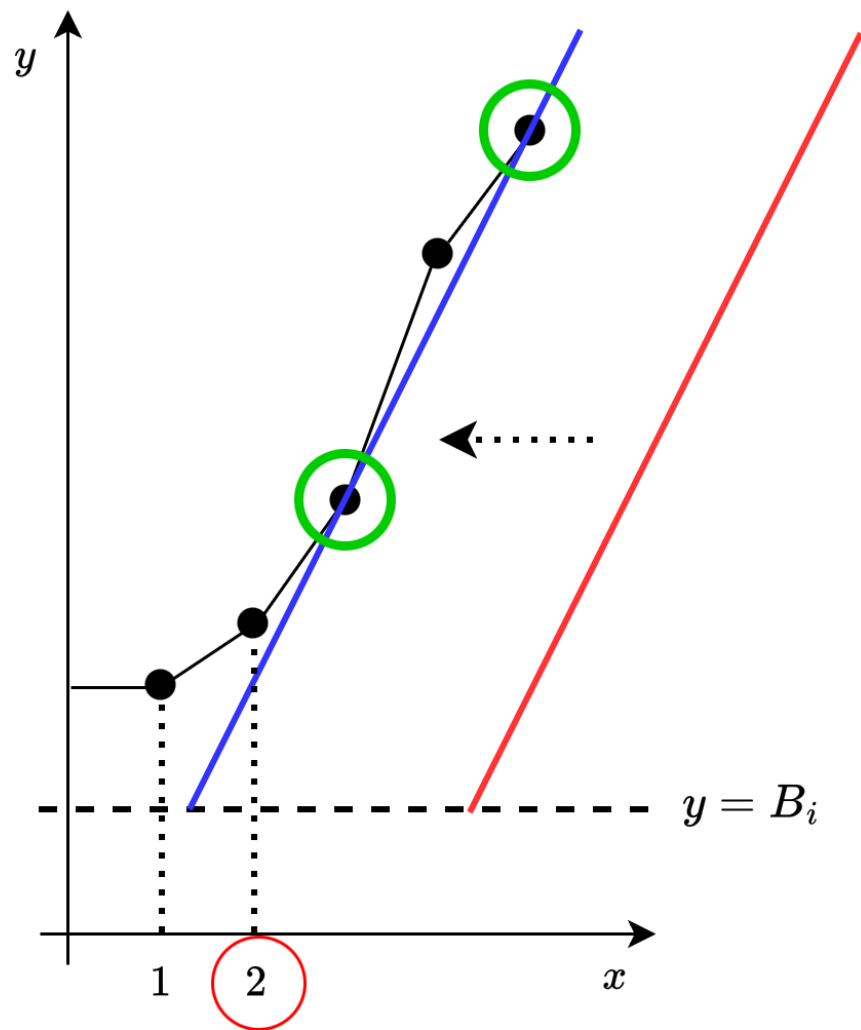












Subtask 2 : $M \leq 500, Q \leq 10^5$

- グラフを使って考える

- M 個の点 $(1, Y_1), (2, Y_2), \dots, (M, Y_M)$ がある ((i, Y_i) を点 i と呼ぶことにする)

- 捌きたい客の人数 k を固定すると、

それぞれの客に対するサーブ時刻は傾き A_q の直線で表される (k 小さいほど右に位置)

- 点 $[L_q, R_q)$ が全てこの直線の上 (境界含む) にあることが必要十分
- 言い換えると、傾き A_q の直線を右から動かした時、いつぶつかるか知りたい
- $-A_q x + y$ を最小にする点を見つければ、その点の情報から $O(1)$ で求解

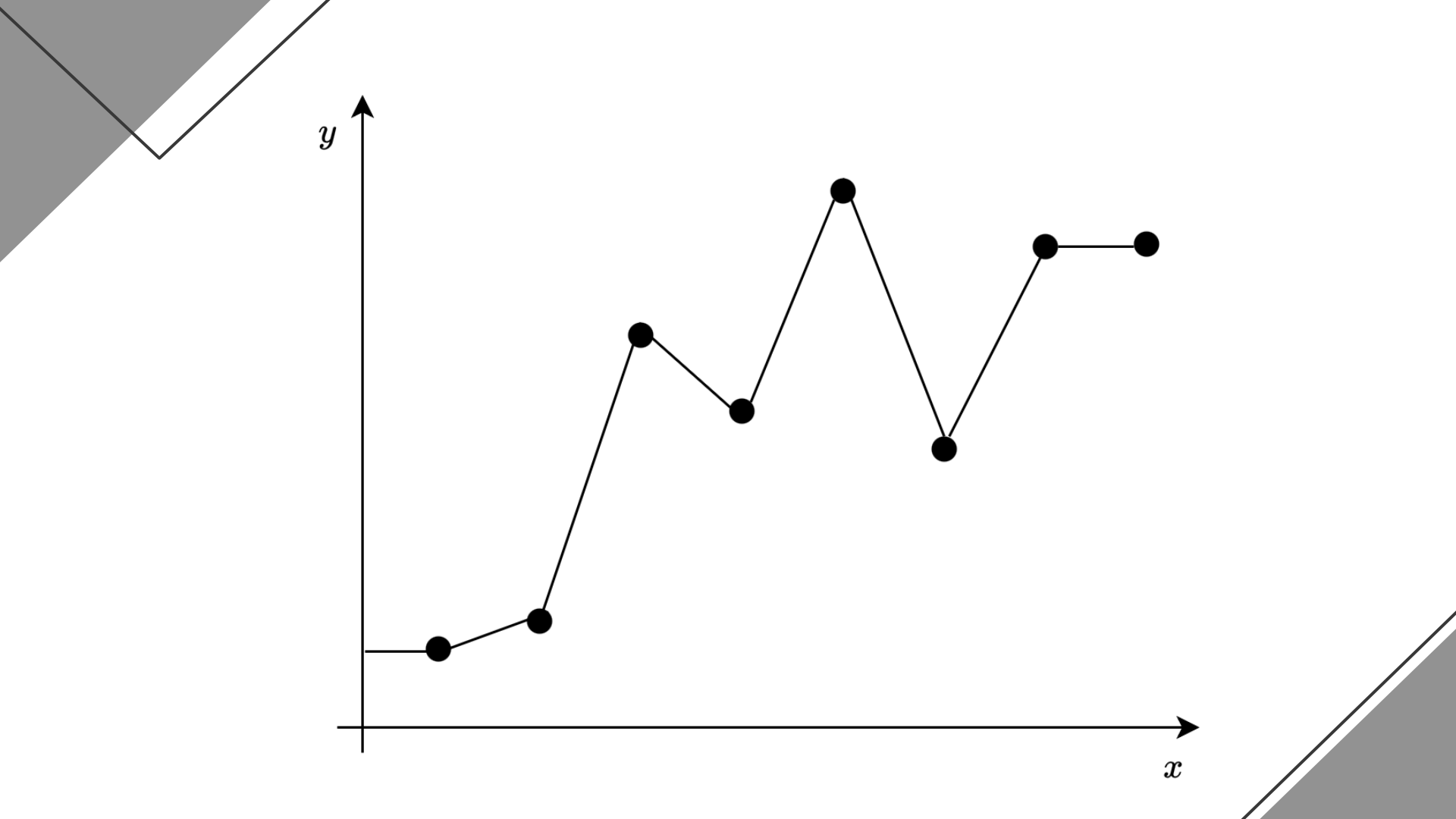
Subtask 3 : $T_M \leq B_q < T_1 + L$

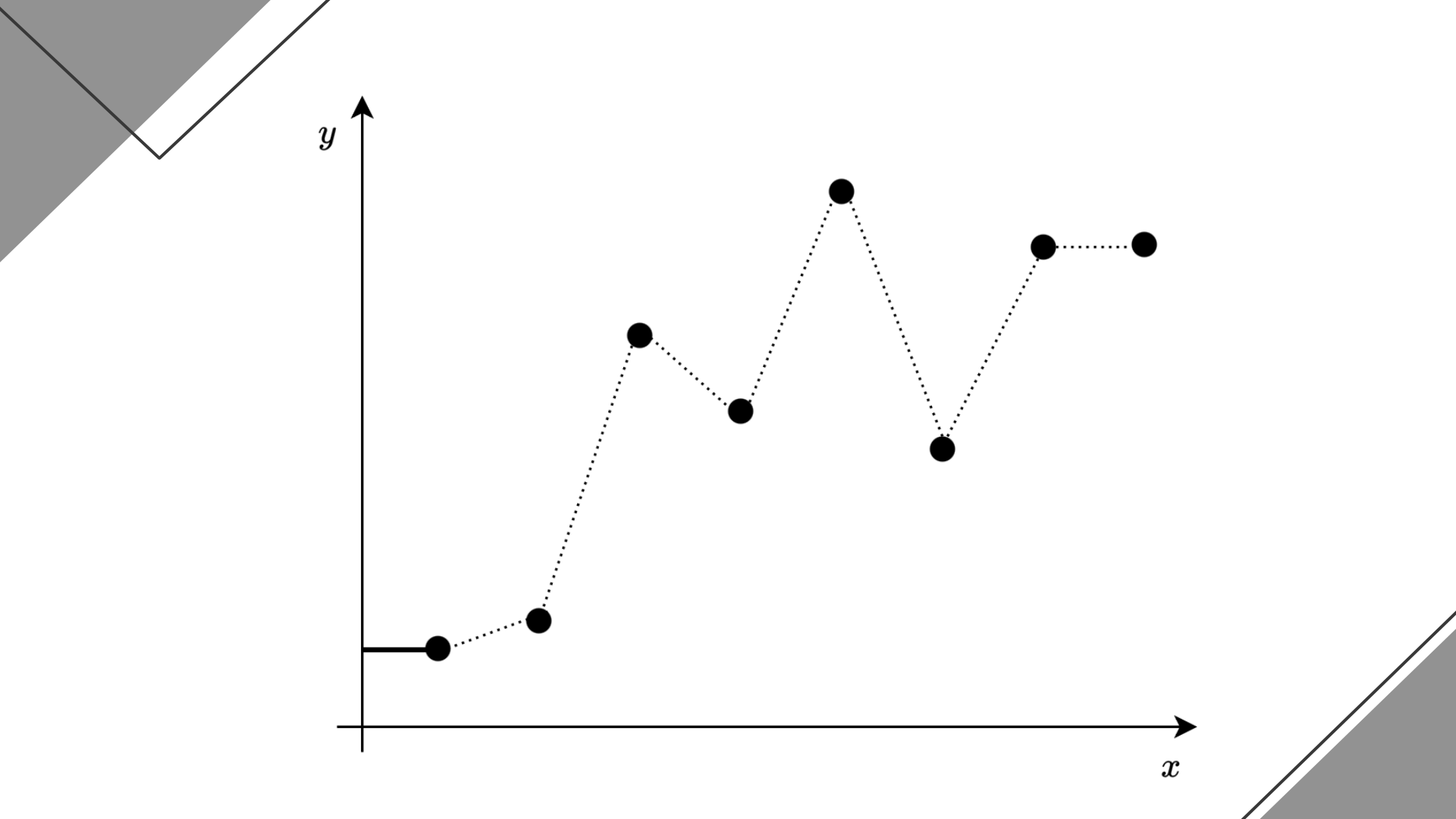
- 帰着後の問題の復習 :
 - M 個の点 $(1, Y_1), (2, Y_2), \dots, (M, Y_M)$ がある
 - 各クエリでは L_q, R_q, A_q が与えられるので、点 $[L_q, R_q)$ のうち、傾き A_q の直線を右から動かした時に最初にぶつかる点を知りたい
 - 本小課題では、常に $L_q = 1, R_q = M + 1$ (全ての点を考慮)

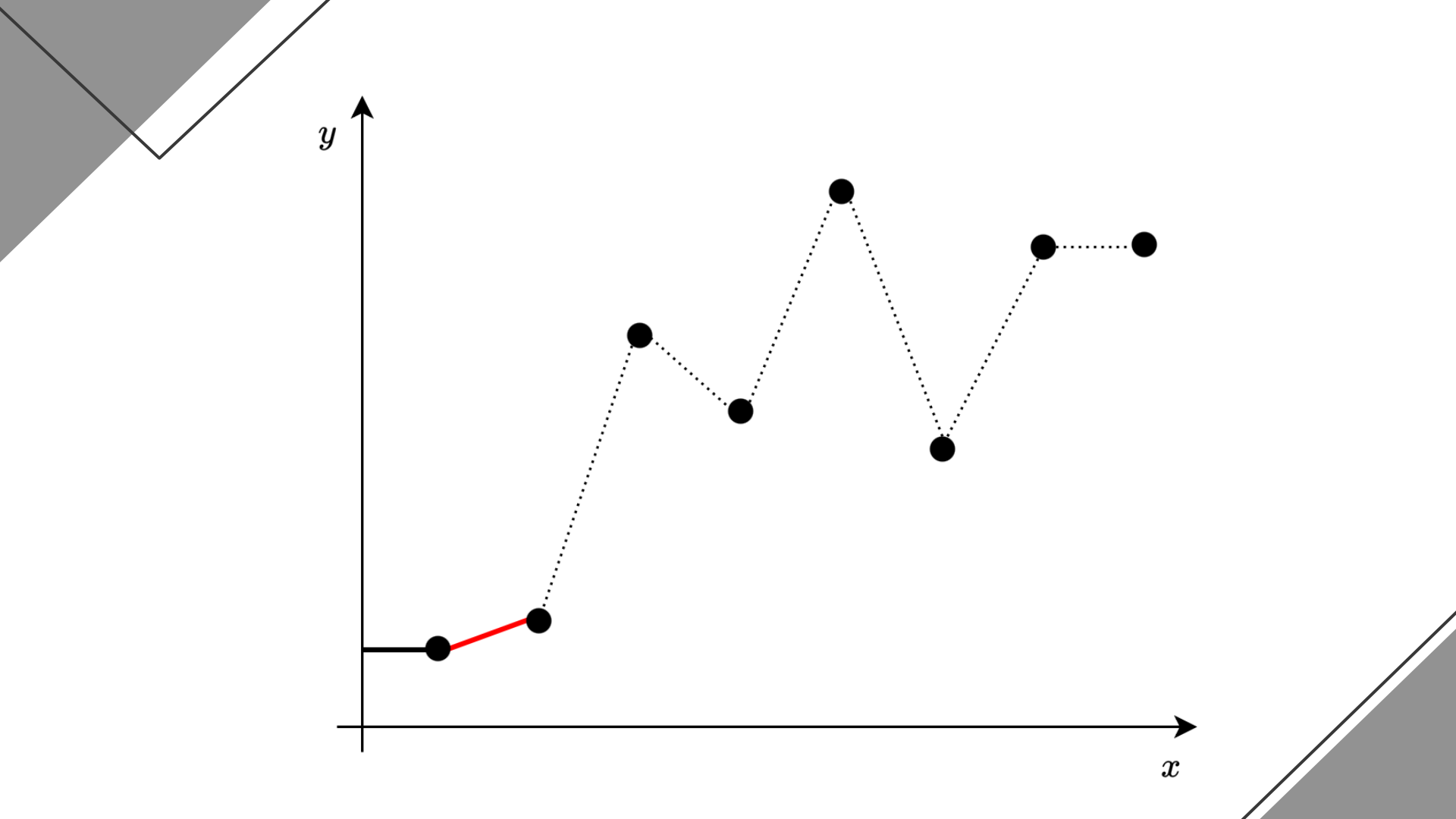
Subtask 3 : $T_M \leq B_q < T_1 + L$

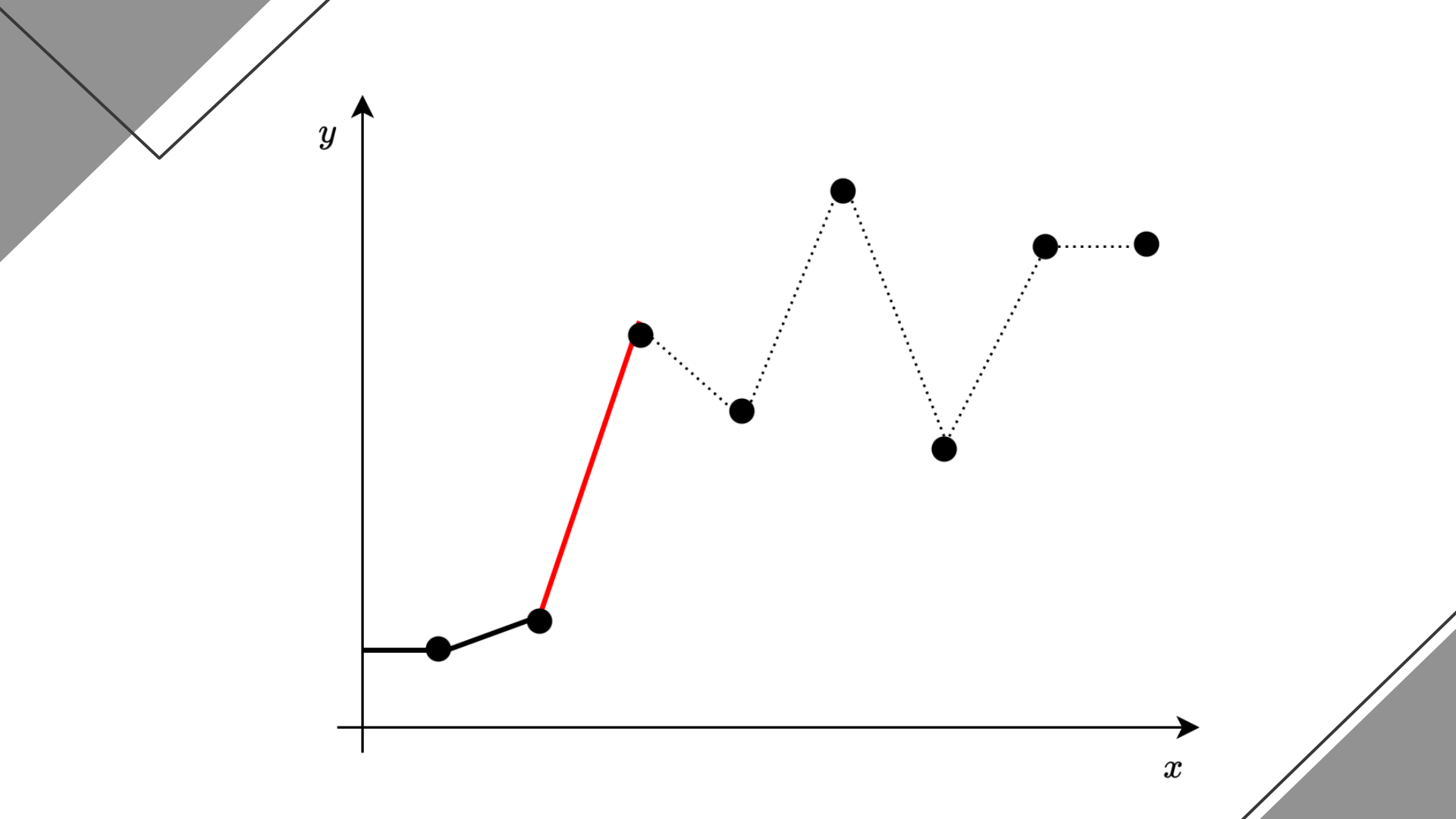
- 帰着後の問題の復習 :

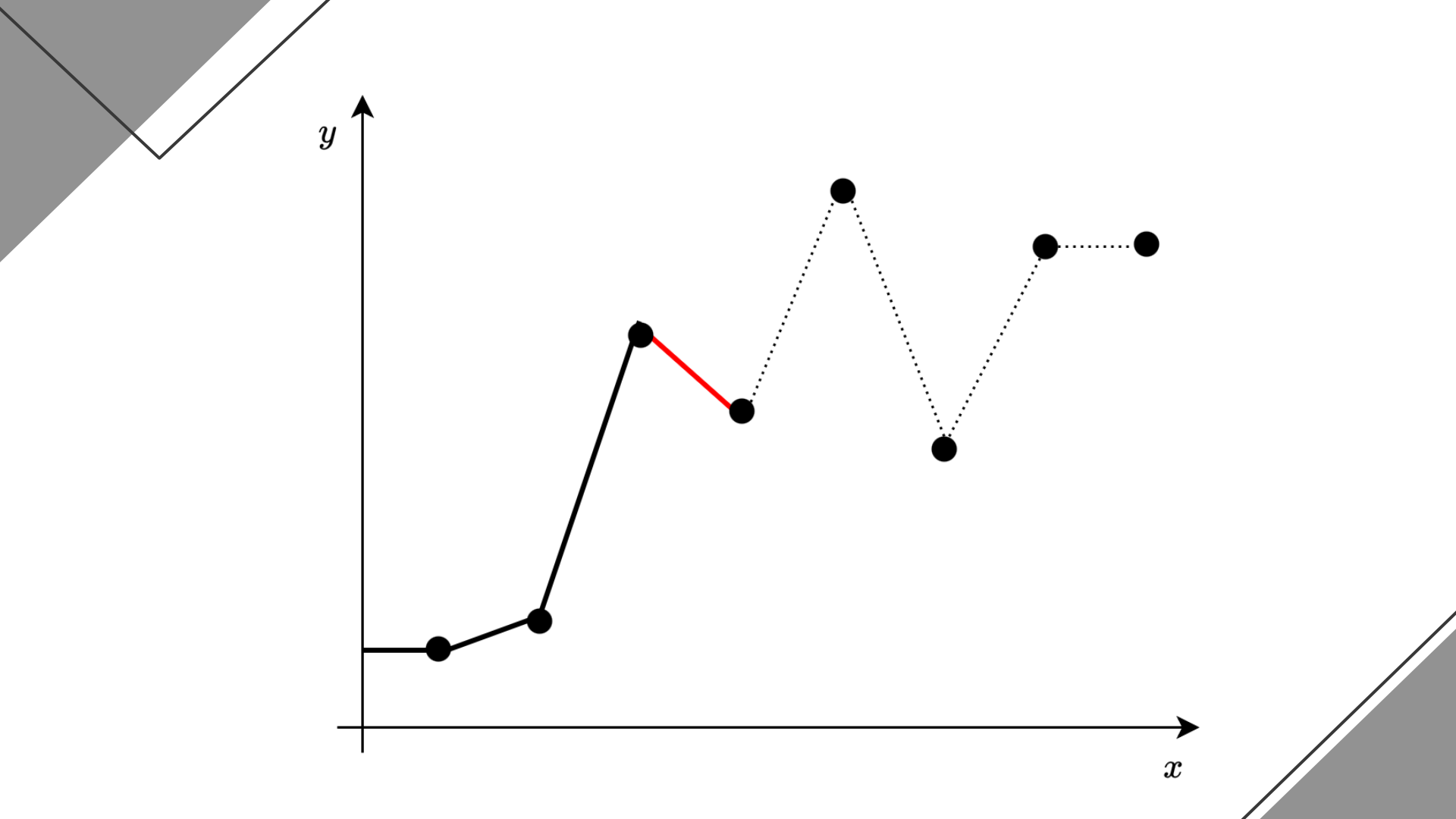
- M 個の点 $(1, Y_1), (2, Y_2), \dots, (M, Y_M)$ がある
- 各クエリで L_q, R_q, A_q が与えられるので、点 $[L_q, R_q)$ のうち、傾き A_q の直線を右から動かした時に最初にぶつかる点を知りたい
- 本小課題では、常に $L_q = 1, R_q = M + 1$ (すべての点を考慮)

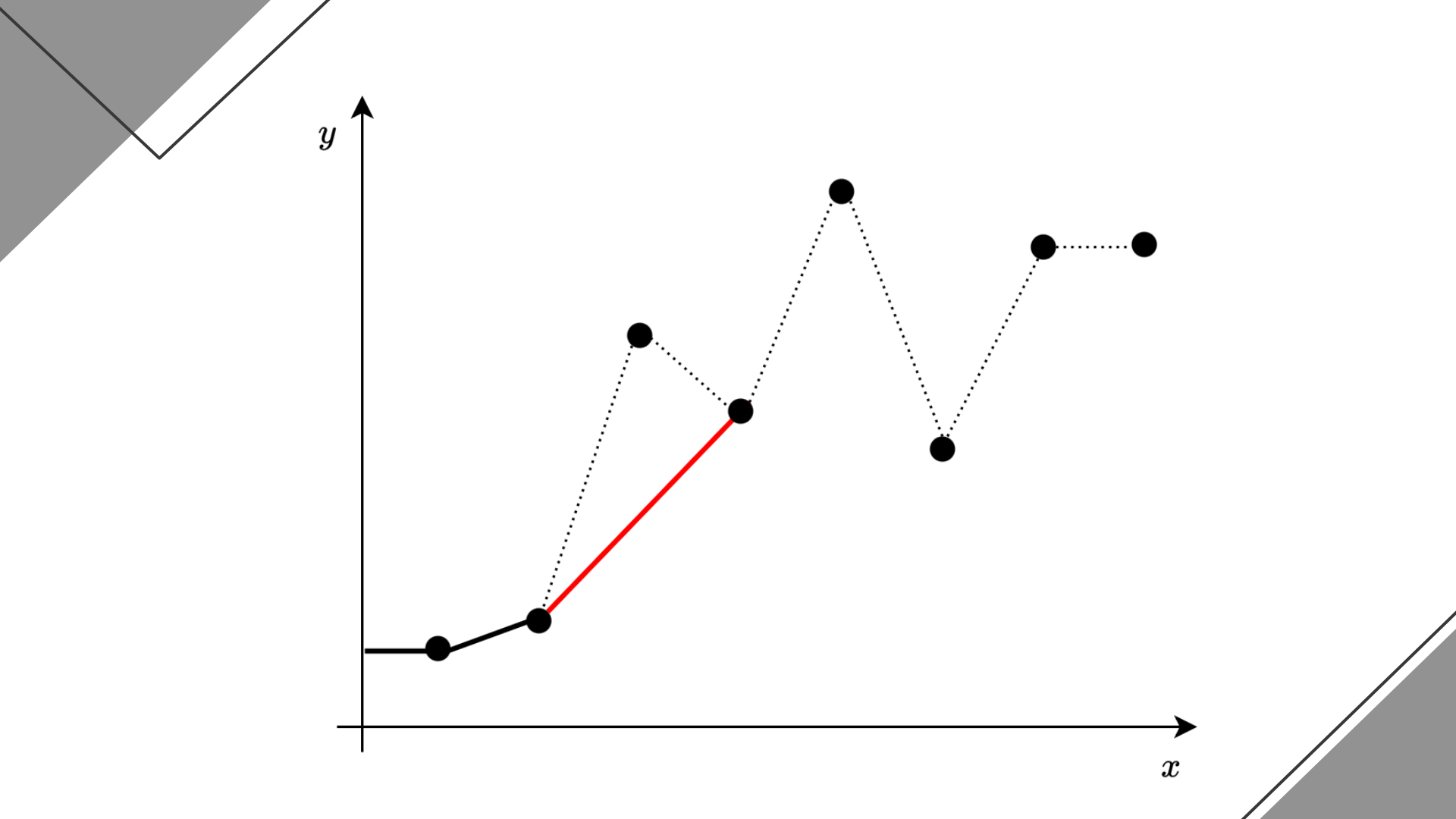


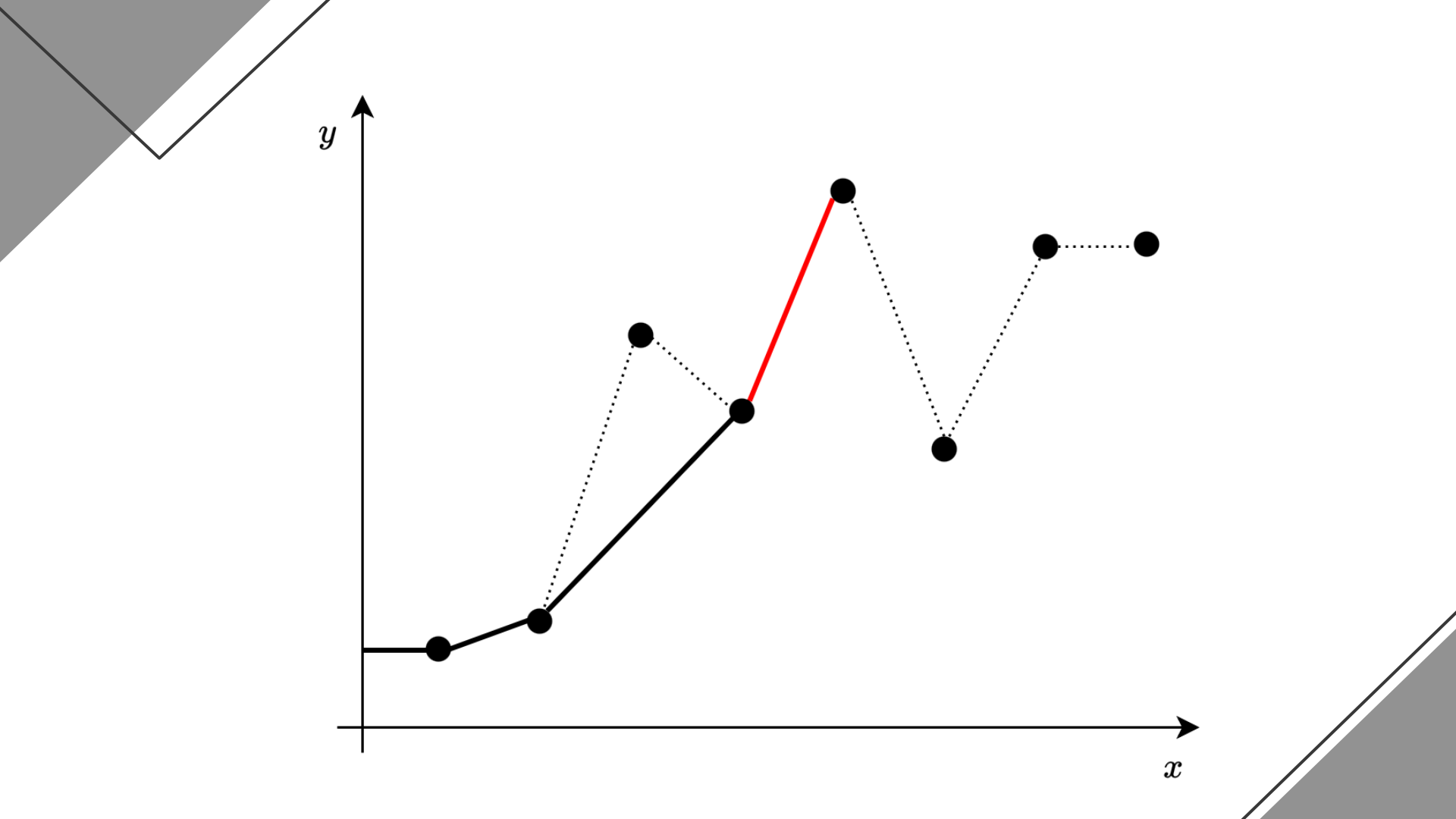


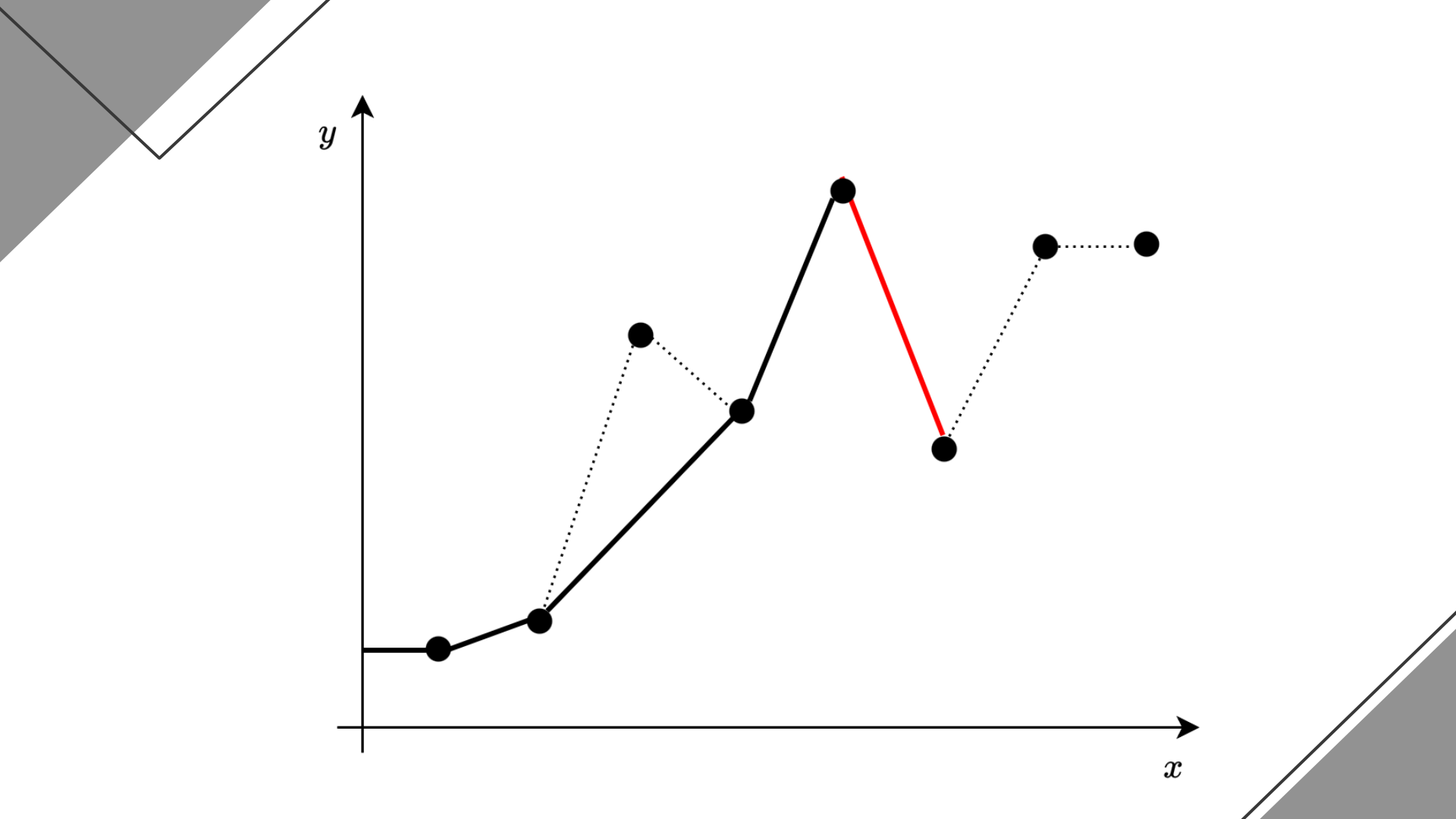


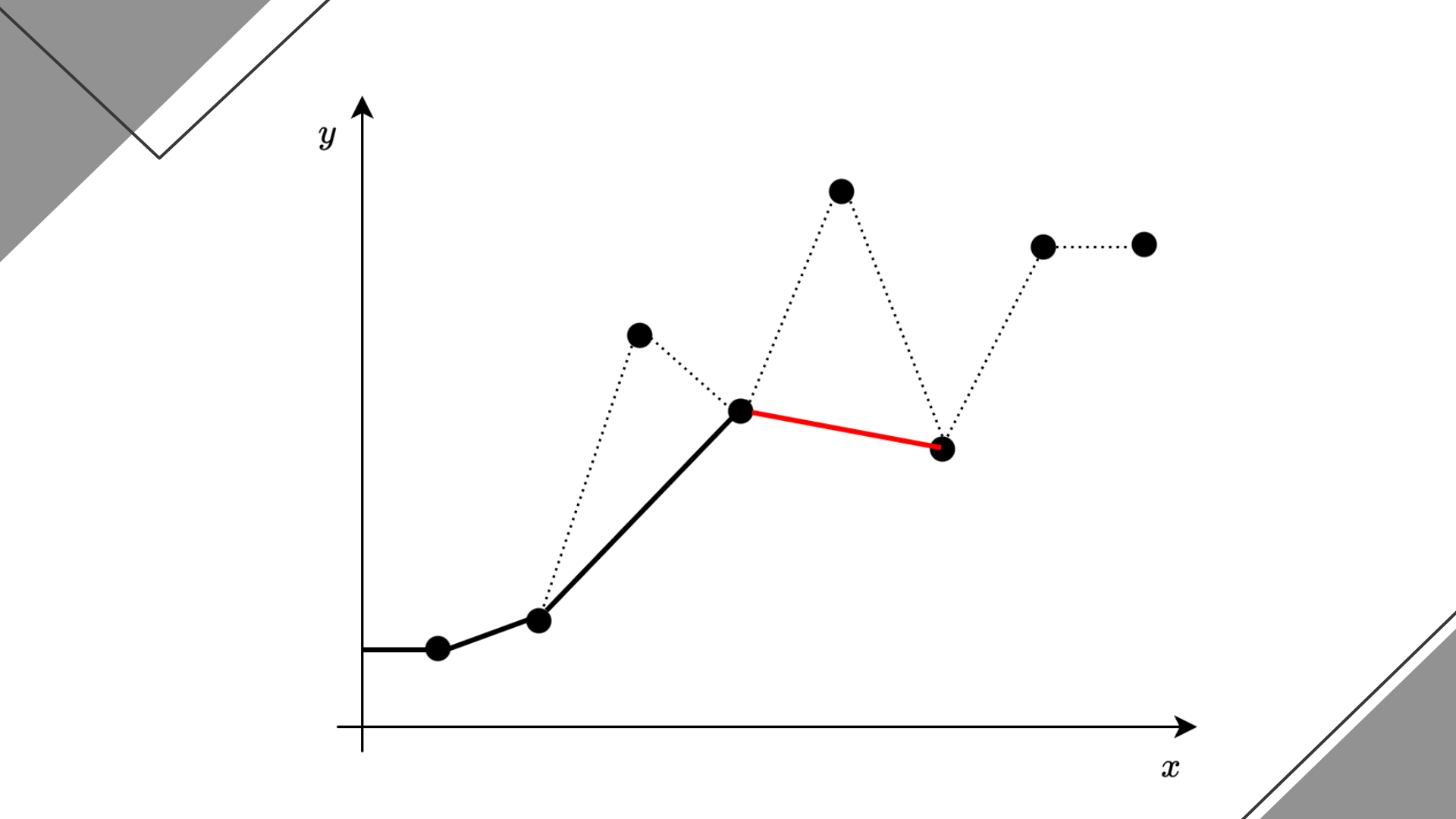


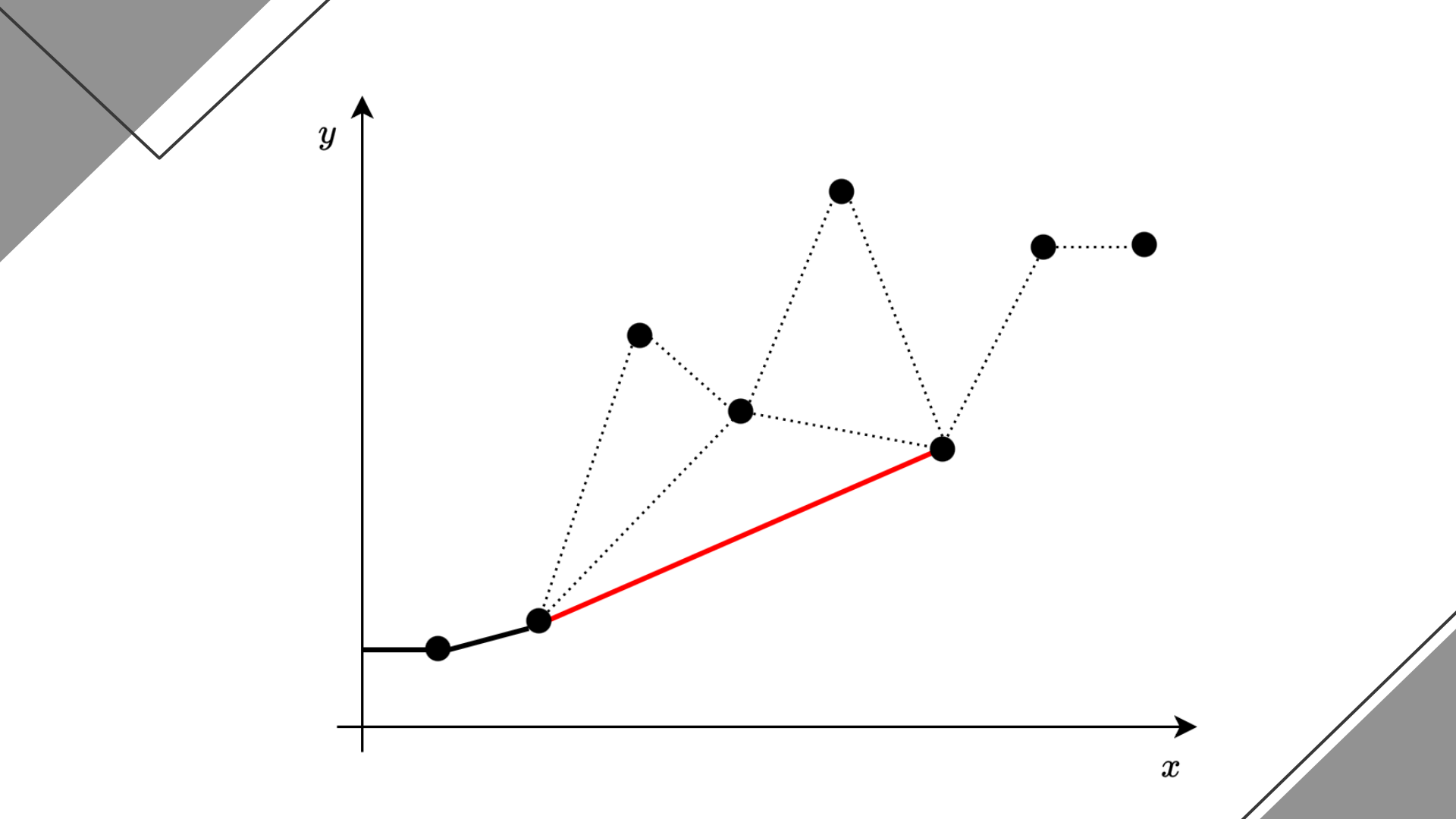


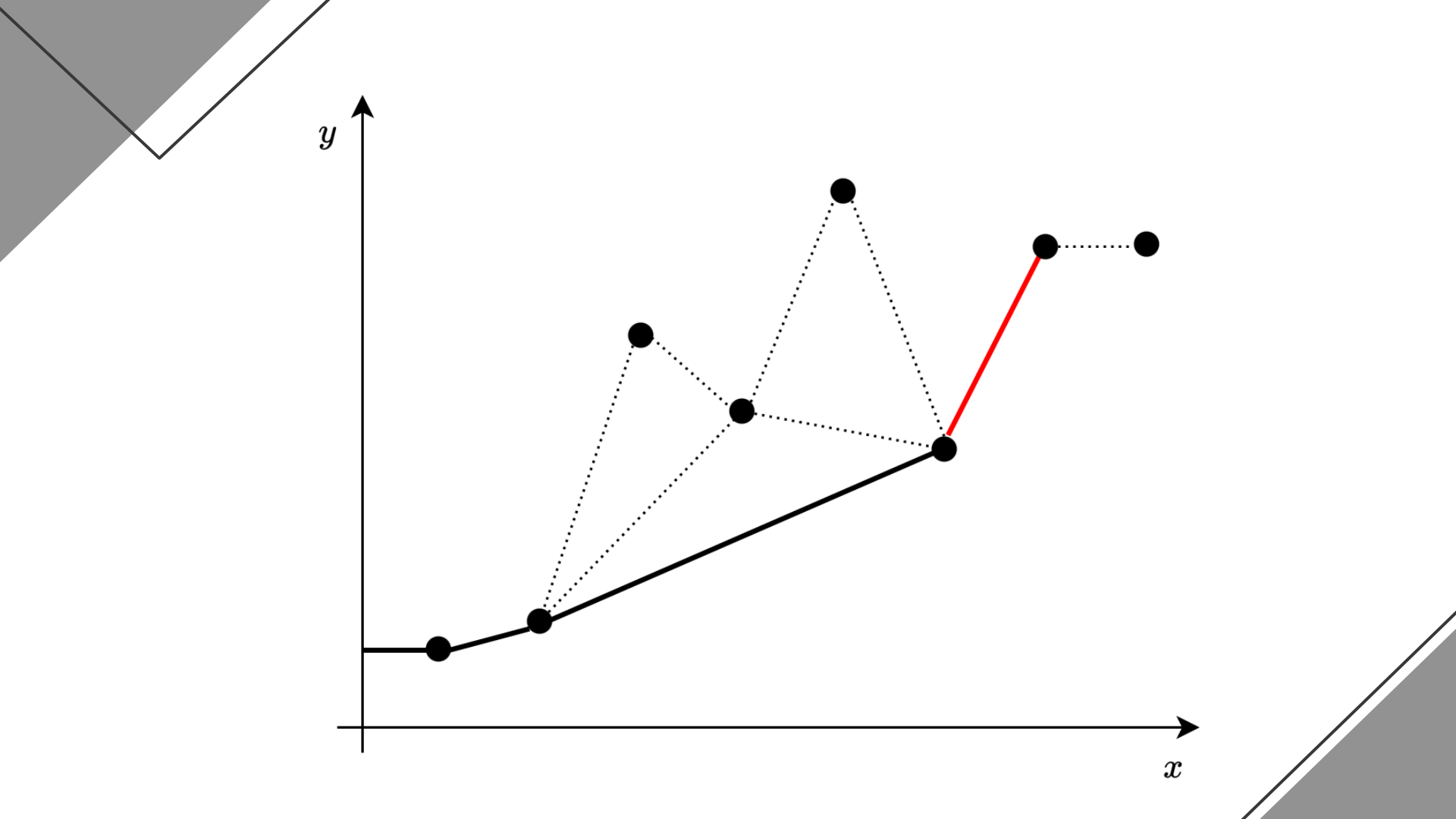


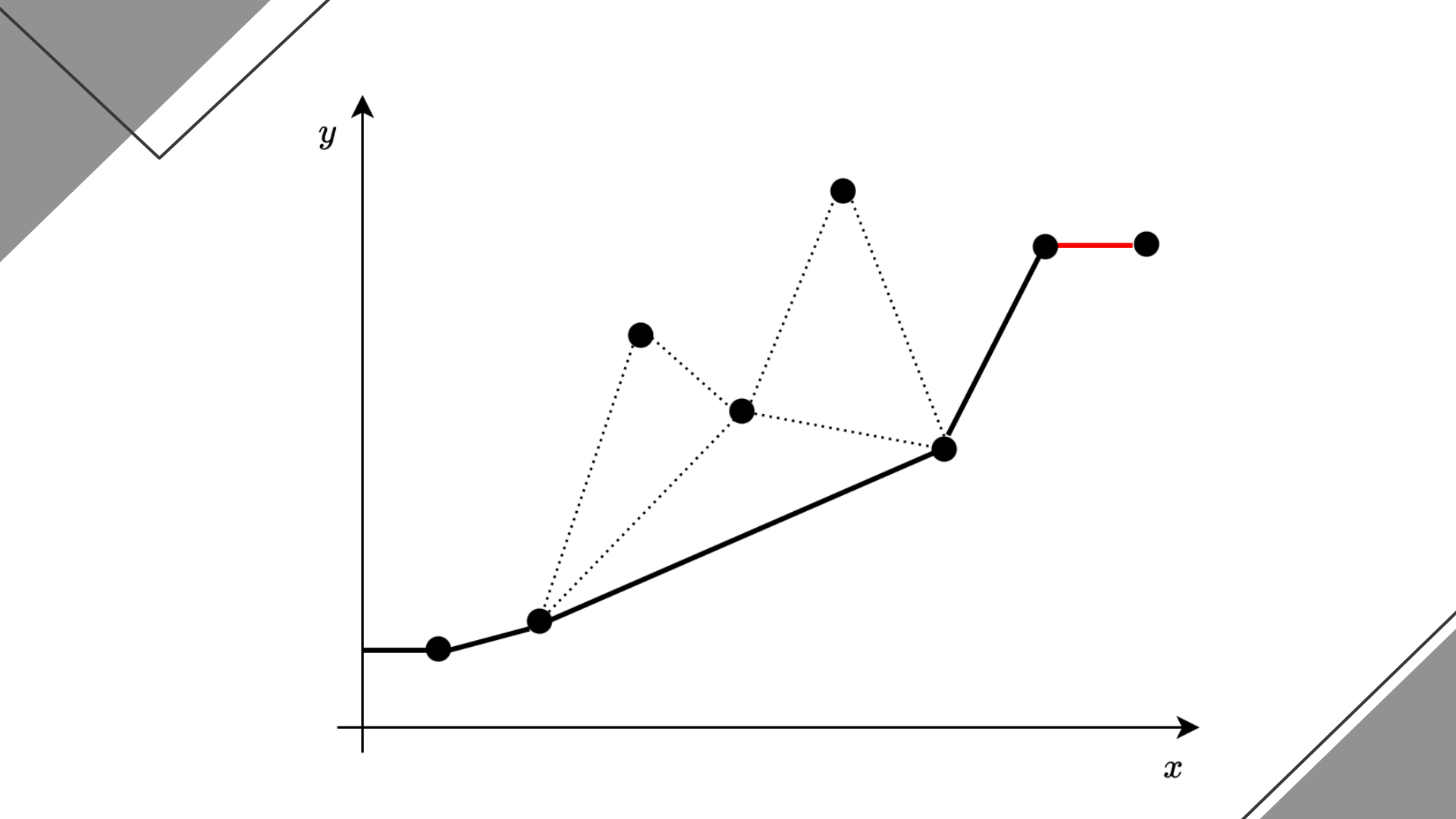


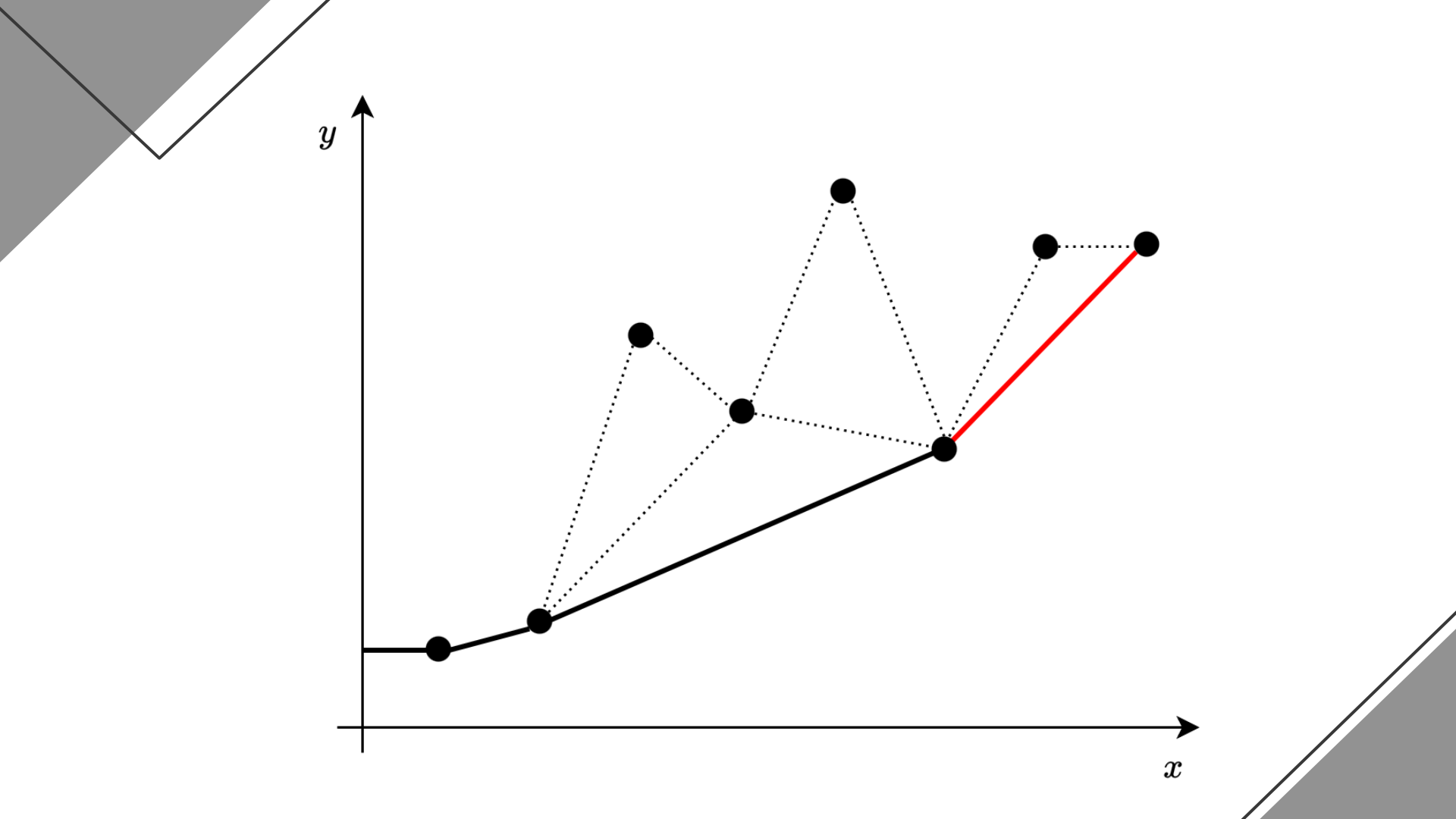


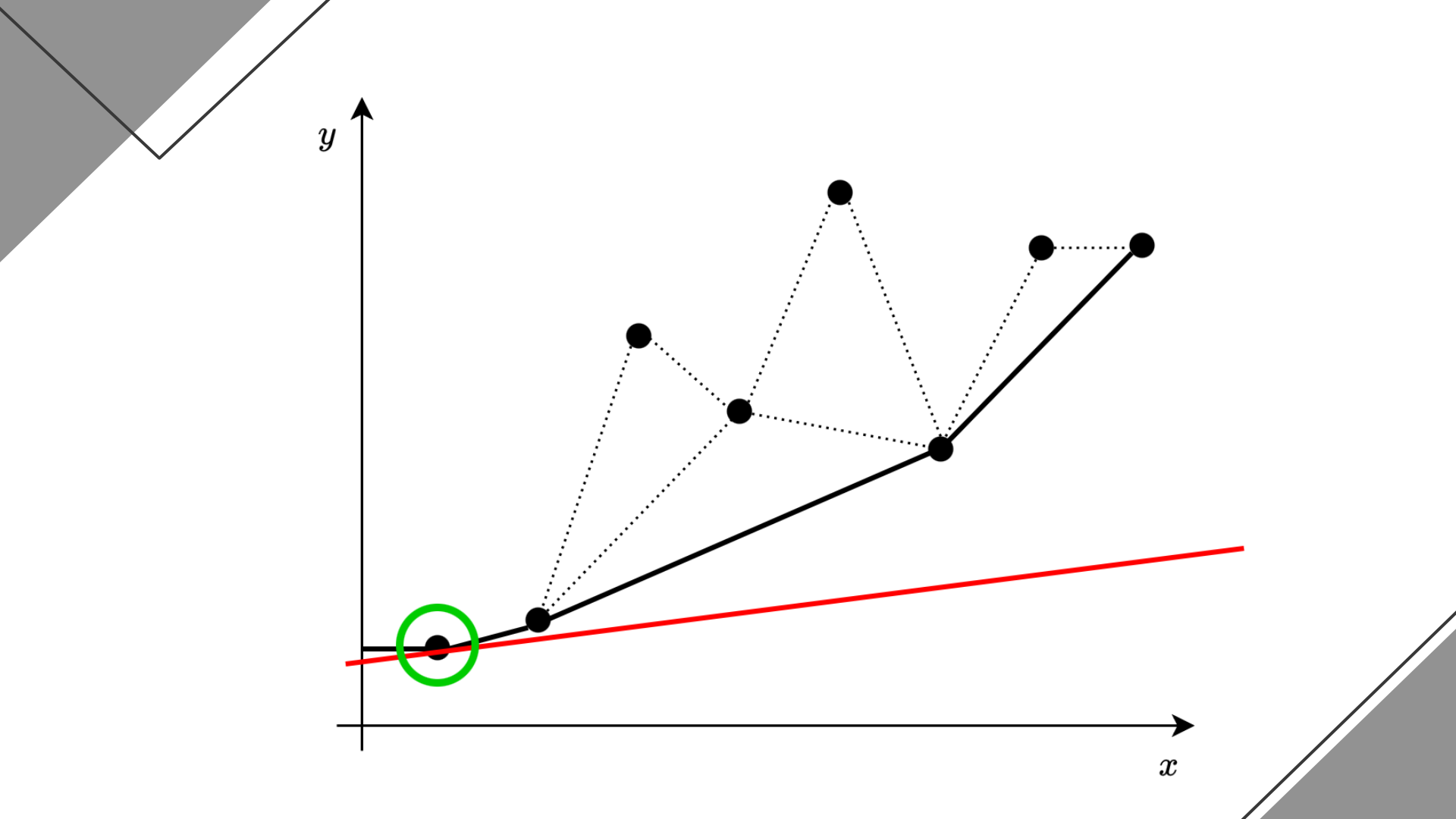


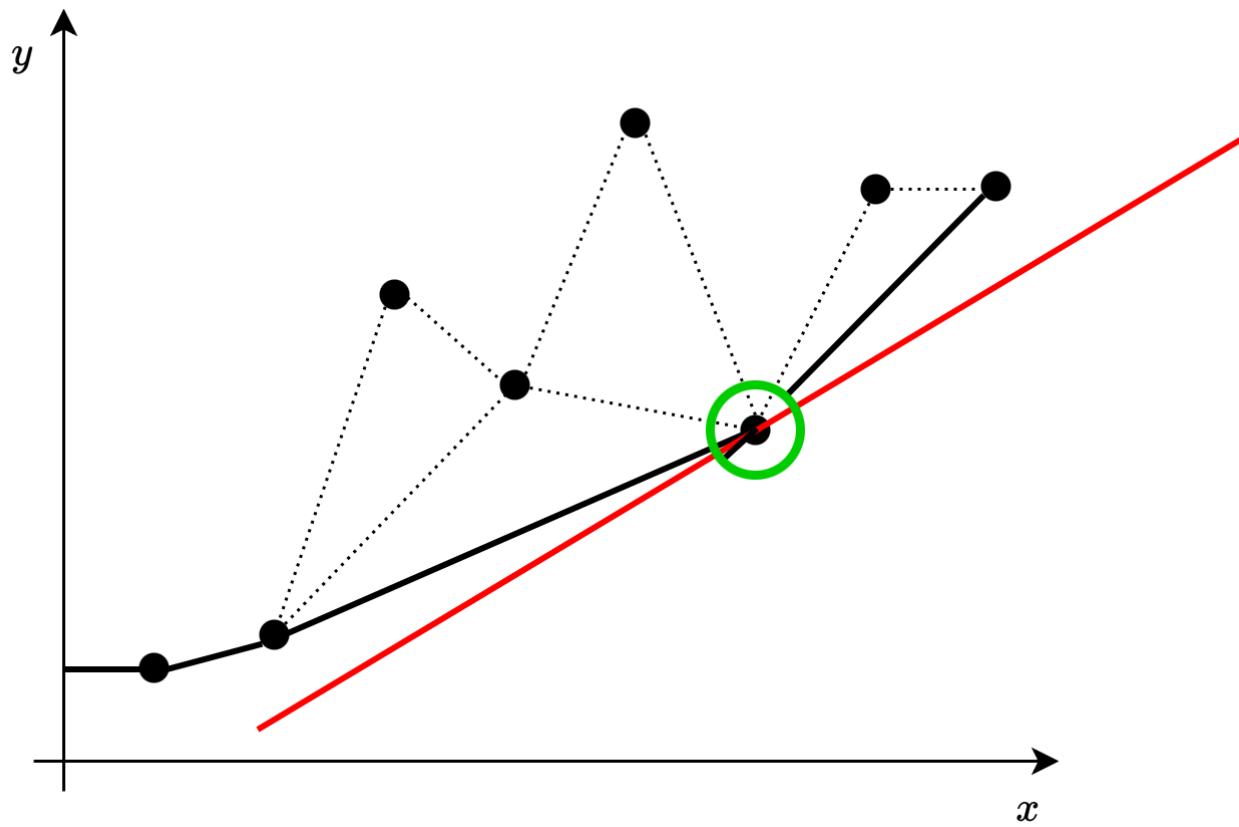










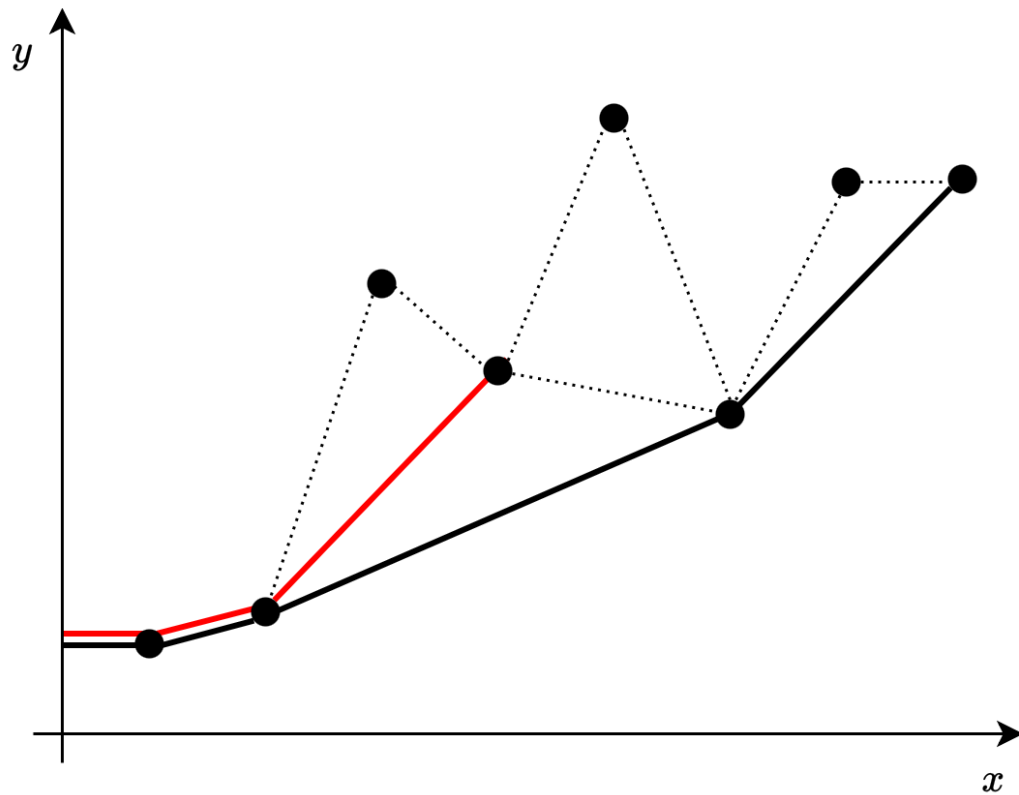


Subtask 3 : $T_M \leq B_q < T_1 + L$

- 与えられた M 個の点の（下側）凸包を前計算（元々ソートされているので線形時間で可能）
- 各クエリについて、
 - 傾き A_q の直線を右から動かした時に最初にぶつかる点を知りたい（復習）
 - 凸包に含まれる点のみを考慮すればよい（証明略）
 - 凸包を構成する線分（傾きの昇順に並んでいる）について、
 A_q 未満から以上に変化する場所を二分探索で見つける
- クエリあたり $O(\log M)$ 時間

Subtask 4 : $T_M \leq B_q$

- 先程との違い : L_q が任意の値を取るようになった ($R_q = M + 1$ は保たれたまま)
- 点 $L_q, L_q + 1, \dots, M$ のみを考慮した凸包上で二分探索をしたい
 - ~~全体の凸包から該当範囲内の点を取り出すだけ？~~



— 全体の凸包

— 点 1 - 4 の凸包

Subtask 4 : $T_M \leq B_q$

- 先程との違い : L_q が任意の値を取るようになった ($R_q = M + 1$ は保たれたまま)
- 点 $L_q, L_q + 1, \dots, M$ のみを考慮した凸包上で二分探索をしたい
 - ~~全体の凸包から該当範囲内の点を取り出すだけ？~~
- 点 $M, M - 1, \dots, 1$ の順に点を追加していきながら凸包を構築することを考える
 - 先程のアルゴリズムの逆バージョン
 - 全ての l について、点 $l, l + 1, \dots, M$ のみを考慮した凸包が途中で登場！
- クエリを L_q の昇順にソートして、全体 $O(M + Q(\log Q + \log M))$ 時間

Subtask 5 : $M \leq 5 \times 10^5, Q \leq 10^5$

- L_q も R_q も固定されなくなってしまった！
 - 点 $L_q, L_q + 1, \dots, R_q$ における $-A_q x + y$ の最小値を求めたい
 - 区間クエリなので、Segment Tree のようなことをしてみたい
- Segment Tree の各ノードについて、その区間内の点からなる凸包を前計算
- 各クエリでは、 $[L_q, R_q)$ を $O(\log M)$ 個の区間に分解し、
各区間において凸包上の二分探索で $-A_q x + y$ の最小値を求める
- 全体 $O(M \log M + Q \log^2 M)$ 時間

Subtask 6 : $M \leq 2 \times 10^6, Q \leq 4 \times 10^5$ (満点)

- 複数の解法があるが、ここではそのうちの一つを紹介（他の例：分割統治）
- 先程の Segment Tree 解法では、任意の区間に対するクエリを処理できた
 - 実際には L_q, R_q は何でも好きに選べるわけではない
 - $L_q = \min i \text{ s.t. } T_i \geq B_q - L ; \quad R_q = \max i \text{ s.t. } T_i \leq B_q ;$
 - クエリを B_q の昇順にソートすると、 L_q と R_q も昇順にソートされる

Subtask 6 : $M \leq 2 \times 10^6, Q \leq 4 \times 10^5$ (満点)

- 以下、 $L_1 \leq L_2 \leq \dots \leq L_Q, R_1 \leq R_2 \leq \dots \leq R_Q$ を仮定
- 与えられた点列の連続部分列 $[L_1, R_1], [L_2, R_2], \dots, [L_Q, R_Q]$ それぞれについて凸包を知りたい
- やりたいこと：空の点列から始めて、
 - 末尾への点の追加
 - 先頭からの点の削除
 - 現在の点列における凸包へのアクセス（具体的には、二分探索）
- Queue のような操作

Subtask 6 : $M \leq 2 \times 10^6, Q \leq 4 \times 10^5$ (満点)

- ところで、以下は実現可能だろうか？
- 空の点列から始めて、
 - 末尾への点の追加
 - **末尾**からの点の削除
 - 現在の点列における凸包へのアクセス

Subtask 6 : $M \leq 2 \times 10^6, Q \leq 4 \times 10^5$ (満点)

- ところで、以下は実現可能だろうか？
- 空の点列から始めて、
 - 末尾への点の追加
 - **末尾**からの点の削除
 - 現在の点列における凸包へのアクセス
- できる（各点を追加する際に、その追加によって削除された点を記録しておく）
- Stack のような操作

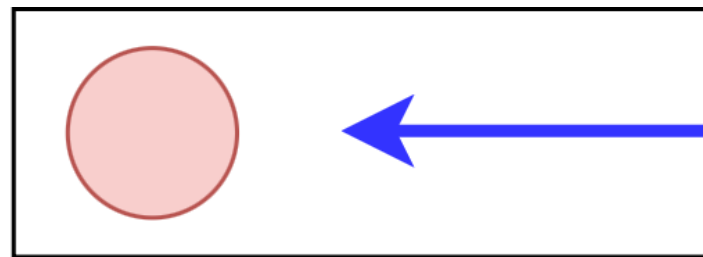
Subtask 6 : $M \leq 2 \times 10^6, Q \leq 4 \times 10^5$ (満点)

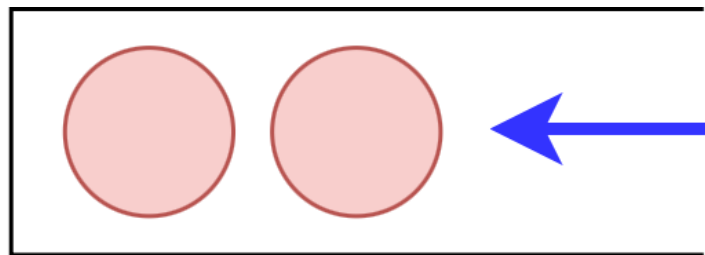
- やりたいのは Queue のような操作 (末尾追加・先頭削除・凸包アクセス)
- Stack のような操作はできる (末尾追加・末尾削除・凸包アクセス)
- つまり……?

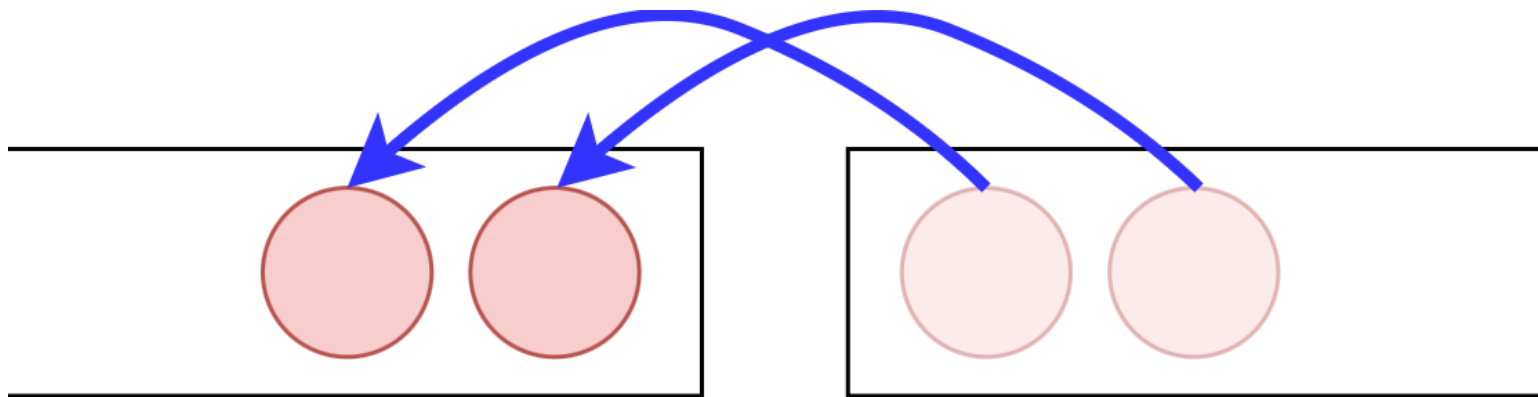
Subtask 6 : $M \leq 2 \times 10^6, Q \leq 4 \times 10^5$ (満点)

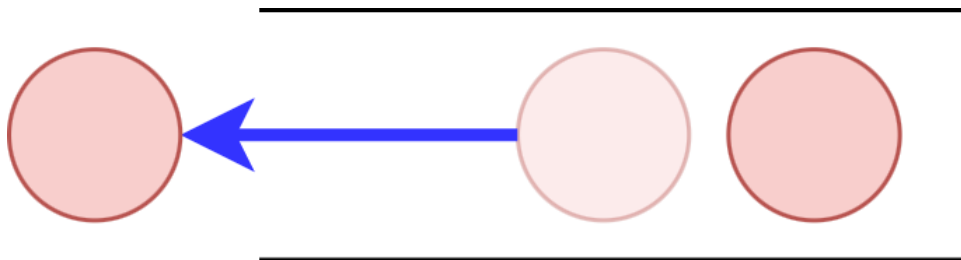
- やりたいのは Queue のような操作 (末尾追加・先頭削除・凸包アクセス)
- Stack のような操作はできる (末尾追加・末尾削除・凸包アクセス)
- つまり……?

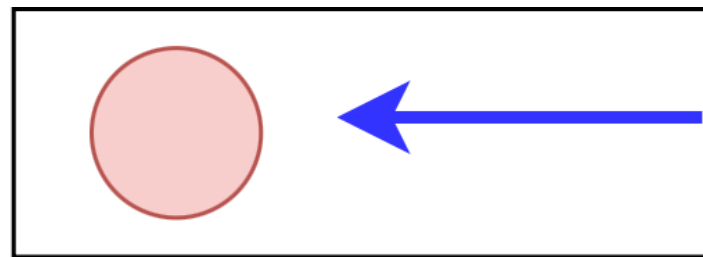
Two Stacks Queue











Subtask 6 : $M \leq 2 \times 10^6, Q \leq 4 \times 10^5$ (満点)

- Two Stacks Queue
 - 2 つのスタックを背中合わせにくっつけて Queue と同等の操作を実現
 - 末尾追加 : 末尾用スタックに追加
 - 先頭削除 : 先頭用スタックが空でないならそこから削除
空なら、一旦末尾用スタックの中身を全て先頭用に移動する
 - 各要素は高々 1 回しか移動されないなので、ならし $O(1)$

Subtask 6 : $M \leq 2 \times 10^6, Q \leq 4 \times 10^5$ (満点)

- これを応用して、各スタックにおいて点列とそれらの凸包を両方管理すれば OK

- クエリ処理：両スタックの凸包それぞれに対して二分探索

- 移動処理（先頭スタックが空のときの先頭削除）：

末尾用スタックに含まれる**元の点列**を参照して、線形時間で**一から**凸包構築

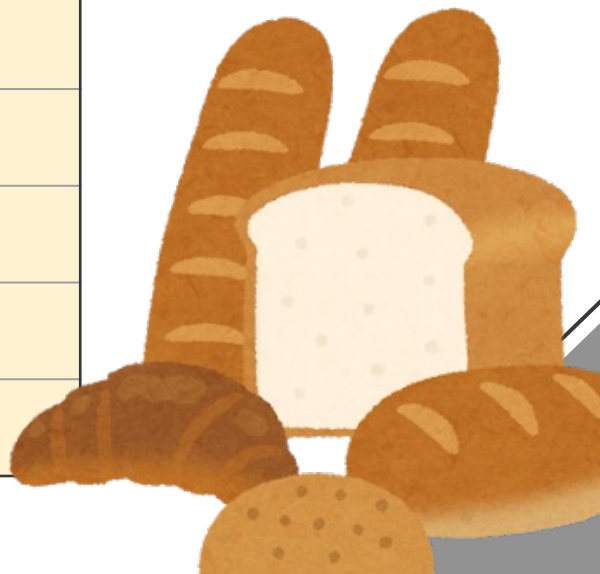
（末尾用スタックの凸包をそのままコピーするだけだと先頭削除ができない！）

- 全体 $O(M + Q(\log Q + \log M))$ 時間

得点分布



得点	小課題	人数
100	1, 2, 3, 4, 5, 6	0
82	1, 2, 3, 4, 5	1
72	1, 2, 3, 5	1
60	1, 2, 3, 4	3
50	1, 2, 3	5
42	1, 2, 5	1
20	1, 2	15
0	∅	4



JOI 国のお祭り事情 3

Festivals in JOI Kingdom 3

平木 康傑

JOI/JOIG チューター

2026/03/24 (Contest 4)

問題概要

問題文

N 頂点の無向木がある．各頂点 v に人気度 C_v ，各辺 e に長さ D_e が設定されている．頂点 v が点火する時刻は以下のように決まる．

- $C_v = 0$ の場合，0．
- $C_v \geq 1$ の場合，隣接する頂点 u に対する「(頂点 u の点火時刻) + (辺 (v, u) の長さ)」のうち C_v 番目に小さい値．

Q 個のクエリを順に処理せよ．クエリは以下の 3 種類．

- 1 頂点 v の人気度 C_v を x に変える．
- 2 辺 e の長さ D_e を x に変える．
- 3 現在の状態の木において，頂点 v が点火する時刻を求める．

制約: $N, Q \leq 1.5 \times 10^5$, $D_e \leq 10^6$

用語の定義

通常の (要素の重複を許さない) 集合 $\{\cdot\}$ に対し, 多重集合 (要素の重複を許す) を $[\cdot]$ で表す.

非負整数および $+\infty$ からなる多重集合 S と非負整数 k に対し, 関数 $\text{kth}(S, k)$ を以下で定める.

$$\text{kth}(S, k) := \begin{cases} 0 & (k = 0) \\ S \text{ の要素のうち } k \text{ 番目に小さい値} & (1 \leq k \leq |S|) \\ +\infty & (k > |S|) \end{cases}$$

この表記を使うと, 頂点 v の点火時刻を $x_v^\top$ とおいたとき

$$x_v^\top = \text{kth} \left(\left[x_u^\top + D_e \mid \text{頂点 } u, v \text{ は辺 } e \text{ で隣接している} \right], C_v \right)$$

と書ける.

小課題 1 ($N, Q \leq 2\,000$) - 愚直解法

パレードが頂点に着く時刻を随時 priority queue に入れていき、「頂点に来たパレードが C_v 個に達した時点で隣接頂点にパレードを飛ばす」をシミュレーションする解法が考えられる． $O(NQ \log N)$ 時間で、小課題 1 に通る (6 点)．

小課題 1 ($N, Q \leq 2000$) - 木 DP による解法

各頂点の点火時刻が相互に影響しあっていて、そのままでは愚直解から発展させるべくい。影響が一方向的であってほしい。

実は以下の考察が成り立つ。

考察 1 (上から降りる方向は無視してよい)

頂点 r を根とする根つき木として見たとき、頂点 v の点火時刻 x_v を

$$x_v = \text{kth}([x_u + D_e \mid \text{頂点 } u \text{ は } v \text{ の子であり, 辺 } e \text{ で隣接している}], C_v)$$

として求めたとき、 r の点火時刻 x_r は真の点火時刻 $x_r^\top$ と等しい。

これにより、子からボトムアップに点火時刻を求める木 DP が正当とわかる。毎クエリで木 DP をおこなっても $O(NQ \log N)$ 時間で、小課題 1 には通る (6 点)。

小課題 1 ($N, Q \leq 2000$) - 木 DP による解法 (補足)

頂点 v に隣接する頂点 u のうち, $x_u^\top + D_{(v,u)}$ が小さいほうから C_v 番目以内であるような u に対して, 辺 (v, u) に向き $u \rightarrow v$ をつける.

補題 1.1

どの辺も高々 1 方向にしか向きがつかない.

(証明概略) 辺 (v, u) に両方の向きがついたとすると, 向き $u \rightarrow v$ から $x_u^\top + D_{(v,u)} \leq x_v^\top$ が, 向き $v \rightarrow u$ から $x_v^\top + D_{(u,v)} \leq x_u^\top$ が成り立つはずだが, $D_{(v,u)} = D_{(u,v)} > 0$ よりこの 2 式は矛盾. よって背理法より示された.

補題 1.2

任意の頂点 u について, $x_u \geq x_u^\top$. 特に, u が根であるか, u の親 v に対して辺 (v, u) が向き $v \rightarrow u$ を持たないようなとき, $x_u = x_u^\top$.

(証明概略) 根つき木の頂点数に対する数学的帰納法が回る.

考察 1 は補題 1.2 の系.

小課題 2 (スターグラフ) - k th の更新を高速に行う

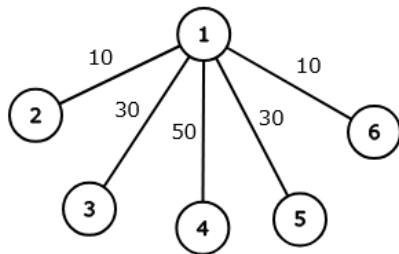
木がスターグラフであり，質問クエリは頂点 1 のみに与えられるときを考える．

多重集合 S に対して以下のことができるデータ構造があればよい．

- 要素の追加・削除 (葉の人気度の変更に相当)，変更 (辺の長さの変更に相当)
- 与えられた k に対する $k\text{th}(S, k)$ の取得 (根の人気度の変更に相当)

これは動的セグメント木上の二分探索や PBDS などによって実現できる．

動的セグメント木による実装でクエリあたり $O(\log A)$ 時間 ($A := \sum_e D_e$) となり，小課題 2 に通る (7 点)．



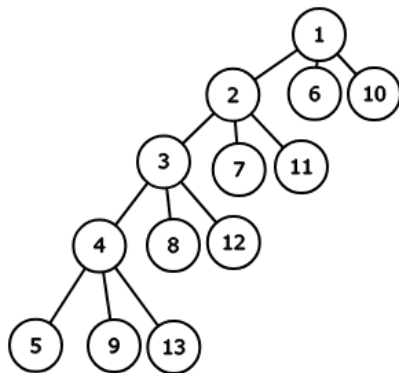
小課題 3 (列のようなグラフ) - 木 DP の高速化

小課題 3 の制約下で、小課題 1 の木 DP を高速化することを考える。

葉の点火時刻は、人気度が 0 なら 0, 人気度が正なら $+\infty$ である。葉 $x_{\frac{N-1}{3}+1}$ から

$x_{\frac{N-1}{3}}, \dots, x_2, x_1$ の順に求めることを考え、それを高速化する。

葉でない頂点 v は 3 個の子 $v+1, v+a, v+b$ をもつ。 $x_{v+1} + D_{(v,v+1)}, x_{v+a} + D_{(v,v+a)}, x_{v+b} + D_{(v,v+b)}$ のうち C_v 番目に小さい値が x_v である。



小課題 3 (列のようなグラフ) - 木 DP の高速化

葉でない頂点 v は 3 個の子 $v+1, v+a, v+b$ をもつ.

$[s_1, s_2] = [x_{v+a} + D_{(v,v+a)}, x_{v+b} + D_{(v,v+b)}]$ ($s_1 \leq s_2$) とすると, x_{v+1}, x_v 間の関係式は以下ようになる.

- $x_{v+1} + D_{(v,v+1)}$ が s_{C_v-1} より小さいとき, $x_v = s_{C_v-1}$.
- $x_{v+1} + D_{(v,v+1)}$ が s_{C_v-1} 以上 s_{C_v-0} 以下のとき, $x_v = x_{v+1} + D_{(v,v+1)}$.
- $x_{v+1} + D_{(v,v+1)}$ が s_{C_v-0} より大きいとき, $x_v = s_{C_v-0}$.

ここで, $s_i = 0$ ($i \leq 0$), $s_i = +\infty$ ($i \geq 3$) とする.

小課題 3 (列のようなグラフ) - 木 DP の高速化

先ほどの 3 本の式を統一的に書くと、以下のようになる。

$$x_v = \min(\max(x_{v+1} + D_{(v,v+1)}, s_{C_v-1}), s_{C_v-0})$$

ここで、 $s_i = 0$ ($i \leq 0$), $s_i = +\infty$ ($i \geq 3$) とする。

この x_{v+1}, x_v 間の関係式を作用 $f_v : x_{v+1} \mapsto x_v$ として見ると、 f_v は $D_{(v,v+1)}$ および s_1, s_2 の値で表される。

求める値 x_1 は、 $f_1(f_2(\cdots f_{\frac{N-1}{3}}(x_{init}) \cdots))$ (x_{init} は $C_{\frac{N-1}{3}+1} = 0$ のとき 0, そうでないとき $+\infty$) によって求まる。

作用素 $(D_{(v,v+1)}, s_1, s_2)$ はモノイドなので、セグメント木に乗せることができる。

したがって、更新クエリは作用素の一点更新に、質問クエリは全体取得に相当し、モノイドどうしの積を適切に実装してセグメント木に乗せることで、小課題 3 に $O(N + Q \log N)$ 時間で通る (14 点)。

小課題 4, 5 (質問クエリは頂点 1 のみ) - 小課題 2, 3 のアイデアを一般の木に拡張

小課題 3 の考え方を一般の木に拡張する.

任意の葉でない頂点 v に対して, 子を 1 つ選んで **heavy** な子として, ほかの子を **light** な子とする. 子側の端点によって, 辺にも heavy/light の区別をつける.

頂点 v の子の個数を $d(v)$ としたとき, light な子 $d(v) - 1$ 個についての $x_w + D_{(v,w)}$ の値を昇順に $s_1, \dots, s_{d(v)-1}$ としたとき, heavy な子 u に対して

$$x_v = \min(\max(x_u + D_{(v,u)}, s_{C_v-1}), s_{C_v-0})$$

が成り立つ. ここで, $s_i = 0$ ($i \leq 0$), $s_i = +\infty$ ($i \geq d(v)$) とする.
上の作用を f_v とおく.

小課題 4, 5 (質問クエリは頂点 1 のみ) - 小課題 2, 3 のアイデアを一般の木に拡張

頂点 v から heavy な子を下方向にたどって通る頂点を $v = h^0(v), h^1(v), \dots, h^k(v)$ とおき、そのうち最後に行きつく葉を $\text{leaf}(v)$ とおく。また、葉 $\text{leaf}(v)$ に対して

$$\text{init}(\text{leaf}(v)) = \begin{cases} 0 & (C_{\text{leaf}(v)} = 0) \\ +\infty & (C_{\text{leaf}(v)} > 0) \end{cases}$$

とおく。このとき、頂点 v の点火時刻 x_v は

$$x_v = f_{h^0(v)}(f_{h^1(v)}(\dots f_{h^{k-1}(v)}(\text{init}(\text{leaf}(v))) \dots))$$

となる。

小課題 4, 5 (質問クエリは頂点 1 のみ) - 小課題 2, 3 のアイデアを一般の木に拡張

ある頂点の作用素は, light な子それぞれの「点火時刻 + 辺の長さ」, および heavy な辺の長さによって決まる. 辺の長さは容易に変更できるので, light な子の点火時刻の更新について考える.

頂点 v に対する変更クエリにおいて, 影響を受ける light な子は v の先祖に限られる. よって, 以下のように反復して, 根に向かって下から対応すればよい. v の初期値は, 変更対象の頂点とする.

- $\text{leaf}(v)$ から上に向かって heavy な辺をたどり続け, 最終的に行きつく頂点 (light な頂点, もしくは根) の点火時刻を求める. その頂点を新たな v とする.
- v が根であれば反復を終了する. v が light な頂点であれば, その親の作用素を更新し, 親を新たな v とする.

辺への変更クエリも同様に処理する.

小課題 4, 5 (質問クエリは頂点 1 のみ) - 小課題 2, 3 のアイデアを一般の木に拡張

heavy な子の選び方において、部分木のサイズが最も大きいような子を heavy な子として選ぶことにする (重軽分解). このとき, 1 回の反復ごとに light な辺を 1 本通り, そのたびに部分木 v のサイズが 2 倍以上になるので, 反復回数は $O(\log N)$ 回で済む. 各反復では作用素の区間取得・一点更新を高々 1 回ずつ行うことになり, これはセグメント木によって $O(\log N)$ 時間でできる.

作用素の更新には, light な子の情報に対する k th の取得が必要になる. これには小課題 2 のアイデア (動的セグメント木上の二分探索や PBDS など) を適用できる.

全体で $O(N \log A + Q \log N (\log N + \log A))$ 時間となるので, 小課題 2,3,4,5 に通る, $7 + 14 + 25 + 12 = 58$ 点を得る.

小課題 6, 7 (満点制約) - Re-rooting

全方位木 DP の要領で、重軽分解上での Re-rooting を行う。

- ① 根 r の点火時刻 x_r を求める。
- ② 求めたい頂点 v に向かって根から heavy パス / light 辺をたどり、真の点火時刻 $x_v^\top$ を求める。

ある辺 (v, u) を下るとき、子頂点 u からさらに heavy 辺を下ることになっている場合は、非 Re-rooting の段階と同様の作用素で書けるので、パスをまとめて処理できる。そうでないときは、 $[s_1, \dots, s_{d(v)-1}]$ に以下の一時変更を加えた多重集合での k th が x_u となる。

- 下るべき light 辺の値を除く。
- heavy 辺の値を加える。

一時変更はクエリあたり $O(\log N)$ 回起こるので、解法全体で $O(N \log A + Q \log N (\log N + \log A))$ 時間となる。これは十分高速に動作し、満点を得ることができる。

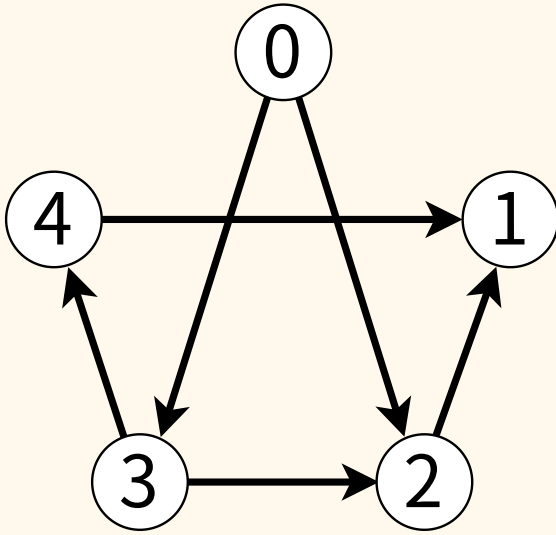
電圧 2 (Voltage 2)

解説担当 : 太田克樹 (shiomusubi496)



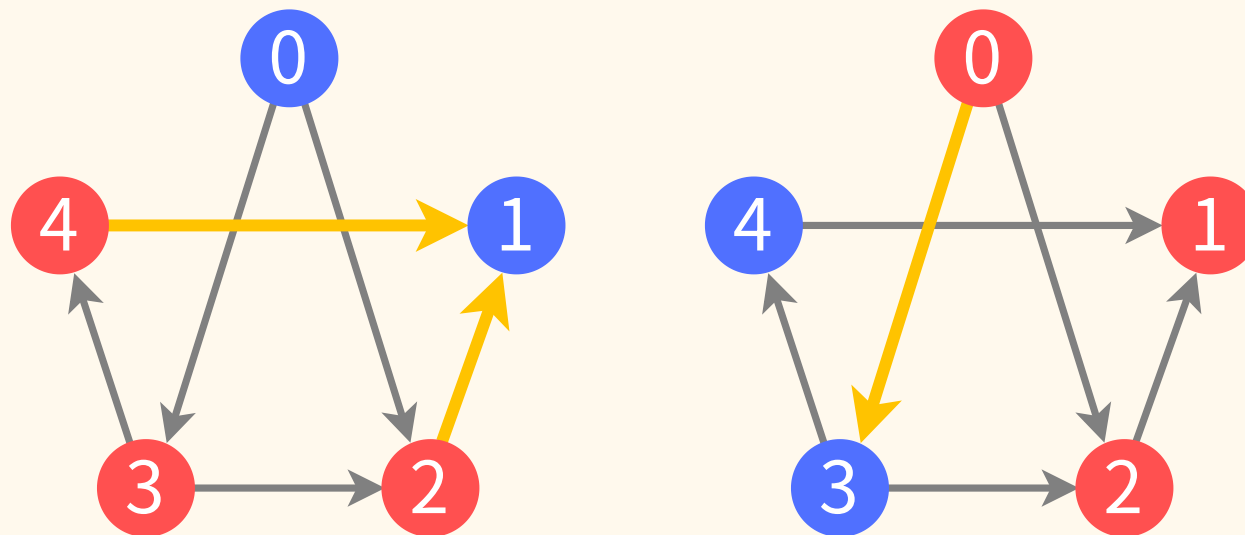
問題概要

問題概要



- N 頂点 M 辺の隠された有向グラフがある
- 30 000 回以下のクエリで全ての辺を特定したい
- クエリ内容は次ページ

問題概要



- 各頂点に高電圧と低電圧を2通りに設定する
- 高電圧から低電圧に向かう辺には電流が流れる
- どちらが電流の流れる辺が多いかを知ることができる

- $2 \leq N \leq 500$
- $1 \leq M \leq 1000$
- 自己ループはない
- 多重辺は向きを無視しても存在しない

小課題

番号	追加制約	配点(JOI)	配点(JOIG)
1	$N = 2$	0	4
2	パスグラフ, $N \leq 100$	10	16
3	パスグラフ	12	21
4	出次数 1, $N \leq 100$	27	24
5	出次数 1	18	14
6	$N \leq 100$	17	11
7	特になし	16	10

- 「電流の流れた辺の本数」は長いので、単に「電流」と呼ぶことにする
 - 「電流が 3 流れた」みたいな言い方をします
- 以降、小課題番号は JOIG に準拠
 - JOI の人は番号から 1 を引いてください

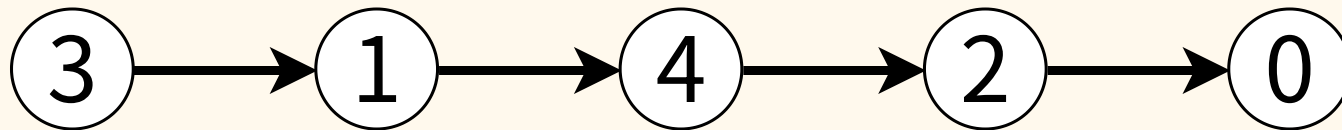
小課題 1 ($N = 2$)

小課題 1 ($N = 2$)

- $1 \leq M$ と多重辺の制約から、答えは 2 通りに限られる
- ふたつの区別ができるクエリを何でもいいので送ろう
- 例えば、全て高電圧だと電流は流れない
- `query([1, 1], [1, 0])` が 1 か 0 かで判別できる

小課題 2,3 (パスグラフ)

小課題 2,3 (パスグラフ)



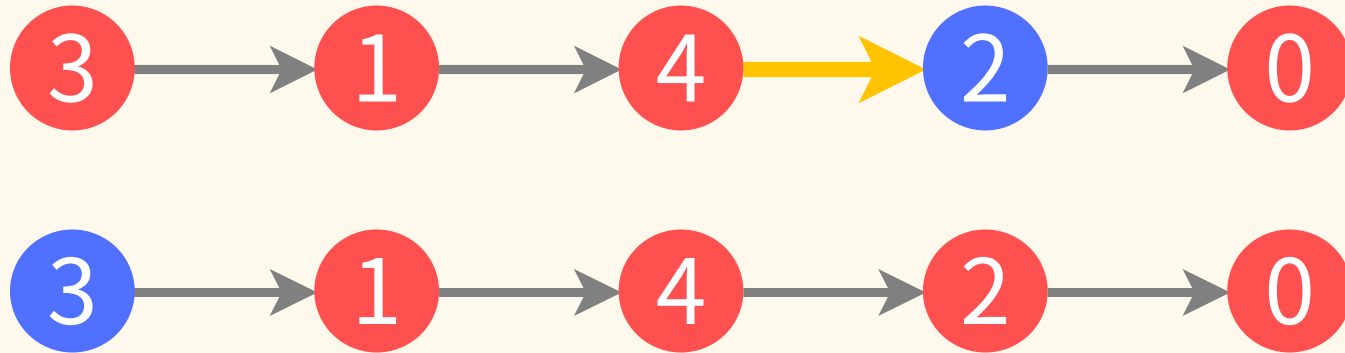
- せっかくパスなので、端点から順番に特定できると嬉しい
- まずは端点を特定する方法を考えてみる

小課題 2,3 (パスグラフ)



- 流れる電流が確実に分かる電圧の設定があると嬉しい
 - 小課題 1 と同じように、全部高電圧にしてみよう
- では、そこから 1 頂点だけ低電圧にするとどうなるか？

小課題 2,3 (パスグラフ)



- 流れる電流は、低電圧にした頂点が、
 - パスの始点なら 0
 - そうでないなら 1

となるので、

全て高電圧のときと比べれば始点かどうか分かる

小課題 2,3 (パスグラフ)

- N 頂点を順に試せば、始点を N 回のクエリで特定できる
- 次は、始点から順番に次の頂点を特定していこう
 - この方法によって小課題 2, 3 が分かれる
- まずは小課題 2 の方から

小課題 2 (パスグラフ)

- 先ほど、始点のみ低電圧で残りは高電圧の設定をした
- 実は、始点を低電圧に固定すると、始点を無視することができる
 - 低電圧から出る辺には絶対に電流が流れないため

小課題 2 (パスグラフ)

- 始点を無視すると $N - 1$ 頂点の小さい問題に帰着できる
- 再帰的に解くことで、最終的に 1 頂点(終点)のみになる
- クエリ $N + (N - 1) + \dots + 1 = \frac{N(N + 1)}{2}$ 回など
- $N \leq 100$ であれば余裕で 30 000 に収まる

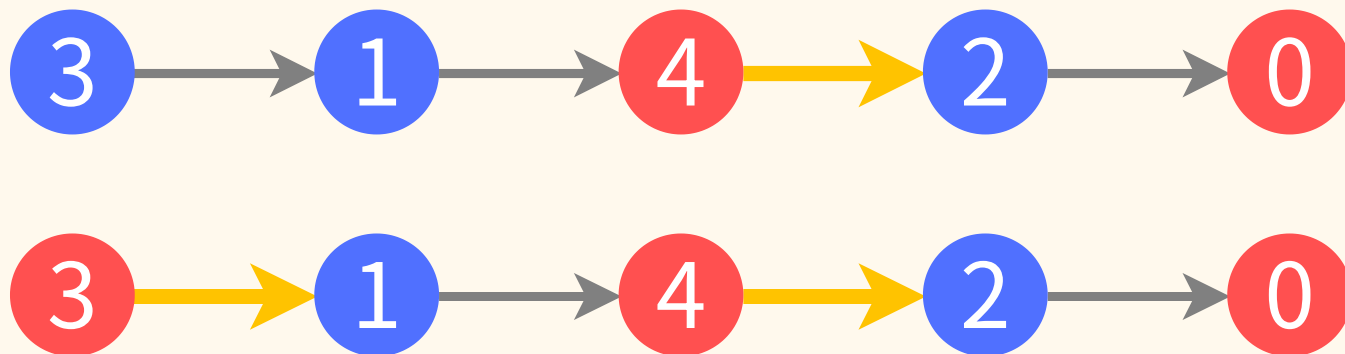
小課題 3 (パスグラフ)

- 次は小課題 3
- $N \leq 500$ だと、 $\Omega(N^2)$ クエリはちょっと厳しそう
- OI Communication では二分探索が典型的
- 始点の次の頂点を見つけるのに二分探索は使えるか？

小課題 3 (パスグラフ)

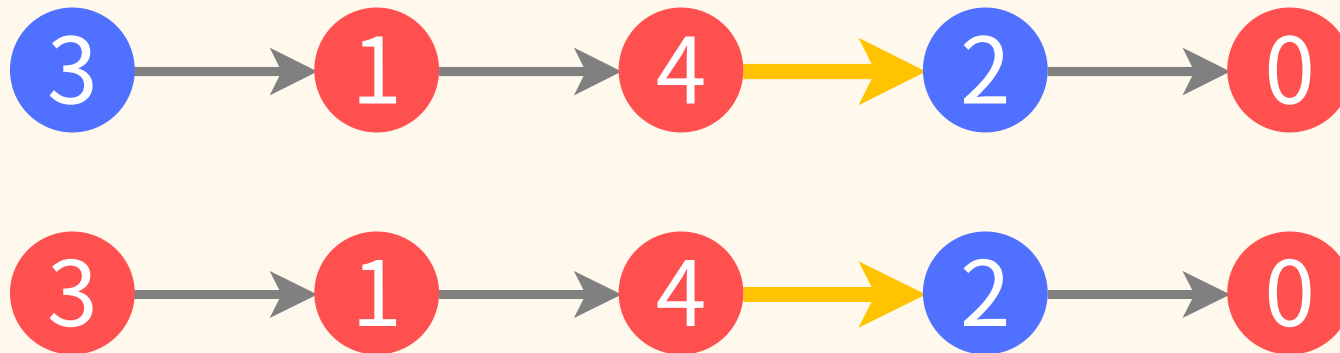
- 二分探索をするには、任意の頂点集合 S に対し、求める頂点があるかどうか分かれば良い
- S の頂点がどうパスに位置しているか分からない
 - 小課題 2 のように S 全体を高電圧/低電圧にするのは嬉しくなさそう
- 逆に、場所が分かっている始点を変化させてみよう

小課題 3 (パスグラフ)



- 始点の次の頂点が低電圧であるとき
 - 始点を低電圧にすると始点から電流は流れない
 - 始点を高電圧にすると始点から電流が流れる
- よって、始点を高電圧にした方が多く電流が流れる

小課題 3 (パスグラフ)



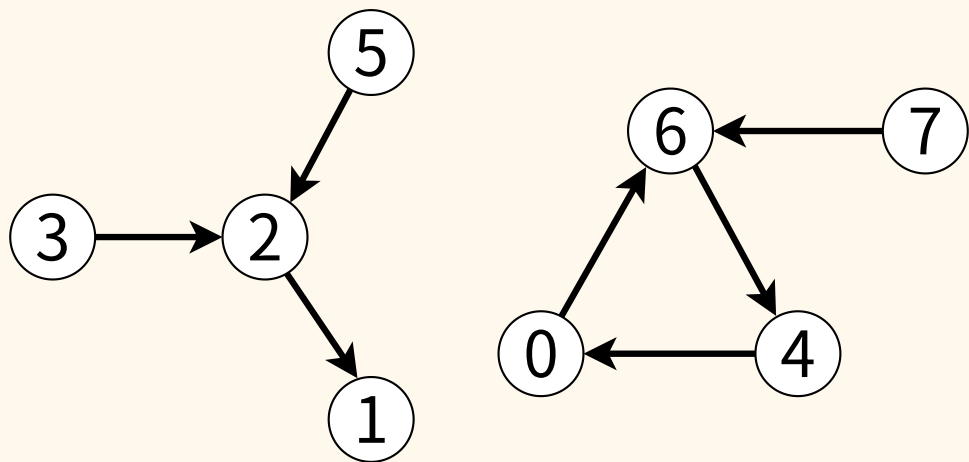
- 始点の次の頂点が高電圧であるとき
 - 始点を低電圧にすると始点から電流は流れない
 - 始点を高電圧にすると始点から電流は流れない
- よって、始点の電圧を変えても電流は変わらない

小課題 3 (パスグラフ)

- 以上より、始点の次の頂点が低電圧か高電圧かを 1 クエリで判定できる
- S を低電圧に、他を高電圧にすれば次の頂点が S に属するかが分かる
- $S = [0, N/2), [N/4, N/2), \dots$ のようにして、二分探索で次の頂点を求められる
- その次の頂点の時には始点を低電圧にして無視すれば良い
- クエリ数は $N + (N - 1) \log_2 N$ など

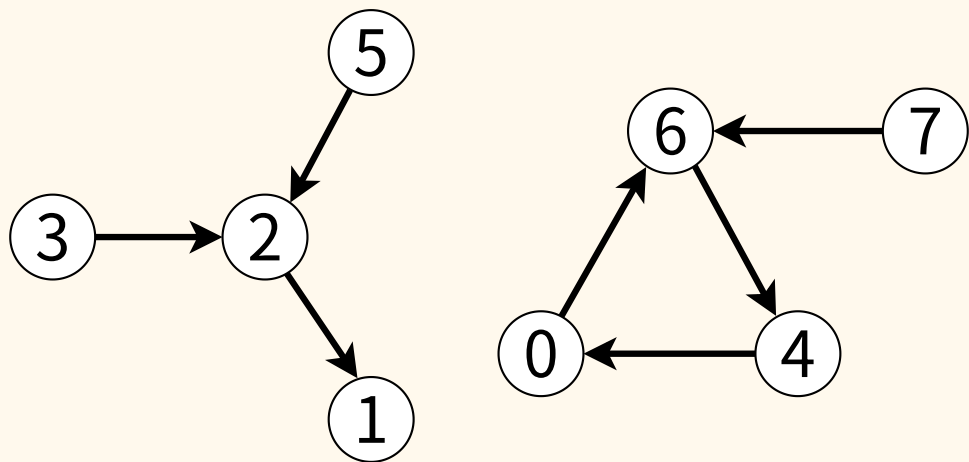
小課題 4,5 (出次数が 1 以下)

小課題 4,5 (出次数が 1 以下)



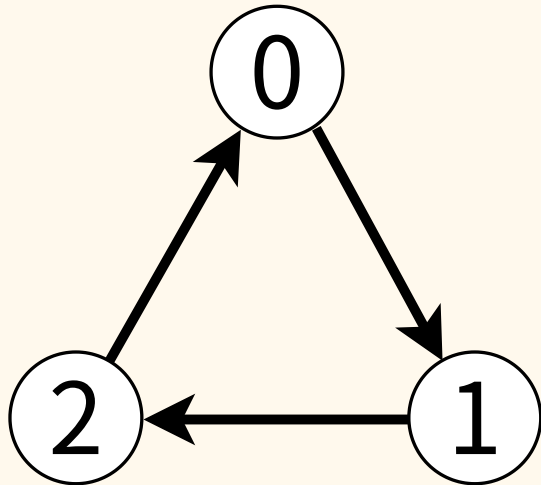
- まずはグラフの形を整理しよう
- だいたい functional graph っぽい形になる
- (弱)連結成分は有向木か functional graph のいずれか
- いちど(弱)連結成分ごとに分けて考えてみよう

小課題 4,5 (出次数が 1 以下)



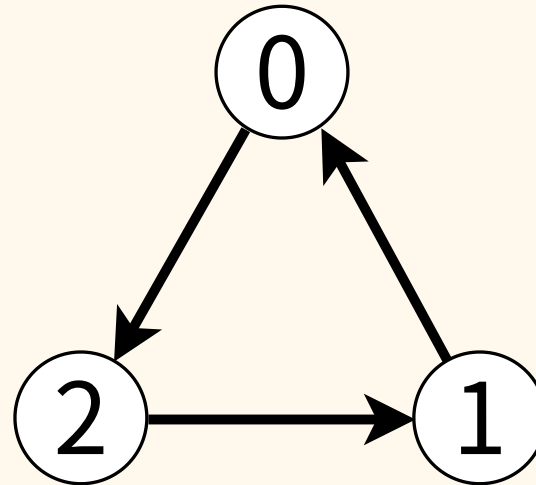
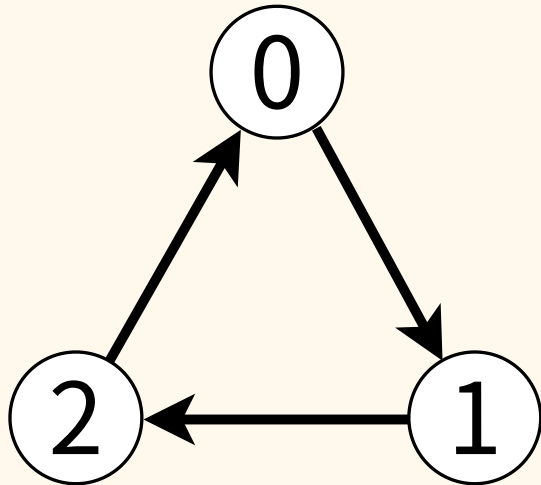
- 有向木はパスの拡張として考えられそう
- 厄介なのは functional graph になっている方
- そういえばパスグラフでは常にグラフが特定可能だった
- ではどういうケースでグラフが特定不可能なのか？

小課題 4,5 (出次数が 1 以下)



- Q. 例えばこういうケースは特定できるか？

小課題 4,5 (出次数が 1 以下)

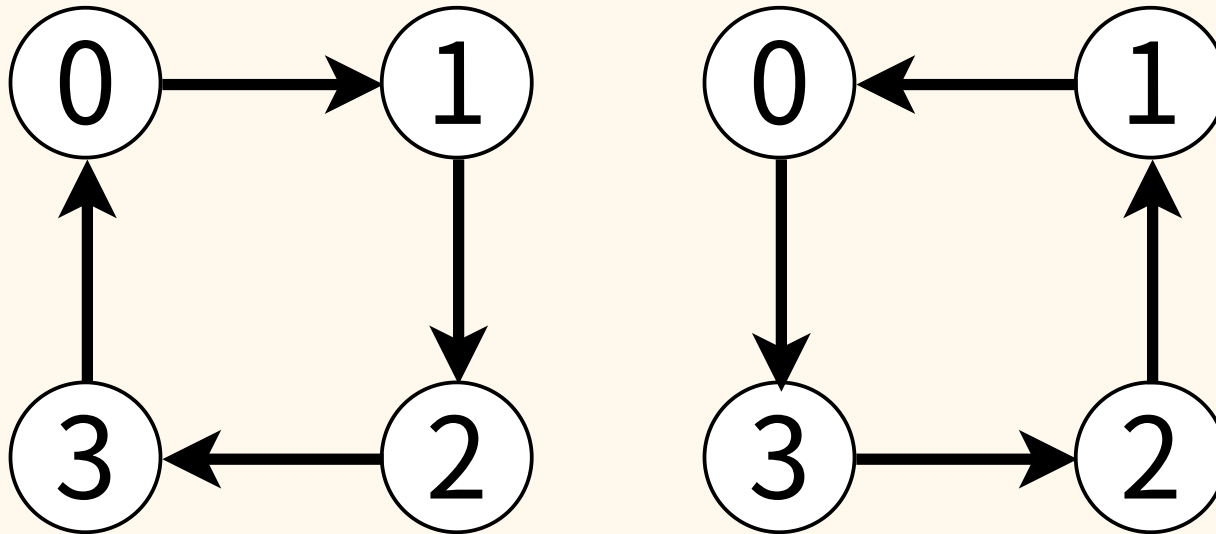


- hint. この二つは区別できるか？

小課題 4,5 (出次数が 1 以下)

- A. できない
- どちらのグラフでも
 - 全て同じ電圧なら電流は流れない
 - 高電圧と低電圧が両方あるなら電流は 1 流れる
- どの電圧設定でも同じ電流が流れるので区別は不可能

小課題 4,5 (出次数が 1 以下)



- Q. この二つは区別できるか？

小課題 4,5 (出次数が 1 以下)

- A. できない
- 一般に、サイクル(閉路)グラフは全て特定不可能
 - 電圧の設定によらず、高電圧から低電圧に向かう辺と低電圧から高電圧に向かう辺が同じ数あるため
 - 制約からサイクルは頂点数 3 以上であることに注意
- より一般に、サイクルを含むグラフは全て特定不可能
 - サイクルの辺を入れ替えたグラフと区別できない
- 先ほどのグラフ(sample 3)は `false` と返すのが正解

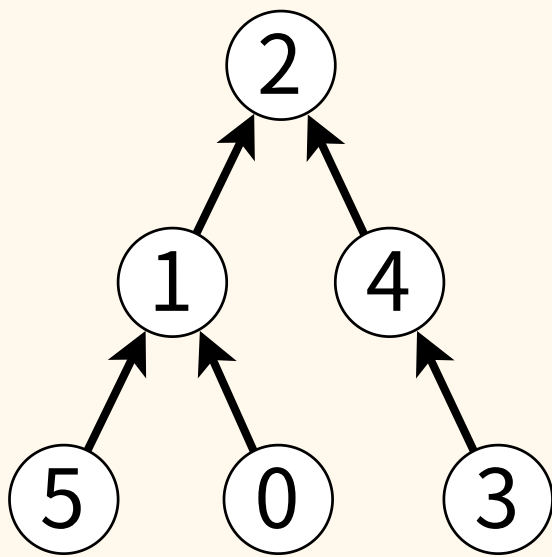
小課題 4,5 (出次数が 1 以下)

- functional graph はサイクルを含む
- よってグラフが特定可能であるためには、
全ての(弱)連結成分が有向木である必要がある
- いったん判定は後回しにして、すべての(弱)連結成分が
有向木であるときの解法を考えてみる

小課題 4,5 (出次数が 1 以下)

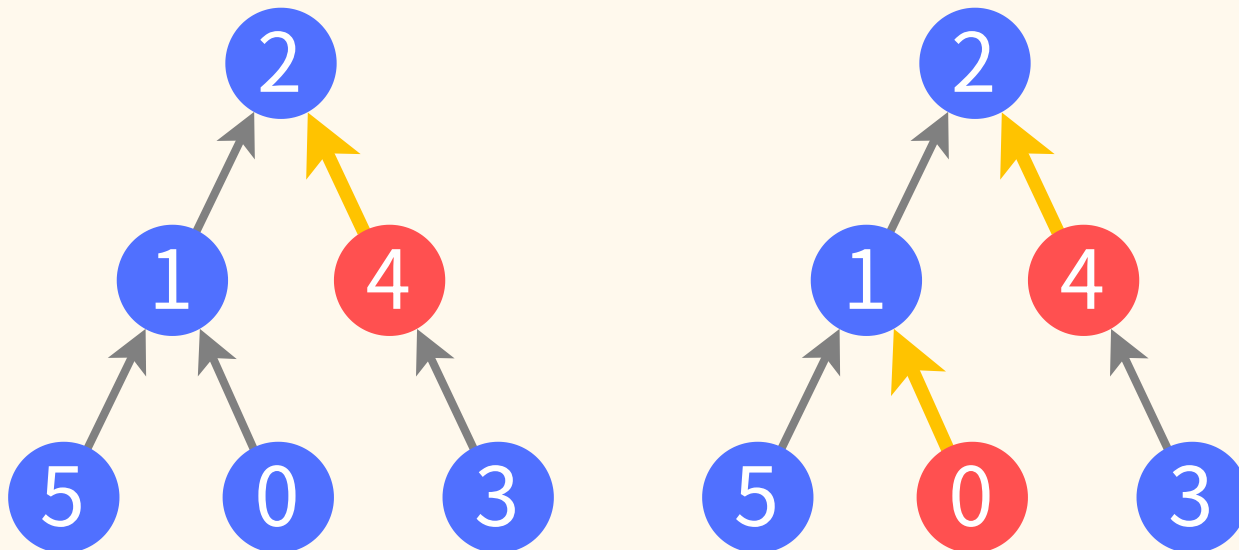
- パスグラフのときと全く同様に、葉(入次数 0 の頂点)は N 回のクエリで全て見つけることができる
- それぞれの始点から辺の向かう先の頂点を求めたい
- 実際、これはパスグラフのときと似た方法でできる

小課題 4,5 (出次数が 1 以下)



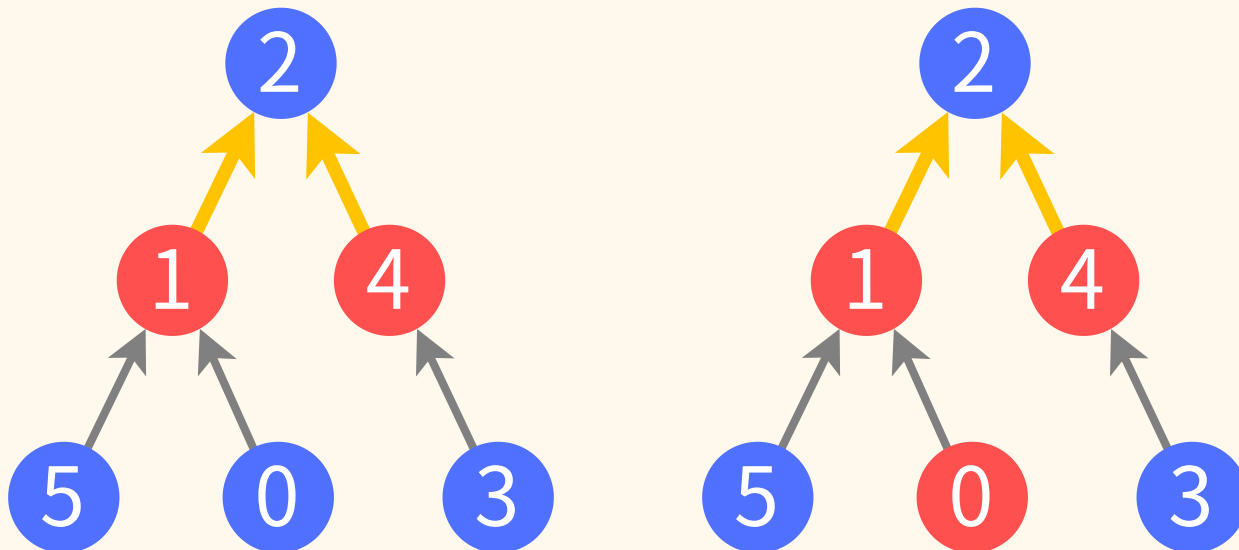
- こういうグラフを考える
- 今、頂点 0, 3, 5 が葉であることは分かっている
- 頂点 0 からどこに辺が出ているかを調べよう
- 頂点 3, 5 は例のごとく **低電圧** に固定すると無視できる

小課題 4,5 (出次数が 1 以下)



- 頂点 0 から辺が出る頂点が存在し、**低電圧**のとき
 - 頂点 0 を**低電圧**にすると頂点 0 から**電流**は流れない
 - 頂点 0 を**高電圧**にすると頂点 0 から**電流**が流れる
- よって、頂点 0 を**高電圧**にした方が多く **電流**が流れる

小課題 4,5 (出次数が 1 以下)



- 頂点 0 から辺が出る頂点が存在しないか、**高電圧**のとき
 - 頂点 0 を**低電圧**にすると頂点 0 から**電流**は流れない
 - 頂点 0 を**高電圧**にすると頂点 0 から**電流**は流れない
- よって、頂点 0 の電圧を変えても**電流**は変わらない

小課題 4,5 (出次数が 1 以下)

- 以上より、頂点 0 の次の頂点が高電圧かどうかを 1 クエリで判定できる
 - 1 頂点ずつ試すと N クエリで次の頂点分かる
 - 二分探索すると $\log_2 N$ クエリで次の頂点分かる
- これを繰り返すと次ページのアルゴリズムが作れる

小課題 4,5 (出次数が 1 以下)

- キュー Q を葉全体の集合で初期化する
- Q が空でない限り繰り返す
 - Q から頂点 v を取り出す
 - v から出る辺 $v \rightarrow u$ を特定する
 - 存在しないときは次のステップを飛ばす
 - u に入る辺を全て見つけ終わっているなら、 u を Q に加える

小課題 4,5 (出次数が 1 以下)

- 最後の行の判定はどうすれば良いか？
- 実は葉の特定とほぼ同じようにできる
- 確認済み(Q から取り出し済み)の頂点を低電圧にすれば葉の特定るときと同様に 1 クエリでできる

小課題 4,5 (出次数が 1 以下)

- ところで、後回しにしていた判定をしなくてはならない
- functional graph に先ほどのアルゴリズムを使うと、サイクル内の頂点が Q に入らないまま終了する
- アルゴリズムが終了したときに全ての頂点を確認したかでサイクルがあるかが判断できる
- サイクルがなければこのアルゴリズムでグラフは特定可能
- これで解けた

小課題 4,5 (出次数が 1 以下)

- クエリ数は、

- 小課題 4 : $N + \frac{N(N-1)}{2} + M$

- 小課題 5 : $N + N \log_2 N + M$

など

- この M はアルゴリズムの最後の行を反映
 - 辺が見つかるたびにチェックするため

小課題 6,7 (追加制約なし)

小課題 6,7 (追加制約なし)

- 小課題 4,5 の解法に既視感がある人も多いはず
- このアルゴリズムはトポロジカルソートをしている
- サイクルを含むグラフで特定不可能であることと整合する
- これを踏まえると一般の有向グラフにも自然に拡張できる
- 必要なのは、頂点 v から出る辺を全て見つける方法

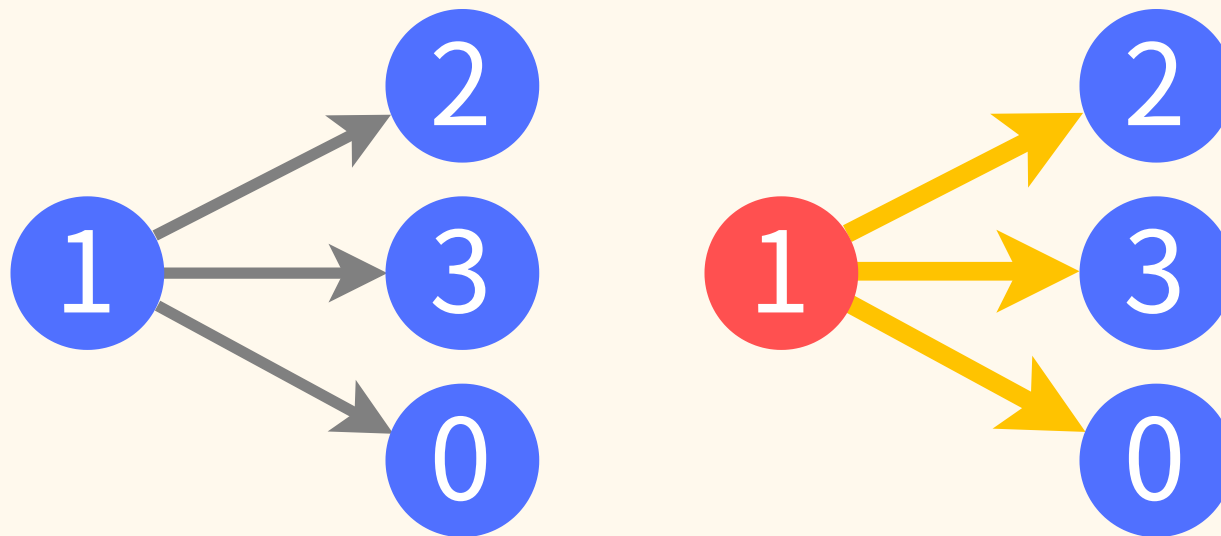
小課題 6 (追加制約なし)

- 小課題 6 の方は(小課題 4 で方針が当たっていれば)簡単
- 小課題 4 では、 N クエリかけて
全ての頂点 u について $v \rightarrow u$ が存在するかを確認した
- 先ほどはこれが高々 1 個なのでそれだけ処理した
- 今度は見つかった全ての辺に対し同じ処理をすれば良い
- クエリ数は変わらず $N + \frac{N(N-1)}{2} + M$ など

小課題 7 (追加制約なし)

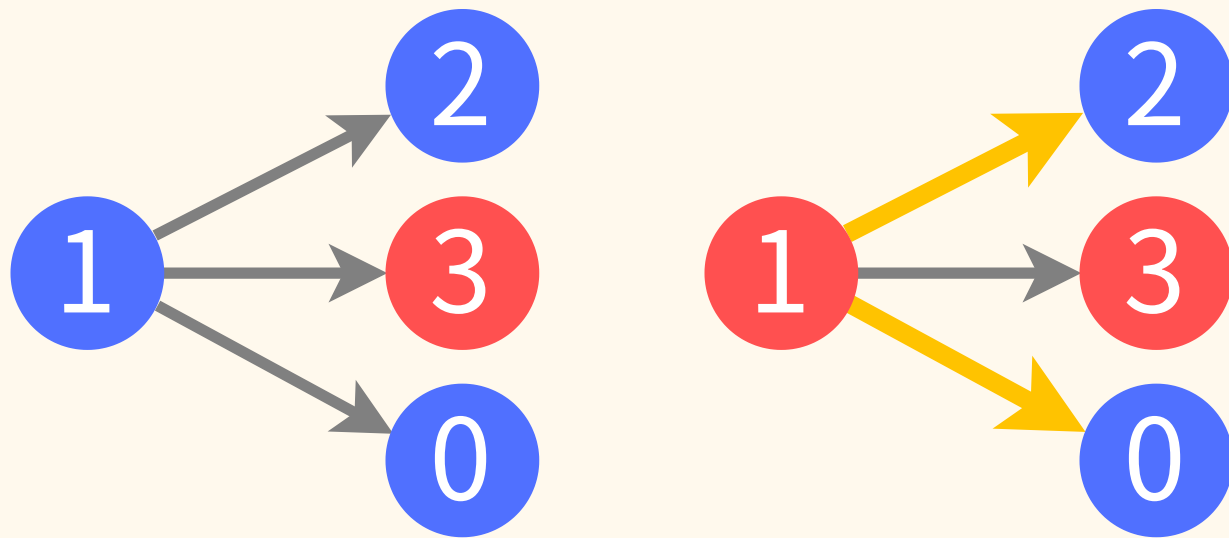
- 小課題 7 の方は小課題 6 より難しい
- 二分探索は正しく機能するだろうか

小課題 7 (追加制約なし)



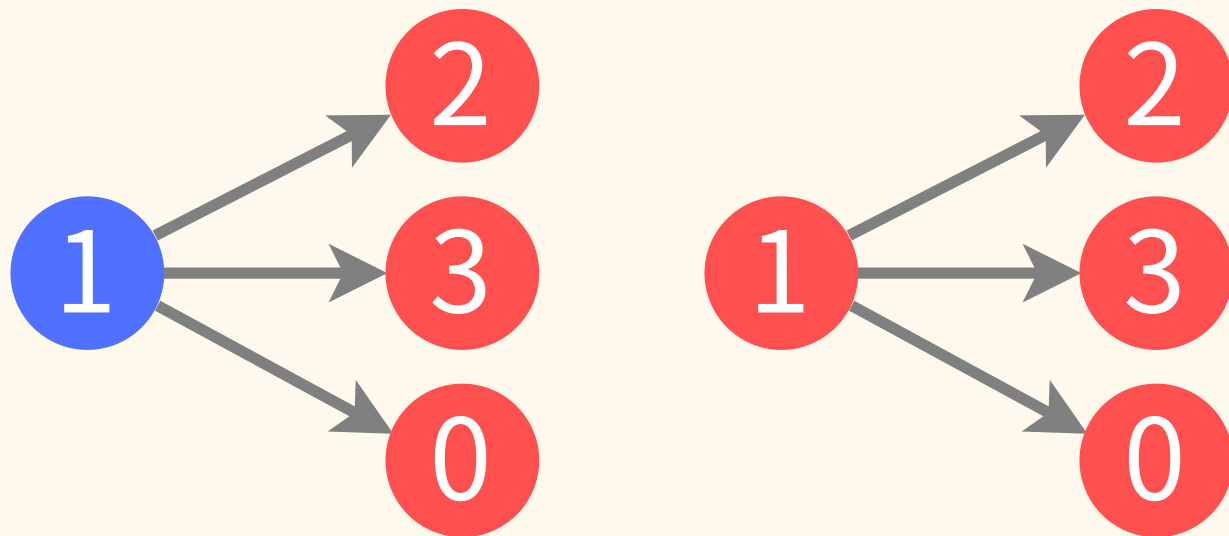
- 頂点 1 の次の頂点がすべて低電圧であるとき
 - 頂点 1 を低電圧にすると頂点 1 から電流は流れない
 - 頂点 1 を高電圧にすると頂点 1 から電流が流れる
- よって、頂点 1 を高電圧にした方が多く電流が流れる

小課題 7 (追加制約なし)



- 頂点 1 の次の頂点に高電圧/低電圧がどちらもあるとき
 - 頂点 1 を低電圧にすると頂点 1 から電流は流れない
 - 頂点 1 を高電圧にすると頂点 1 から電流が流れる
- このときも頂点 1 を高電圧にした方が多く電流が流れる

小課題 7 (追加制約なし)



- 頂点 1 の次の頂点がすべて高電圧であるとき
 - 頂点 1 を低電圧にすると頂点 1 から電流は流れない
 - 頂点 1 を高電圧にすると頂点 1 から電流は流れない
- よって頂点 1 の電圧を変えても電流は変わらない

小課題 7 (追加制約なし)

- 以上の観察から分かる通り、この方法で分かるのは、「頂点 v から出て低電圧の頂点に入る辺があるか」である
- よって、 S の頂点を低電圧に、他を高電圧にすると、 $v \rightarrow u$ が存在する u ($u \in S$) があるかが分かる
- これで二分探索すると、 $v \rightarrow u$ が存在する u のうち頂点番号が最も小さいものが求まる
- u が求まった次は、その u を S に入れなければ良い

小課題 7 (追加制約なし)

- キュー Q を入次数 0 の頂点全体の集合で初期化する
- Q が空でない限り繰り返す
 - Q から頂点 v を取り出す
 - 以下を繰り返す
 - 辺 $v \rightarrow u$ が存在する u のうち最小を求める
 - 存在しないときは `break`
 - u に入る辺を全て見つけ終わっているなら、 u を Q に加える

小課題 7 (追加制約なし)

- クエリ数の解析をする
- 二分探索が行われるのは、各頂点で (出次数)+1 回
- 出次数の合計は M なので、合計で $M + N$ 回
- よって全部で $N + (M + N) \log_2 N + M$ クエリ
 - $N + M(1 + \log_2 N) + N + M$ にもできる

小課題 7 (追加制約なし)

- 以上で満点が得られる

得点分布

得点分布(JOI)

得点	1	2	3	4	5	6	人数
100	o	o	o	o	o	o	3
54	o	x	o	x	o	x	2
49	o	o	o	x	x	x	3
37	o	x	o	x	x	x	1
22	o	o	x	x	x	x	5
10	o	x	x	x	x	x	11
0	x	x	x	x	x	x	5

得点分布(JOIG)

得点	1	2	3	4	5	6	7	人数
20	o	o	x	x	x	x	x	10
4	o	x	x	x	x	x	x	4
0	x	x	x	x	x	x	x	1